

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

03/2026

NOTICE
This document was downloaded from the following site: https://docs.tia.siemens.cloud Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided. Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

 DANGER
indicates that death or severe personal injury will result if proper precautions are not taken.
 WARNING
indicates that death or severe personal injury may result if proper precautions are not taken.
 CAUTION
indicates that minor personal injury can result if proper precautions are not taken.
NOTICE
indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:



WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

03/2026 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1	TIA Portal Openness: API for automation of engineering workflows	15
1.1	Cybersecurity information	15
1.2	What's new in TIA Portal Openness?	16
1.3	Basics	20
1.3.1	Installation	20
1.3.1.1	Requirements for TIA Portal Openness	20
1.3.1.2	Installing TIA Portal Openness	21
1.3.1.3	Adding users to the 'Siemens TIA Openness' user group	21
1.3.1.4	Using Windows Service	23
1.3.1.5	Accessing the TIA Portal	24
1.3.1.6	Configurations	24
1.3.2	Openness tasks	27
1.3.2.1	Introduction	27
1.3.2.2	Applications	28
1.3.2.3	Export/import	29
1.4	TIA Portal Openness API	29
1.4.1	TIA Portal Openness object	29
1.4.1.1	TIA Portal Openness object model	29
1.4.1.2	Blocks and types of the TIA Portal Openness object model	34
1.4.1.3	Hierarchy of hardware objects of the object model	39
1.4.1.4	Object list	41
1.4.2	General functions	47
1.4.2.1	Introduction	48
1.4.2.2	Information about installed TIA Portal Openness versions	48
1.4.2.3	TIA Portal Openness IntelliSense support	49
1.4.2.4	Standard libraries	50
1.4.2.5	Use of the code examples	51
1.4.2.6	Example program	54
1.4.2.7	Programming steps	61
1.4.2.8	Connecting to the TIA Portal	62
1.4.2.9	TIA Portal Openness firewall	66
1.4.2.10	Event handlers	68
1.4.2.11	Program-controlled acknowledgement of dialogs with system events	71
1.4.2.12	Terminating the connection to the TIA Portal	74
1.4.2.13	Diagnostic interfaces on TIA Portal	75
1.4.2.14	Exclusive access	82
1.4.2.15	Dynamic behaviours	84
1.4.2.16	Working with associations	90
1.4.2.17	Working with compositions	90

1.4.2.18	Verifying object equality	91
1.4.2.19	Read operations for attributes	93
1.4.2.20	Transaction handling	94
1.4.2.21	Creating a DirectoryInfo/FileInfo object	100
1.4.2.22	Self-description support for attributes, navigators, actions, and services	104
1.4.2.23	Setting attributes of Blocks, DBs & UDTs	109
1.4.2.24	Notes on performance of TIA Portal Openness	111
1.4.3	Functions for projects and project data	112
1.4.3.1	Opening a project	112
1.4.3.2	Creating a project	118
1.4.3.3	Accessing general settings of the TIA Portal	121
1.4.3.4	Archiving and Retrieving a project	124
1.4.3.5	Accessing read-only TIA Portal project	130
1.4.3.6	Accessing languages	130
1.4.3.7	Determining the object structure and attributes	134
1.4.3.8	Access software target	136
1.4.3.9	Accessing and enumerating multilingual texts	137
1.4.3.10	Updating project properties	139
1.4.3.11	Read project related attributes	140
1.4.3.12	Deleting project graphics	145
1.4.3.13	Compiling a project	145
1.4.3.14	Modifying project in UMAC	149
1.4.3.15	Saving a project	150
1.4.3.16	Closing a project	151
1.4.3.17	Export/Import system diagnostics settings	152
1.4.3.18	Identifying cross session object for Hardware & Software	154
1.4.3.19	Managing exception for missing component in project	157
1.4.3.20	Accessing multilingual support	158
1.4.4	Functions for accessing Teamcenter Gateway	164
1.4.4.1	Connecting to the Teamcenter Gateway	164
1.4.4.2	Disconnecting to the Teamcenter Gateway	167
1.4.4.3	Connecting to the ConnectSSO	169
1.4.4.4	Saving Project/Global library in Teamcenter	171
1.4.4.5	Saving Project/Global library with proxy object	174
1.4.4.6	Saving Project/Global library with new item	177
1.4.4.7	Accessing custom attributes in Teamcenter	180
1.4.4.8	Saving Project to Teamcenter as new item	184
1.4.4.9	Saving project using save to item with proxy object	189
1.4.4.10	Saving Project as new item with proxy object	192
1.4.4.11	Saving project using save as new revision with proxy object	197
1.4.4.12	Saving Project to Teamcenter as new revision	199
1.4.4.13	Downloading Project/Global library from Teamcenter	202
1.4.4.14	Searching Project / Global library from Teamcenter	205
1.4.4.15	Checkout Project/Global library dataset in Teamcenter	208
1.4.4.16	Checkin Project/Global library dataset in Teamcenter	211
1.4.4.17	Cancel checkout dataset in Teamcenter	215

1.4.5	Functions for Connections	218
1.4.5.1	Configurable attributes of a port-to-port connection	218
1.4.6	Functions on libraries	223
1.4.6.1	Functions for objects and instances	223
1.4.6.2	Accessing global libraries	224
1.4.6.3	Accessing library base type	228
1.4.6.4	Accessing global library languages	229
1.4.6.5	Opening libraries	232
1.4.6.6	Comparing libraries	233
1.4.6.7	Enumerating open libraries	238
1.4.6.8	Saving and closing libraries	239
1.4.6.9	Archiving and retrieving a library	240
1.4.6.10	Creating global libraries	244
1.4.6.11	Accessing folders in a library	245
1.4.6.12	Accessing types	250
1.4.6.13	Accessing type versions	253
1.4.6.14	Accessing blocks located in library	261
1.4.6.15	Accessing instances	264
1.4.6.16	Accessing master copies	266
1.4.6.17	Create master copy from a project in library	269
1.4.6.18	Cleaning up library	271
1.4.6.19	Create an object from a master copy	0
1.4.6.20	Copying master copies	0
1.4.6.21	Creating new type from document	0
1.4.6.22	Updating type from document	0
1.4.6.23	Determining out-of-date type instances	0
1.4.6.24	Updating the project	0
1.4.6.25	Updating a library	0
1.4.6.26	Cleaning up library	0
1.4.6.27	Harmonize project from project library	0
1.4.6.28	Updating Structure during Library Update	0
1.4.6.29	Updating libraries type	0
1.4.6.30	Deleting library content	0
1.4.6.31	Setting default version of a type	0
1.4.6.32	Getting default version of a type	0
1.4.6.33	Improved types consistency state	0
1.4.6.34	Editing library type version	0
1.4.6.35	Discarding library type version	0
1.4.6.36	Managing alarm text list in PLC and Mastercopy library	0
1.4.6.37	Managing DoNotUse in Library	0
1.4.6.38	Exporting library type version as document	0
1.4.6.39	Retrieving supported export formats for library	0
1.4.6.40	Releasing library type version	0
1.4.7	Functions for accessing devices, networks and connections	0
1.4.7.1	Open the 'Devices & networks' editor	0
1.4.7.2	Querying PLC and HMI targets	0

1.4.7.3	Accessing attributes of an address object	0
1.4.7.4	Accessing the channels of a module	0
1.4.8	Functions on networks	0
1.4.8.1	Creating a subnet	0
1.4.8.2	Accessing subnets	0
1.4.8.3	Accessing internal subnets	0
1.4.8.4	Get type identifier of subnets	0
1.4.8.5	Accessing attributes of a subnet	0
1.4.8.6	Deleting a global subnet	0
1.4.8.7	Enumerate all participants of a subnet	0
1.4.8.8	Enumerate IO systems of a subnet	0
1.4.8.9	Accessing nodes	0
1.4.8.10	Accessing attributes of a node	0
1.4.8.11	Connecting a node to a subnet	0
1.4.8.12	Disconnect a node from a subnet	0
1.4.8.13	Creating an IO system	0
1.4.8.14	Accessing the attributes of an IO system	0
1.4.8.15	Connecting an IO connector to an IO system	0
1.4.8.16	Get master system or IO system of an interface	0
1.4.8.17	Get an IO Controller	0
1.4.8.18	Get an IO Connector	0
1.4.8.19	Disconnecting an IO connector from an IO system or a DP mastersystem	0
1.4.8.20	Accessing attributes of a DP mastersystem	0
1.4.8.21	Accessing attributes of PN Driver	0
1.4.8.22	Accessing attributes of a profinet io system	0
1.4.8.23	Deleting a DP master system	0
1.4.8.24	Deleting a profinet io system	0
1.4.8.25	Creating a DP master system	0
1.4.8.26	Accessing port interconnection information of port device item	0
1.4.8.27	Attributes of port inter-connection	0
1.4.8.28	Accessing the attributes of a port	0
1.4.8.29	Enumerate DP master systems of a subnet	0
1.4.8.30	Enumerate assigned IO connectors	0
1.4.8.31	Connecting a DP IO connector to a DP master system	0
1.4.8.32	Accessing AS-i profile and parameter attributes for virtual slaves	0
1.4.8.33	Accessing CM DP as DP slave and transfer area	0
1.4.8.34	Coupling 1516 isochron central and decentral Profinet	0
1.4.8.35	Openness for CP 1604/CP 1616/CP 1626	0
1.4.8.36	Openness transfer areas for PnP coupler	0
1.4.8.37	Openness virtual modules/submodules for ET 200SP PN HF	0
1.4.8.38	Accessing domain settings	0
1.4.8.39	Accessing redundancy mode attribute	0
1.4.8.40	Creating CCDX transfer area	0
1.4.8.41	Deleting CCDX transfer area	0
1.4.8.42	Modifying CCDX transfer areas configuration	0
1.4.9	Functions on devices	0

1.4.9.1	Mandatory attributes of devices	0
1.4.9.2	Get type identifier of devices and device items	0
1.4.9.3	Set App ID on device and device Items	0
1.4.9.4	Get App ID on device and device Items	0
1.4.9.5	Remove App ID on device and device items	0
1.4.9.6	Creating a device	0
1.4.9.7	Enumerating devices	0
1.4.9.8	Accessing devices	0
1.4.9.9	Accessing the TIA Portal hardware catalog	0
1.4.9.10	Deleting a device	0
1.4.9.11	Bulk changing hardware parameters	0
1.4.10	Functions on device items	0
1.4.10.1	Mandatory attributes of device items	0
1.4.10.2	Creating and plugging a device item	0
1.4.10.3	Moving device items into another slot	0
1.4.10.4	Copying a device item	0
1.4.10.5	Setting user defined logo settings	0
1.4.10.6	Deleting a device item	0
1.4.10.7	Enumerate device items	0
1.4.10.8	Accessing device items	0
1.4.10.9	Accessing device item as interface	0
1.4.10.10	Accessing change device	0
1.4.10.11	Accessing attributes of an I/O device interface	0
1.4.10.12	Getting subnet of device item	0
1.4.10.13	Accessing attributes of IoController	0
1.4.10.14	Accessing attributes of IoConnector	0
1.4.10.15	Accessing address controller	0
1.4.10.16	Accessing addresses	0
1.4.10.17	Accessing hardware identifiers	0
1.4.10.18	Accessing hardware identifier controller	0
1.4.10.19	Accessing channels of device items	0
1.4.10.20	Accessing normalized type identifiers	0
1.4.10.21	Creating and exporting psc file	0
1.4.10.22	Connection handling for extension racks	0
1.4.10.23	Telecontrol management datapoint	0
1.4.11	Functions for accessing the data of a PLC device	0
1.4.11.1	Functions for downloading data to PLC device	0
1.4.11.2	Functions for accessing PLC service	0
1.4.11.3	Blocks	0
1.4.11.4	Tags and Tag tables	0
1.4.11.5	Functions for software units	0
1.4.11.6	Functions for fail-safe unit	0
1.4.12	Functions for accessing the data of an HMI device	0
1.4.12.1	Screens	0
1.4.12.2	Cycles	0
1.4.12.3	Text lists	0

1.4.12.4	Graphic lists	0
1.4.12.5	Connections	0
1.4.12.6	Tag table	0
1.4.12.7	VB scripts	0
1.4.13	Functions for Version Control Interface	0
1.4.13.1	Accessing VCI system group in project	0
1.4.13.2	Enumerating user groups in VCI group	0
1.4.13.3	Accessing VCI Workspace with library properties	0
1.4.13.4	Enumerating VCI mapped objects in VCI workspace	0
1.4.13.5	Creating VCI user group in VCI group	0
1.4.13.6	Connecting object	0
1.4.13.7	Updating VCI group properties	0
1.4.13.8	Deleting VCI user group	0
1.4.13.9	Enumerating VCI workspaces in VCI group	0
1.4.13.10	Creating VCI workspace in VCI group	0
1.4.13.11	Updating VCI workspace properties	0
1.4.13.12	Deleting VCI workspace	0
1.4.13.13	Exporting individual object to workspace	0
1.4.13.14	Getting supported file formats	0
1.4.13.15	Getting VCI workspace mapping properties	0
1.4.13.16	Deleting mapped object	0
1.4.13.17	Synchronizing mapped object	0
1.4.13.18	Getting status of mapped object	0
1.4.13.19	Getting child status of a mapped hierarchical object	0
1.4.13.20	Accessing synchronization status of child objects	0
1.4.13.21	Creating VCI workspace with workspace language	0
1.4.13.22	Initializing VCI object status	0
1.4.13.23	Importing Simatic ML with inactive culture	0
1.4.14	Functions support for Multiuser	0
1.4.14.1	Creating project server connection	0
1.4.14.2	Adding project to server	0
1.4.14.3	Editing project server connection	0
1.4.14.4	Deleting project server connection	0
1.4.14.5	Getting project server connection from TIA Portal Setting	0
1.4.14.6	Getting project server group	0
1.4.14.7	Adding a project to server	0
1.4.14.8	Getting server projects	0
1.4.14.9	Getting available local sessions	0
1.4.14.10	Creating a local session	0
1.4.14.11	Saving a local session	0
1.4.14.12	Deleting local session from server	0
1.4.14.13	Opening local/exclusive session	0
1.4.14.14	Opening server project	0
1.4.14.15	Checking marked objects	0
1.4.14.16	Closing local session	0
1.4.14.17	Committing server project changes	0

1.4.14.18	Getting lockstate provider	0
1.4.14.19	Checking project locked	0
1.4.14.20	Getting lock owner	0
1.4.14.21	Checking session UptoDate	0
1.4.14.22	Getting markings from local session	0
1.4.15	Functions for accessing data of the user management	0
1.4.15.1	Functions for accessing user, roles and function rights	0
1.4.15.2	Functions for UMAC Global Users and UMC Server	0
1.4.16	Functions on OPC	0
1.4.16.1	Configuring OPC UA server secure communication protocol	0
1.4.16.2	Setting OPC UA security policy	0
1.4.16.3	Accessing OPC UA reference namespace	0
1.4.16.4	Accessing OPC UA server (SIMATIC) interface	0
1.4.16.5	Accessing OPC UA server interface	0
1.4.17	Functions for SIMATIC Drive Controller	0
1.4.17.1	Accessing PLC and X142 channel properties	0
1.4.17.2	Accessing PROFIdrive Integrated properties	0
1.4.17.3	Creating SINAMICS Integrated	0
1.4.17.4	Getting address of central module (X142)	0
1.4.17.5	Getting address of drive controller telegram	0
1.4.18	Exceptions	0
1.4.18.1	Handling exceptions	0
1.4.19	F-related Openness	0
1.4.19.1	F-related Openness	0
1.4.19.2	SafetyBaselProvider	0
1.4.19.3	SafetyModificationsPossible	0
1.4.19.4	UsernameForFChangeHistory	0
1.4.19.5	F-Signatures	0
1.4.19.6	SafetySignatureProvider	0
1.4.19.7	SafetyAdministration	0
1.4.19.8	SafetyPrintout	0
1.4.19.9	Hardware attributes	0
1.4.20	Functions for Safety Validation Assistant	0
1.4.20.1	Accessing Safety Validation Assistant	0
1.4.20.2	Querying the Activation Tests	0
1.4.20.3	Querying the Evaluation Devices	0
1.4.20.4	Creating new Activation Tests	0
1.4.20.5	Duplicating Activation Tests	0
1.4.20.6	Copying Activation Tests from a library	0
1.4.20.7	Querying Available Devices of the Activation Test	0
1.4.20.8	Modifying Activation Tests	0
1.4.20.9	Overall state of the Activation Tests	0
1.4.20.10	Validity of the Activation Tests	0
1.4.20.11	Generating Activation Test reports	0
1.4.20.12	Exporting Activation Tests	0
1.4.20.13	Importing Activation Tests	0

1.4.20.14	Querying the Safety Functions	0
1.4.20.15	Creating new Safety Functions	0
1.4.20.16	Duplicating Safety Functions	0
1.4.20.17	Resetting Test Results of Safety Functions	0
1.4.20.18	Modifying Safety Functions	0
1.4.20.19	State of the Safety Functions	0
1.4.20.20	Validity of the Safety Functions	0
1.4.20.21	Exporting Safety Functions	0
1.4.20.22	Importing Safety Functions	0
1.4.20.23	Querying the Conditions	0
1.4.20.24	Creating new Conditions	0
1.4.20.25	Validity of the Conditions	0
1.4.20.26	Getting the Trace Configuration	0
1.4.20.27	Modifying Trace Configuration	0
1.4.20.28	Validity of the Trace Configuration	0
1.5	Export/import	0
1.5.1	Overview	0
1.5.1.1	Basic principles of importing/exporting	0
1.5.1.2	Field of application for Import/Export	0
1.5.1.3	Version Specific Simatic ML Import	0
1.5.1.4	Editing the XML file	0
1.5.1.5	Exporting configuration data	0
1.5.1.6	Importing configuration data	0
1.5.2	Import/export of project data	0
1.5.2.1	Project graphics	0
1.5.2.2	Project texts	0
1.5.3	Importing/exporting data of an HMI device	0
1.5.3.1	Importing/exporting data of an HMI device	0
1.5.3.2	Data structure for import/export	0
1.5.3.3	Tag tables	0
1.5.3.4	VB scripts	0
1.5.3.5	Text lists	0
1.5.3.6	Graphic lists	0
1.5.3.7	Connections	0
1.5.3.8	Screens	0
1.5.4	Importing/exporting data of a PLC device	0
1.5.4.1	Blocks	0
1.5.4.2	Tag tables	0
1.5.5	Importing/exporting hardware data	0
1.5.5.1	AML file format	0
1.5.5.2	Pruned AML	0
1.5.5.3	Overview of the objects and parameters of the CAx import/export	0
1.5.5.4	Structure of the CAx data for importing/exporting	0
1.5.5.5	AML type identifiers	0
1.5.5.6	Export/Import of base unit Information via AML	0
1.5.5.7	Export/Import AML with extension rack connection	0

1.5.5.8	Export/import AML file with GSD/GSDML custom attributes	0
1.5.5.9	Export/Import AML file with non-GSD/GSDML custom attributes	0
1.5.5.10	Export/import AML file with pluggable port configuration	0
1.5.5.11	Export/Import AML file with IO Link	0
1.5.5.12	Connection handling for extension racks	0
1.5.5.13	Export/Import AML file with complex tag configuration	0
1.5.5.14	Export of CAx data	0
1.5.5.15	Import of CAx data	0
1.5.5.16	Export/Import of sub modules	0
1.5.5.17	Export/Import AML file in UMAC environment	0
1.5.5.18	Export/Import AML file with normalized type identifier	0
1.5.5.19	Import CAx data without logical address	0
1.5.5.20	Exceptions during import and export of CAx data	0
1.5.5.21	Round trip exchange of devices and modules	0
1.5.5.22	Export/Import topology	0
1.5.5.23	Import of device with library references	0
1.5.5.24	Export of a device object	0
1.5.5.25	Import of a device object	0
1.5.5.26	Export/Import of device with set address	0
1.5.5.27	Export/Import of device with channels	0
1.5.5.28	Export of device item objects	0
1.5.5.29	Import of device item objects	0
1.5.5.30	Export/Import of GSD/GSDML based devices and device items	0
1.5.5.31	Export/Import device configuration with virtual interface	0
1.5.5.32	Export/Import of subnets	0
1.5.5.33	Export/Import of IO-systems	0
1.5.5.34	Export/Import of multilingual comments	0
1.5.5.35	Export/Import ET200 SP/ET200 AL devices with extension rack connections	0
1.5.5.36	Export/Import of PLC tags	0
1.5.5.37	Export/Import of RH/PLC	0
1.5.5.38	Export/Import AML with customized tag and deviceitems	0
1.5.5.39	Export/Import of FailSafe PLC	0
1.5.5.40	Export/Import of Failsafe IO	0
1.5.5.41	Export/Import vendor specific attribute	0
1.5.5.42	AML attributes versus TIA Portal Openness attributes	0
1.6	Major Changes	0
1.6.1	Major changes for long-term stability in TIA Portal Openness V20	0
1.6.2	Major changes for long-term stability in TIA Portal Openness V19	0
1.6.3	Major changes for long-term stability in TIA Portal Openness V18	0
1.6.4	Major changes for long-term stability in TIA Portal Openness V17	0
1.6.5	Major changes in TIA Portal Openness V16	0
1.6.6	Major changes in TIA Portal Openness V15.1	0
1.6.7	Major changes in TIA Portal Openness V15	0
1.6.8	Major changes in V14 SP1	0
1.6.8.1	Major changes in V14 SP1	0
1.6.8.2	Major changes in the object model	0

1.6.8.3	Changes on pilot functionality	0
1.6.8.4	Changes for export and import	0
1.6.9	Major changes in V14	0
1.6.9.1	Major changes of the object model	0
1.6.9.2	Before updating an application to TIA Portal Openness V14	0
1.6.9.3	Major string changes	0
1.6.9.4	Import of files generated with TIA Portal Openness V13 SP1 and previous	0

TIA Portal Openness: API for automation of engineering workflows 1

This section contains information on the following topics:

- [Cybersecurity information](#)
- Readme TIA Portal Openness
- [What's new in TIA Portal Openness?](#)
- [Basics](#)
- [TIA Portal Openness API](#)
- [Export/import](#)
- [Major Changes](#)

1.1 Cybersecurity information

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions only form one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit

<https://www.siemens.com/global/en/products/automation/topic-areas/industrial-cybersecurity.html>

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer

supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under

<https://new.siemens.com/global/en/products/services/cert.html>

1.2 What's new in TIA Portal Openness?

Compatibility and long term stability

Siemens.Engineering libraries

The TIA Portal Openness architecture V21 has been extensively redesigned for seamless integration and long-term stability. The result is new modular TIA Portal Openness libraries with simplified assembly management.

Because the Siemens.Engineering libraries of V17, V18, V19, V20 are not included in the scope of delivery, applications and Add-Ins based on V17, V18, V19, V20 must be adapted to run in TIA Portal V21.

The new Siemens.Engineering libraries can be found in the installation directory under "PublicAPI\V21\net48". For example, the V21 base library can be found in "C:\Program Files\Siemens\Automation\Portal V21\PublicAPI\V21\net48\Siemens.Engineering.Base.dll".

See also: Major changes for long-term stability in TIA Portal Openness V21

Exporting SimaticML files

The Siemens.Engineering libraries in the TIA Portal V21 installation will create SimaticML files of engineering version V21.

Importing SimaticML files

The Siemens.Engineering libraries in the TIA Portal V21 installation can import SimaticML files from engineering versions V18, V19, V20, and V21.

New features

The following new features and innovations are available in TIA Portal Openness V21. For more details on the various topics, refer to the relevant sections of the product documentation.

- General
 - TIA Portal Openness is no longer a setup option but an inherent feature of TIA Portal
 - Improved setup and update performance of products and options
 - Extensive redesign of the TIA Portal Openness architecture with new modular Openness libraries; see also: Major changes for long-term stability in TIA Portal Openness V21
 - Simplified assembly resolving dependent assemblies
 - Version-independent AllowList for the TIA Portal Openness firewall
 - Extended self-description support of APIs

- Improved formatting of SimaticML
- TIA Portal Add-Ins
 - New modular Openness libraries; see also: Major changes for long-term stability in TIA Portal Openness V21
 - New types of TIA Portal Add-Ins: Corporate Add-Ins, System Add-Ins, and User Add-Ins
 - Create shared .NET class libraries for Openness and Add-Ins with uniformed Openness libraries
 - Possibility to get and attach to other TIA Portal processes
- Projects
 - Device specific export of project texts with category filter
- Libraries
 - Use master copies and library types for PLC documents such as Named value types (NVT)
- Hardware configuration
 - Create and manage communication connections (FDL, HMI, ISO, ISO-on-TCP, PtP, S7, TCP, UDP)
 - Features for hardware configuration in run (CiR)
 - Export PN-GSD file of an I-Device
 - Manage CP / TIM TeleControl data points
 - Configure project integrated shared device
 - Query status of GSDX signature for GSD devices
- Security
 - Support additional confidential configuration data protection settings of a PLC
- Hardware modules
 - Hardware parameter access for S7-1200 G2 (Standard)
 - Bulk operation support for Safety module parameters: ET200 SP Safety, ET200 AL Safety, ET200pro Safety, ET200eco PN Safety, ET200MP Safety
 - Access to further hardware parameters of SCALANCE
 - Resource configuration and export of configuration file for S7-1500 Software Controller (V30.0)
- CAx data exchange
 - Release of CAx Publication Tools V4.0 for hardware data management compatible with TIA Portal V21
- Online scenarios
 - Verify certificate details of a secure connection in online event handlers
 - Check secure communication state for passwords in online event handlers

- Go online on a protected PLC
- Go online on a PLC by specifying the IP address
- Go online on a primary or backup R/H PLC by specifying the IP address
- Speedup of reading online fingerprints of a PLC for quick station compare
- Improved error message in case of issues with a secure connection
- Download additional user files to PLC
- Station Upload
 - Continue upload device as new station for a PLC if a PROFINET IO device cannot be uploaded
- PLC user programs
 - Extended new text-based file exchange format for PLC blocks for programming language LAD, Safety-LAD, FBD, Safety-FBD, SCL and blocks with mixed languages as well as data blocks and PLC data types
 - Create snapshot of actual values for a data block
 - Load start values as actual values for a data block
 - Load snapshot as actual values for a data block
 - Read and write further block attributes such as for information, compilation, and download
 - Rename groups and objects in PLC software
 - Manage write protection of blocks
 - Access multilingual title and comment of objects in PLC software
 - Get PLC tags of a specific device item
- Safety engineering
 - PROFIsafe addressing capability F-BaseID
 - Read F-signatures from Safety Administration Editor
 - Enable and disable F-activation of an F-PLC
- Safety Validation Assistant
 - Create and manage activation tests
- Technology objects
 - Configure technology object SIMATIC Ident
 - Connecting with Superimposing Axes
 - Data block member and interpreter mapping for motion technology objects
- WinCC Unified
 - Create and manage HMI graphic lists

- Access additional HMI runtime settings
- New HMI controls: Alarm Line, Process Diagnostic Criteria Analysis, Process Diagnosis Graph Overview, Process Diagnosis Overview, Process Diagnosis PLC Code Viewer
- New HMI widgets: Alarm Indicator
- Export and import layout fields of a screen
- Alarming and ProDiag
 - Export and import supervision names and user labels
- SIMATIC Visualization Architect (SiVArc)
 - Configure acquisition mode
 - Configure layout data
 - Upgrade SiVArc definitions
 - Access tag members
- Web server
 - Manage default web pages and web applications
- OPC UA
 - Generate OPC UA nodes during creation of reference namespaces
 - Rename existing OPC UA reference namespace
 - Configure OPC UA access control for users and roles
 - Dynamic management of OPC UA client certificates
- Version Control Interface (VCI)
 - Support of F-compliant PLC data types, FBD/LAD/SCL and mixed blocks and data blocks in SIMATIC SD format
 - Support of types in the project library: PLC blocks, PLC data types
- SINAMICS Startdrive
 - Access drive item hardware modules and their component names
 - Upload drive parameters

See also

Major changes for long-term stability in TIA Portal Openness V21

See also

[Major changes in TIA Portal Openness V16](#)

[Major changes for long-term stability in TIA Portal Openness V17](#)

[TIA Portal Openness object model](#)

[Major changes in TIA Portal Openness V15.1](#)

[Major changes in TIA Portal Openness V15](#)

[Major changes in V14 SP1](#)

[Major changes in V14](#)

1.3 Basics

1.3.1 Installation

1.3.1.1 Requirements for TIA Portal Openness

Requirements for using TIA Openness applications

- A product based on the TIA Portal is installed on the PC, for example, "STEP 7 Professional" or "WinCC Professional".
- The "TIA Portal Openness" is installed on the PC.

See [Installing TIA Portal Openness](#)

Supported Windows operating systems

The following table shows which combinations of Windows operating system, TIA Portal and user application are mutually compatible:

Windows operating system	TIA Portal	User application
64-bit	64-bit	32-bit, 64-bit and "Any CPU"

Requirements for programming TIA Portal Openness applications

- Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8 and the Windows Classic Desktop package

Necessary user knowledge

- Knowledge as a system engineer
- Advanced knowledge of Microsoft Visual Studio 2017 or later with .Net Framework SDK 4.8
- Advanced knowledge of C# / VB.net and .Net Framework
- User knowledge of the TIA Portal

TIA Portal Openness remoting channels

The TIA Portal Openness remoting channels are registered as type `IpcChannel` with the "ensureSecurity" parameter set to "false".

Note

You should avoid registering another `IpcChannel` using a "ensureSecurity" parameter value other than "false" with a priority higher than or equal to "1".

The IpcChannel is defined with the following attributes:

Attribute	Settings
"name" and "portName"	Set to "\${Process.Name}_{Process.Id}" or "\${Process.Name}_{Process.Id}_{AppDomain.Id}" when registered in an AppDomain other than the application's default.
"priority"	Set with the default value of "1".
"typeFilterLevel"	Set to "Full".
"authorizedGroup"	Set to the NTAccount value string for the built-in user account (i.e. everyone).

See also

[Adding users to the "Siemens TIA Openness" user group](#)

1.3.1.2 Installing TIA Portal Openness

Introduction

TIA Portal Openness is an inherent feature of TIA Portal and is automatically installed.

Result

Depending on the different TIA Portal products and options, additional modular Siemens.Engineering assemblies will be installed.

A local Windows group "Siemens TIA Openness" is being created once during setup.

Note

Administrators must grant individual users the execution of any TIA Portal Openness application in general.

See also: [Adding users to the "Siemens TIA Openness" user group](#)

1.3.1.3 Adding users to the 'Siemens TIA Openness' user group

Introduction

When you install TIA Portal on your computer, the local Windows group "Siemens TIA Openness" is automatically created.

Whenever you try to access a TIA Portal process with your TIA Portal Openness application, TIA Portal verifies that you are a member of the "Siemens TIA Openness" user group, either directly or indirectly by way of another user group. If you are a member of the "Siemens TIA Openness" user group, the TIA Portal Openness application starts and establishes a connection to the TIA Portal instance, else an `EngineeringSecurityException` will be thrown telling you can't access this process.

Note

This group membership permits you to execute any TIA Portal Openness application.

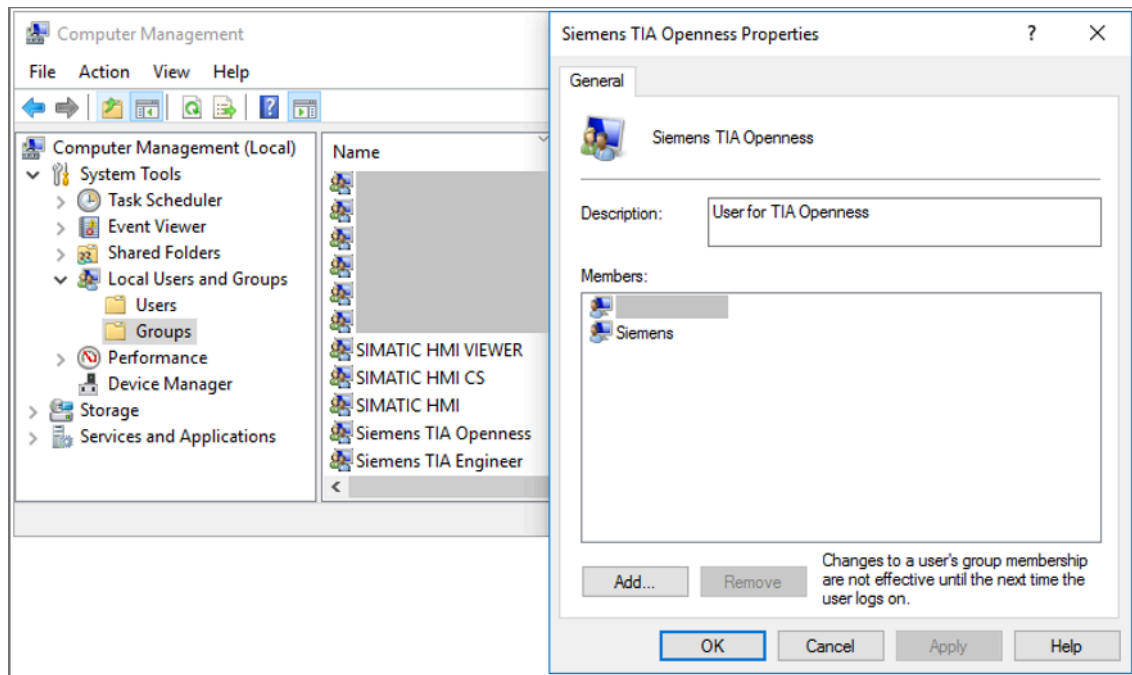
See also: [TIA Portal Openness firewall](#)

Procedure

Your administrator must manage the local group membership for "Siemens TIA Openness" in the local Microsoft Windows control panel.

1. Start the local "Computer Management" as administrator from the search in the Windows start menu
2. Go to "Groups" in "Local Users and Groups" to list the local user groups
3. Select "Siemens TIA Openness" and show the properties
4. Click on "Add..." to add a user
5. The affected user must fully sign out and sign in again

Example:



1.3.1.4 Using Windows Service

Introduction

Windows Services are long-running executables that run without a user interface and are installed using an installation utility. It is possible to develop such a Windows service application that uses TIA Portal Openness.

To do so, it is necessary to create a project of type "Windows Service (.NET Framework)" in Microsoft Visual Studio.

Requirement

- The process executable of the Windows Service application needs to be added to the AllowList beforehand. This can be done by creating the Windows registry entries on installation of the Windows Service, for example as part of a setup created with a Windows Installer project.
For more information about the AllowList, see section "Addition of an AllowList entry by connecting to TIA Portal" in chapter [TIA Portal Openness firewall](#)
- It is not possible to access running TIA Portal processes or to start TIA Portal with graphical interface. Only a new TIA Portal instance without user interface can be created.
- It is highly recommended to run the Windows Service with credentials of a dedicated Windows user account. It is possible but not recommended to use the local service account or the network service account. The local system account is not supported due to security restrictions and will always throw a `EngineeringSecurityException`.

- The used Windows account (either service or user) needs to be part of the local Windows user group "Siemens TIA Openness".

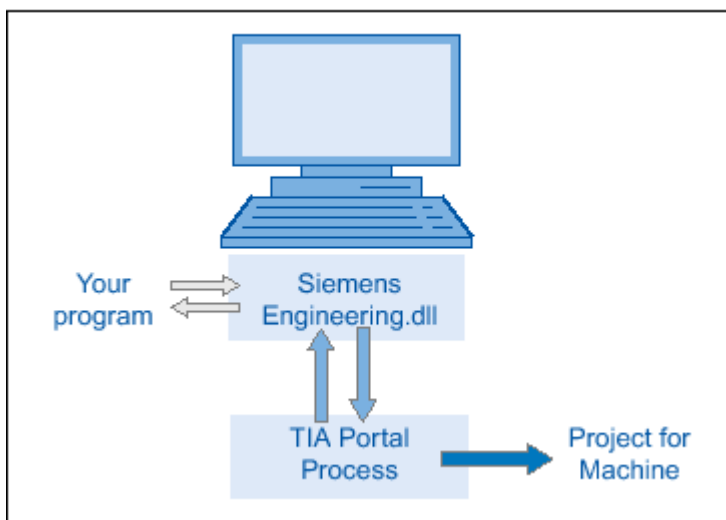
See also

TIA Portal Openness firewall

1.3.1.5 Accessing the TIA Portal

Overview

Configuration PC



Procedure

1. Set up the development environment to access and start the TIA Portal.
2. Instantiate the object of the portal application in your program to start the portal.
3. Find the desired project and open it.
4. Access the project data.
5. Close the project and exit the TIA Portal.

See also

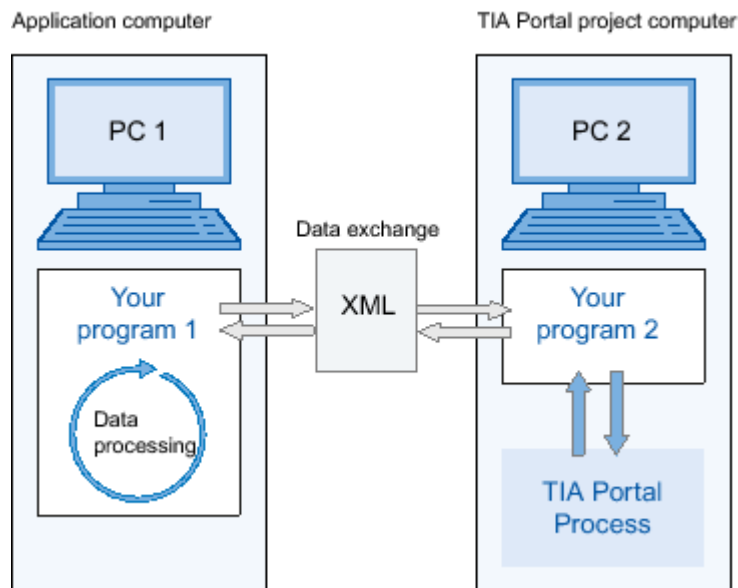
Connecting to the TIA Portal

Terminating the connection to the TIA Portal

1.3.1.6 Configurations

You can work with two variants of the TIA Portal Openness:

Application and the TIA Portal are on different computers



- Data exchange takes place by XML files. The XML files can be exported or imported by your programs.
- The data exported from the TIA Portal project to PC2 can be modified on PC1 and re-imported.

Note

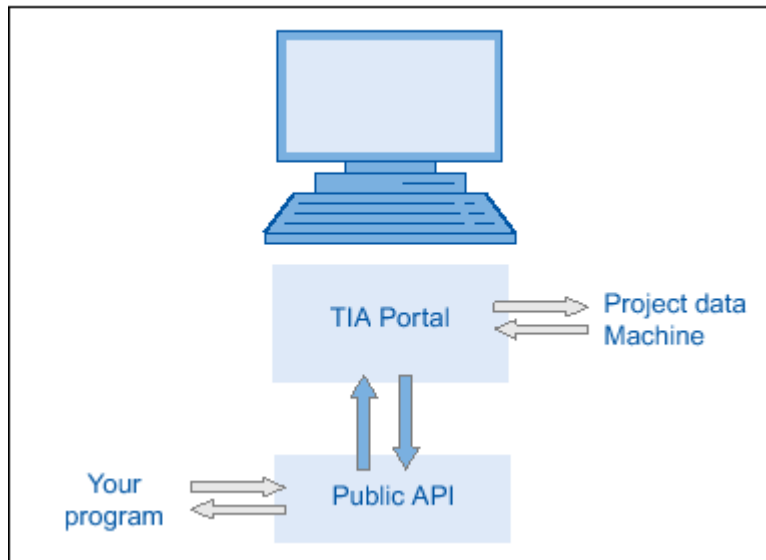
You have to develop an executable program "Your Program 2" for PC2, such as "program2.exe". The TIA Portal runs with this program in the background.

Import and export of XML files takes place exclusively via the TIA Portal Openness API.

- You can archive exchanged files for verification purposes.
- Exchanged data can be processed at different locations and times.

Application and the TIA Portal are on the same computer

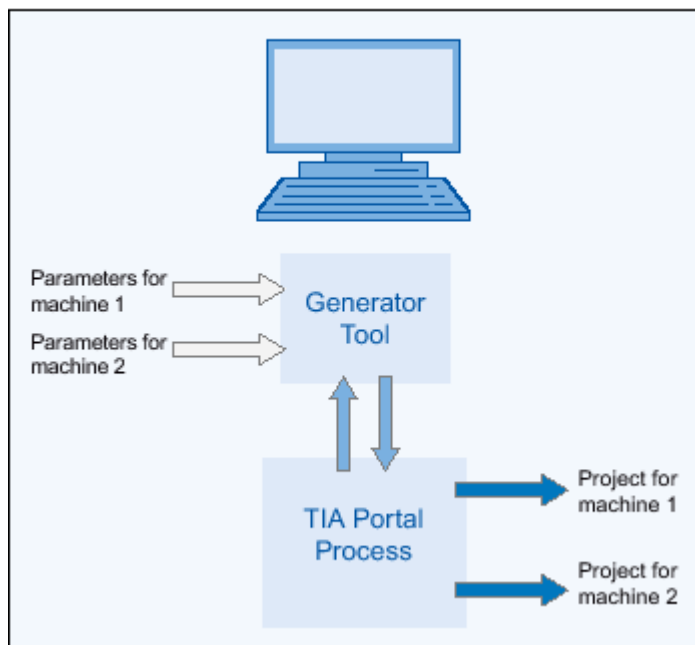
Configuration PC



- Your program launches the TIA Portal either with or without the user interface. Your program opens, saves and/or closes a project. The program can also connect to a running TIA Portal.
- You can then use the TIA Portal functionality to request, generate and modify project data or to initiate import or export processes.
- The data is created under control of the TIA Portal processing and stored in the project data.

Typical application in modular mechanical engineering

Configuration computer



- An efficient automation system is to be applied to similar machines.
- A project is available in the TIA Portal, which contains the components of all machine variants.

- The Generator tool controls the creation of the project for a specific machine variant.
- The Generator tool obtains the defaults by reading in the parameters for the requested machine variant.
- The Generator tool filters out the relevant elements from the overall TIA Portal project, modifies them if necessary and generates the requested machine project.

1.3.2 Openness tasks

1.3.2.1 Introduction

Introduction

TIA Portal Openness describes open interfaces for engineering with the TIA Portal. You will find further information about "TIA Portal Openness - Efficient generation of program code using code generators" in the [SIEMENS YouTube channel](#).

You automate the engineering with TIA Portal Openness by controlling the TIA Portal externally from a program you have created.

You can perform the following actions with TIA Portal Openness:

- Create project data
- Modify projects and project data
- Delete project data
- Read in project data
- Make projects and project data available for other applications.

Note

Siemens is not liable for and does not guarantee the compatibility of the data and information transported via these interfaces with third-party software.

We expressly point out that improper use of the interfaces can result in data loss or production downtimes.

Note

The code snippets contained in this documentation are written in C# syntax.

Due to the shortness of the used code snippets the error handling description has been shortened as well.

Application

The TIA Portal Openness interface is used to do the following:

- Provide project data.
- Access the TIA Portal process.
- Use project data.

Using defaults from the field of automation engineering

- By importing externally generated data
- By remote control of the TIA Portal for generating projects

Providing project data of the TIA Portal for external applications

- By exporting project data

Ensuring competitive advantages through efficient engineering

- You do not have to configure existing engineering data in the TIA Portal.
- Automated engineering processes replace manual engineering.
- Low engineering costs strengthen the bidding position as compared to the competition.

Working together on project data

- Test routines and bulk data processing can take place in parallel to the ongoing configuration.

See also

[Configurations](#)

1.3.2.2 Applications

Introduction

TIA Portal Openness provides you with various ways to access the TIA Portal and offers a selection of functions for defined tasks.

You access the following areas of the TIA Portal by using the TIA Portal Openness API interface :

- Portal data
- Project data
- PLC data
- HMI data
- Drive data

Accessing the TIA Portal

TIA Portal Openness offers various ways to access the TIA Portal. You create an external TIA Portal instance in the process either with or without UI. You can also access ongoing TIA Portal processes at the same time.

Accessing projects and project data

When accessing projects and project data, you mainly use TIA Portal Openness for the following tasks:

- Close, open and save the project
- Enumerate and query objects
- Create objects
- Delete objects

1.3.2.3 Export/import

Introduction

TIA Portal Openness supports the import and export of project data by means of XML files. The import/export function supports external configuration of existing engineering data. You use this function to make the engineering process effective and free of error.

Application

You use the import/export function for the following purposes:

- Data exchange
- Copying parts of a project
- External processing of configuration data, for example, for bulk data operations using find and replace
- External processing of configuration data for new projects based on existing configurations
- Importing externally-created configuration data, for example, text lists and tags
- Providing project data for external applications

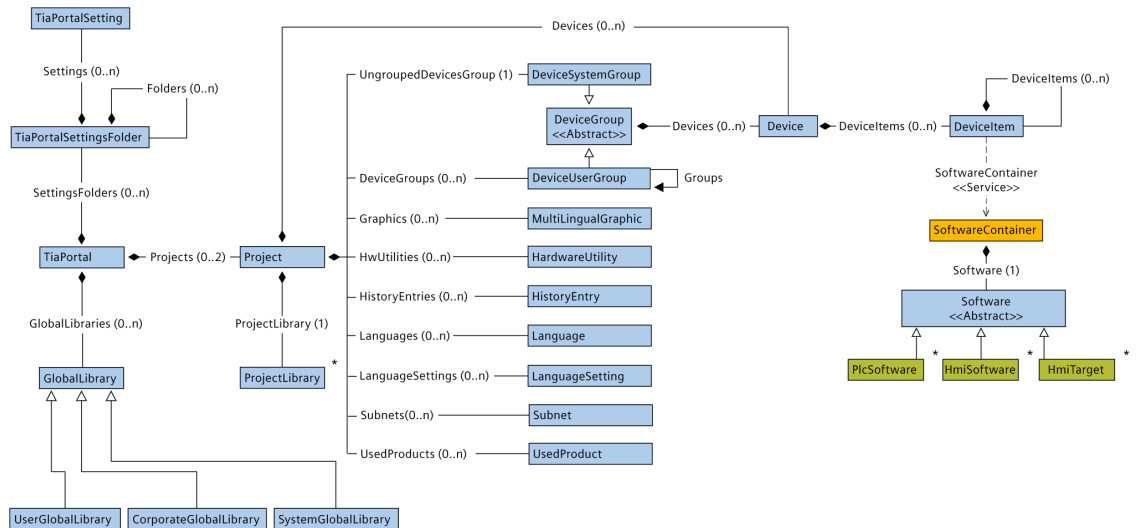
1.4 TIA Portal Openness API

1.4.1 TIA Portal Openness object

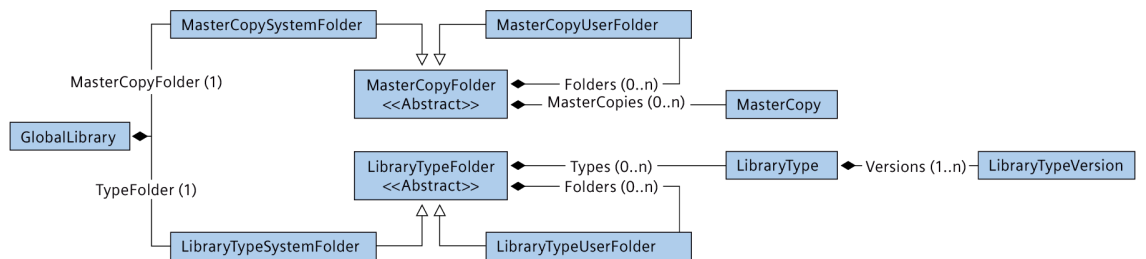
1.4.1.1 TIA Portal Openness object model

Overview

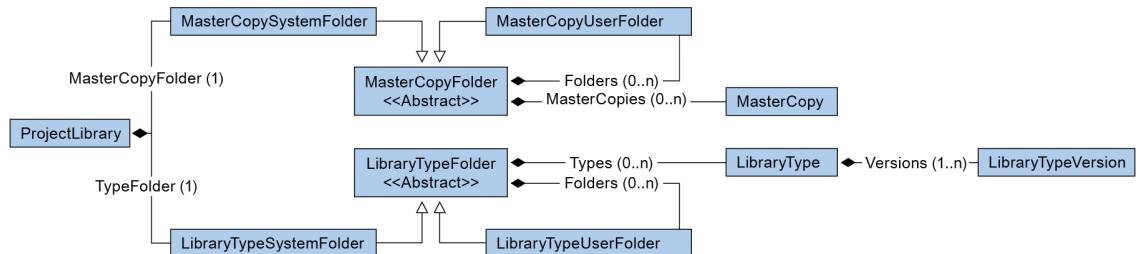
The following diagram describes the highest level of the TIA Portal Openness object model:



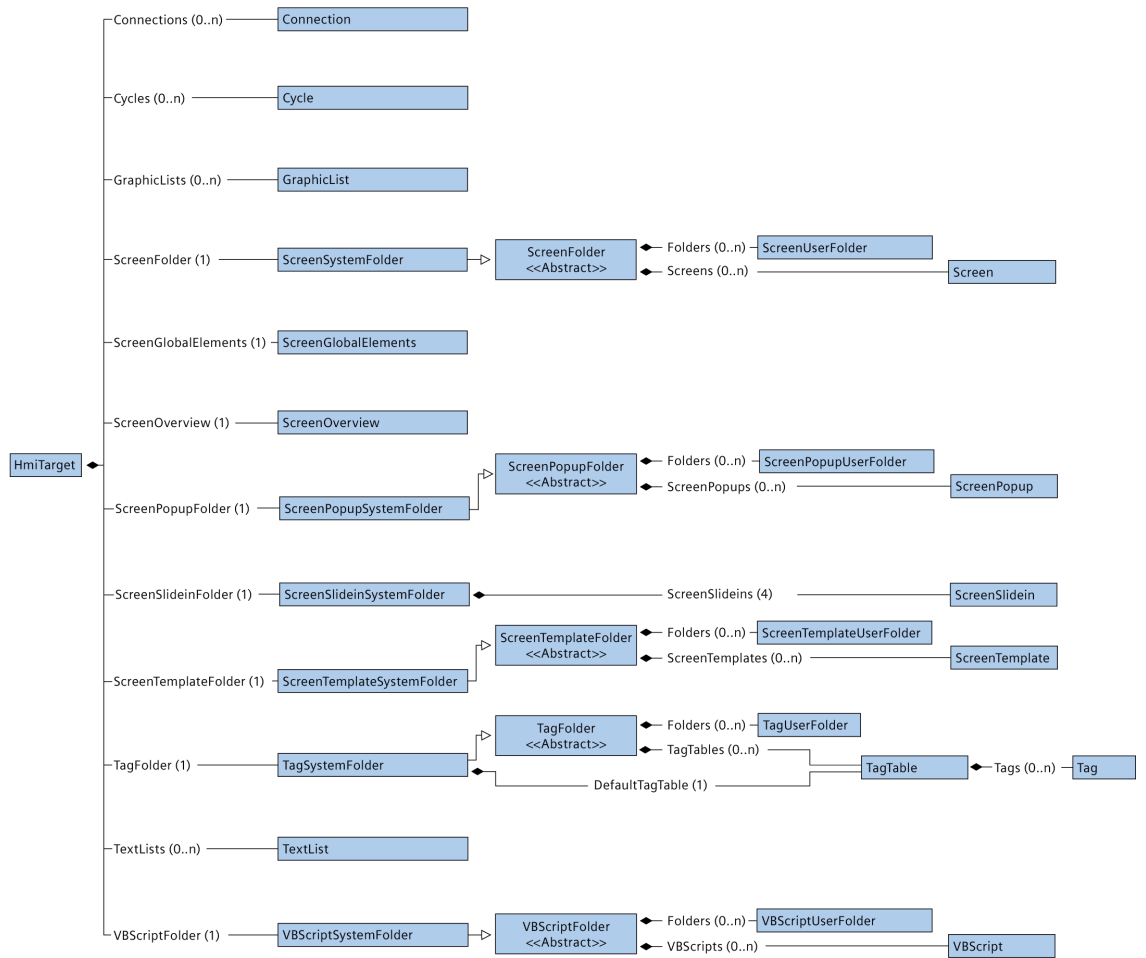
The following diagram describes the objects which are located under GlobalLibrary.



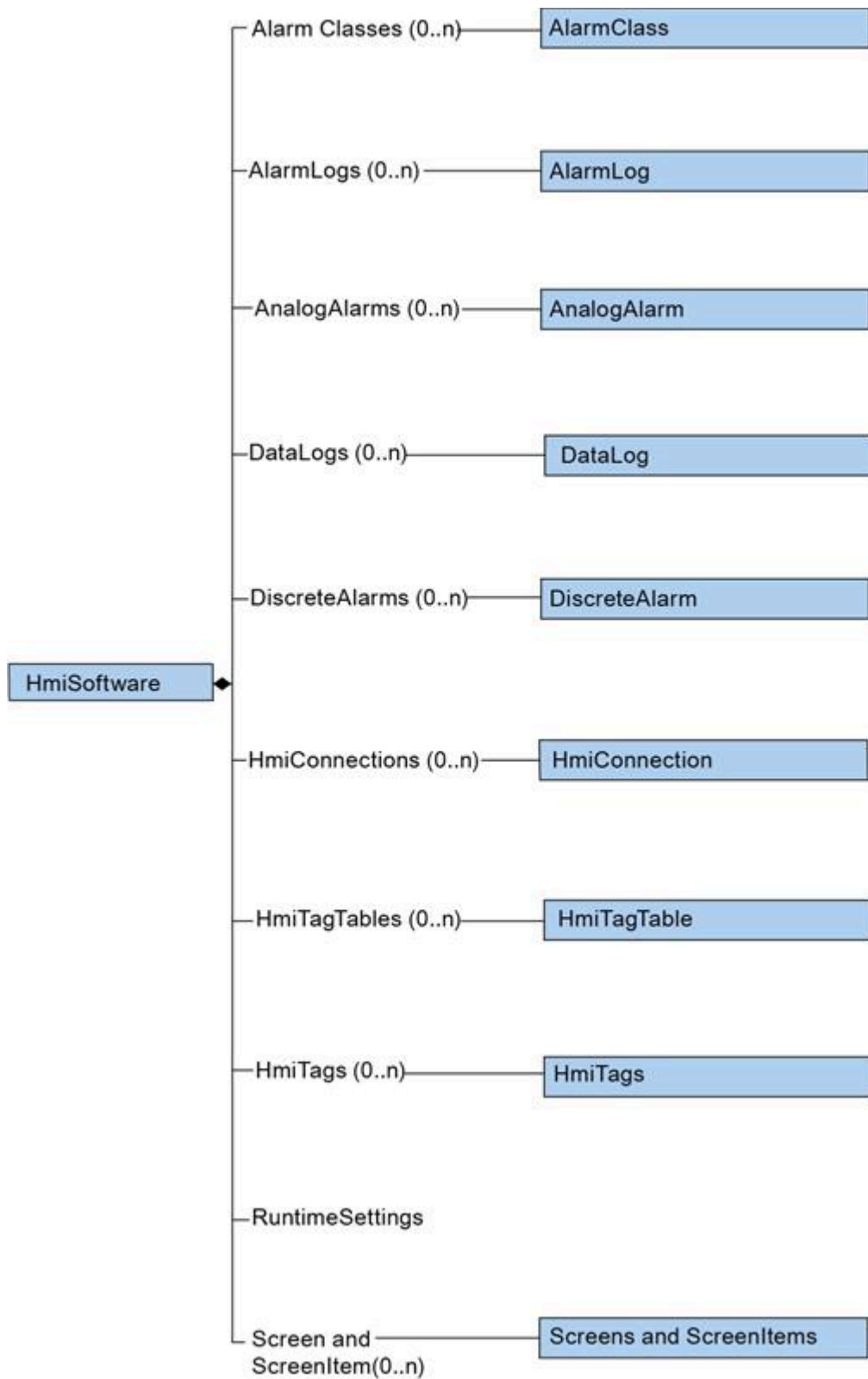
The following diagram describes the objects which are located under ProjectLibrary.



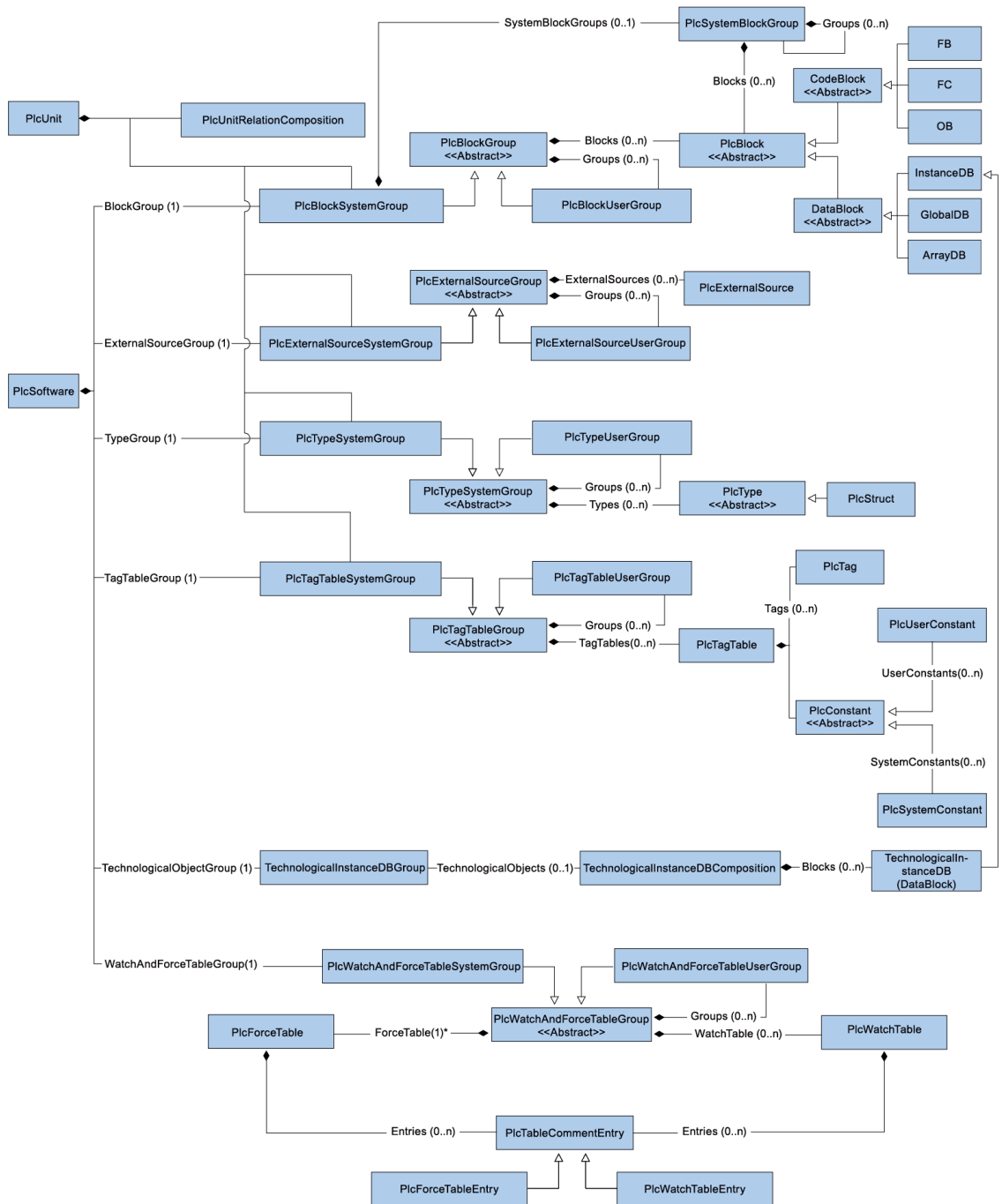
The following diagram describes the objects which are located under HmiTarget.



The following diagram describes the object which are located under HmiSoftware.



The following diagram describes the objects which are located under PlcSoftware.



Note

*The Force Table must be in PlcWatch and ForceTableSystemGroup

Access to objects in lists

You have the following options for addressing an object in a list:

- Address via the index. The counting within the lists starts with 0.

- Use Find method.

Use this method to address an object via its name. You can use this method for an composition or list. The Find method is not recursive.

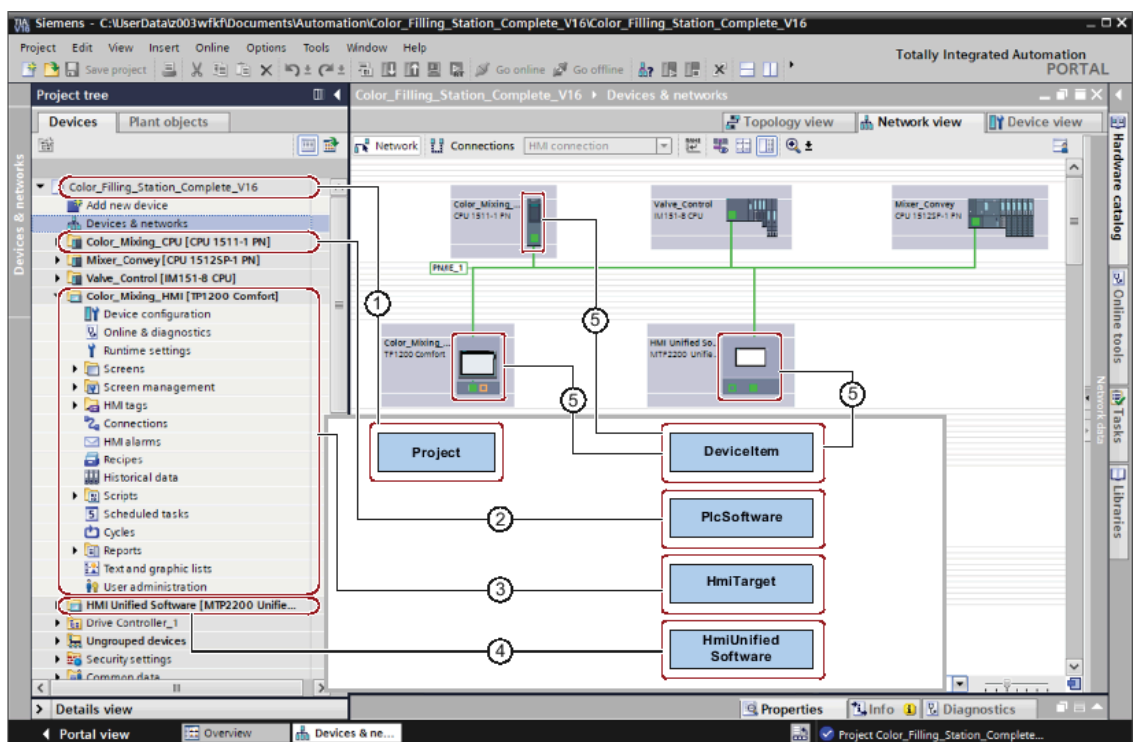
Example:

```
ScreenComposition screens = folder.Screens;
Screen screen = screens.Find("myScreen");
```

- Use symbolic names.

Relationship between TIA Portal and TIA Portal Openness object model

The figure below shows the relationship between the object model and a project in the TIA Portal:



See also

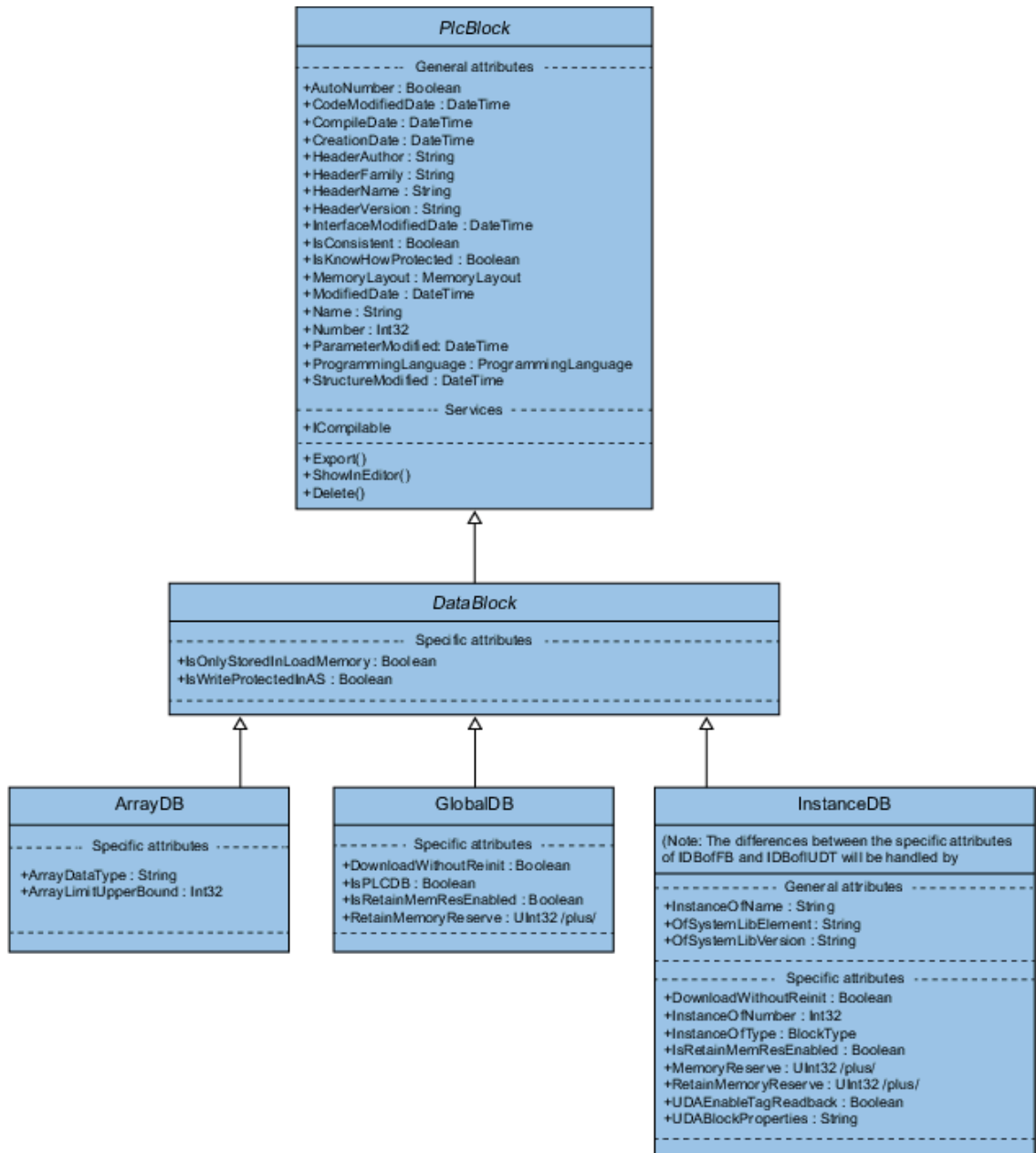
[Blocks and types of the TIA Portal Openness object model](#)

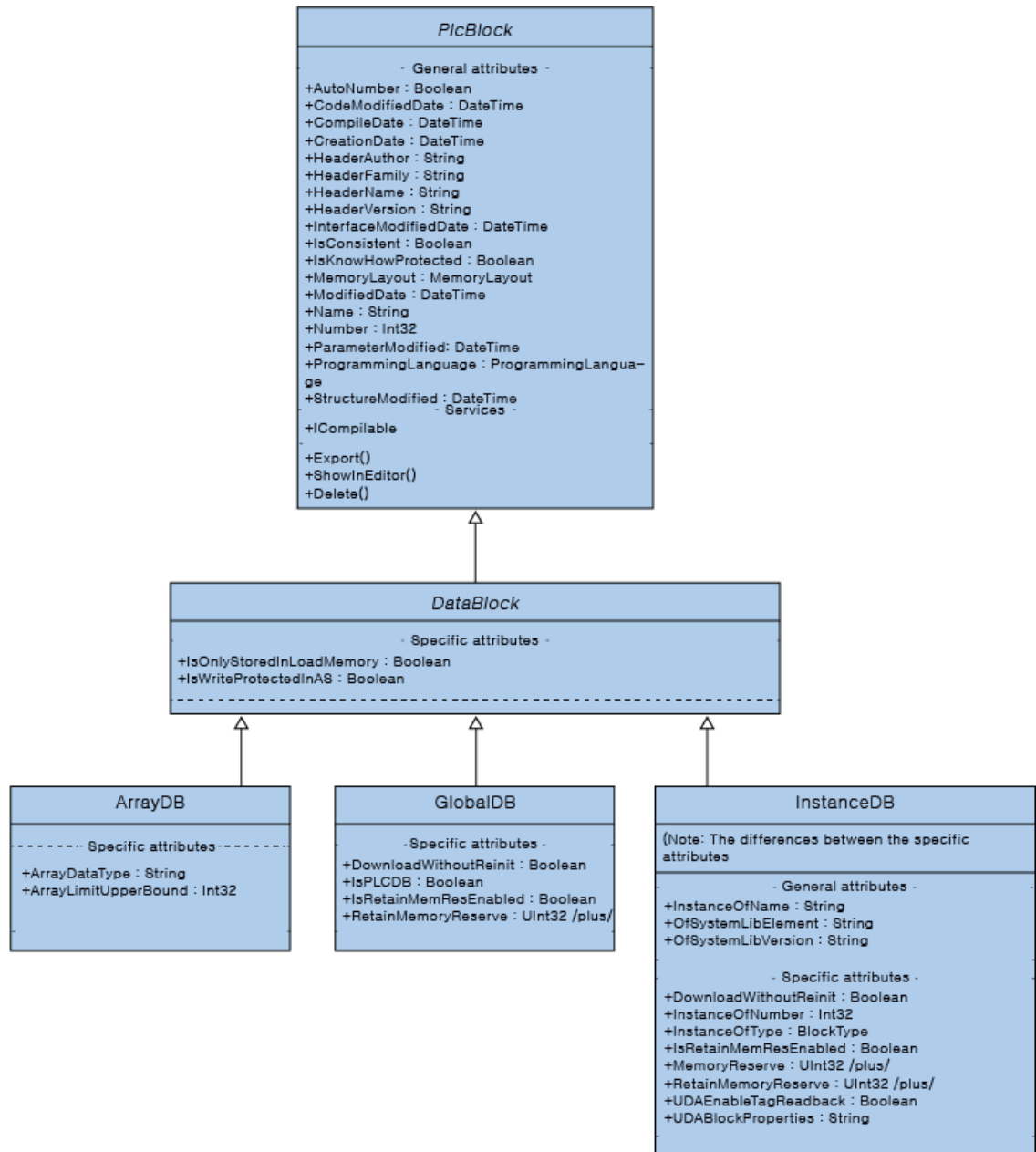
[Hierarchy of hardware objects of the object model](#)

1.4.1.2 Blocks and types of the TIA Portal Openness object model

Introduction

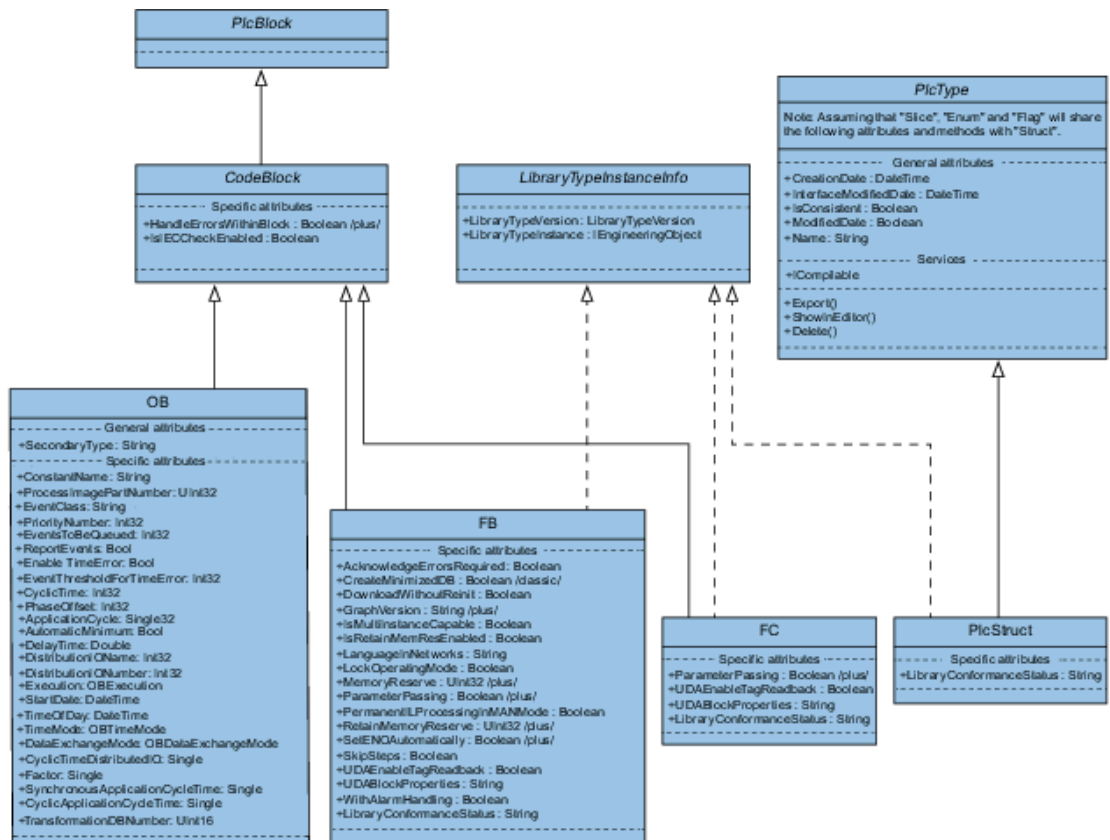
The following diagram describes the domain model of the PLCs to give an overview of the current modeling in TIA Portal Openness.





Representation of block diagram

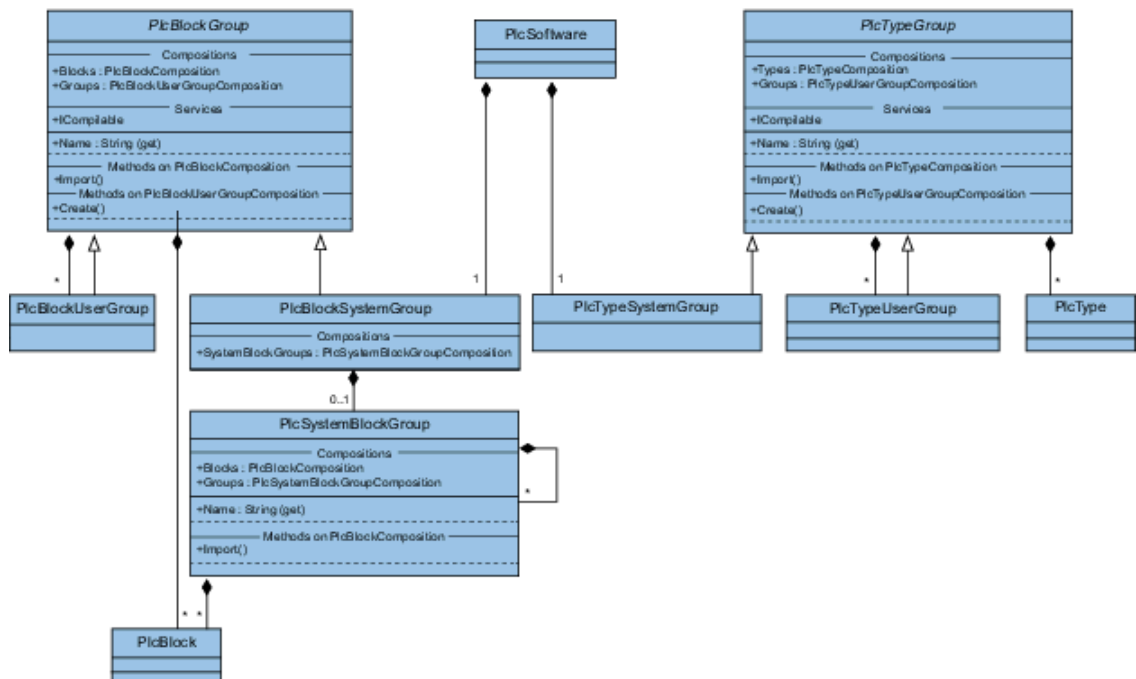
Code and Type



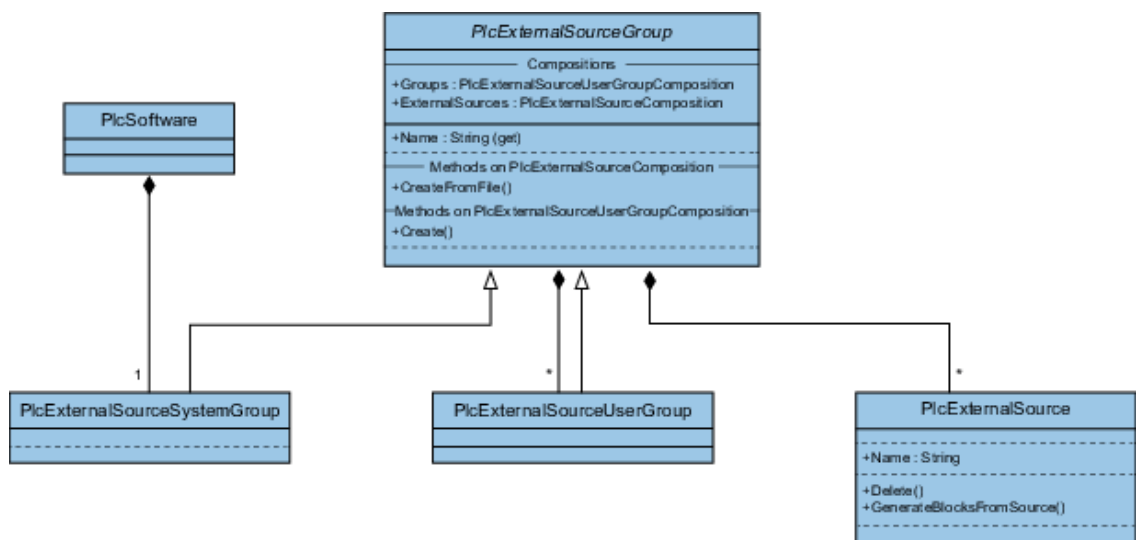
Representation of groups for blocks and types in the TIA Portal Openness API

The two high level groups "PlcBlocks" ("Program blocks" in the GUI of TIA Portal) and "PlcTypes" ("Plc data types" in the GUI of TIA Portal) contain blocks and type definitions. These groups provide the import and the compile functions for blocks. Due to the fact that most of the methods of functionalities of the groups are only achievable via collections, there is an "embedded" or "compacted" representation of the collections and their methods at the "host" classes.

Blocks and Types



External Sources



See also

[TIA Portal Openness object model](#)

[Hierarchy of hardware objects of the object model](#)

1.4.1.3 Hierarchy of hardware objects of the object model

Relation between the visible elements in the TIA Portal and the modeled elements in the object model

Hardware object	Explanation
Device (Device)	The container object for a central or distributed configuration.
Device item (DeviceItem)	Each device item object has a container object. The logical relation is "Items".

To understand the hierarchy relationship of hardware objects between what is displayed in the TIA Portal view and what is modeled in the TIA Portal Openness, it is necessary to consider some aspects of the hierarchy of hardware objects.

Example: A HardwareObject has the items link. The Device extends the HardwareObject and has the UnpluggedItems link. The Device is the container object for a station.

The DeviceItem extends the HardwareObject and has the Container link. Every plugged DeviceItem object has got a container object which is a HardwareObject. The reverse relation is Items.

An unplugged DeviceItem object that does not have a container is found via the link UnpluggedItems of the Device.

The container relation is comparable to the relation of the modules for the device item objects.

Example: A device includes one or more racks. A rack includes modules. A module includes submodules.

This is the relation similar to the representation in the network view and device view of the TIA Portal. The "PositionNumber" attribute of a device item is unique in the items area, within a container.

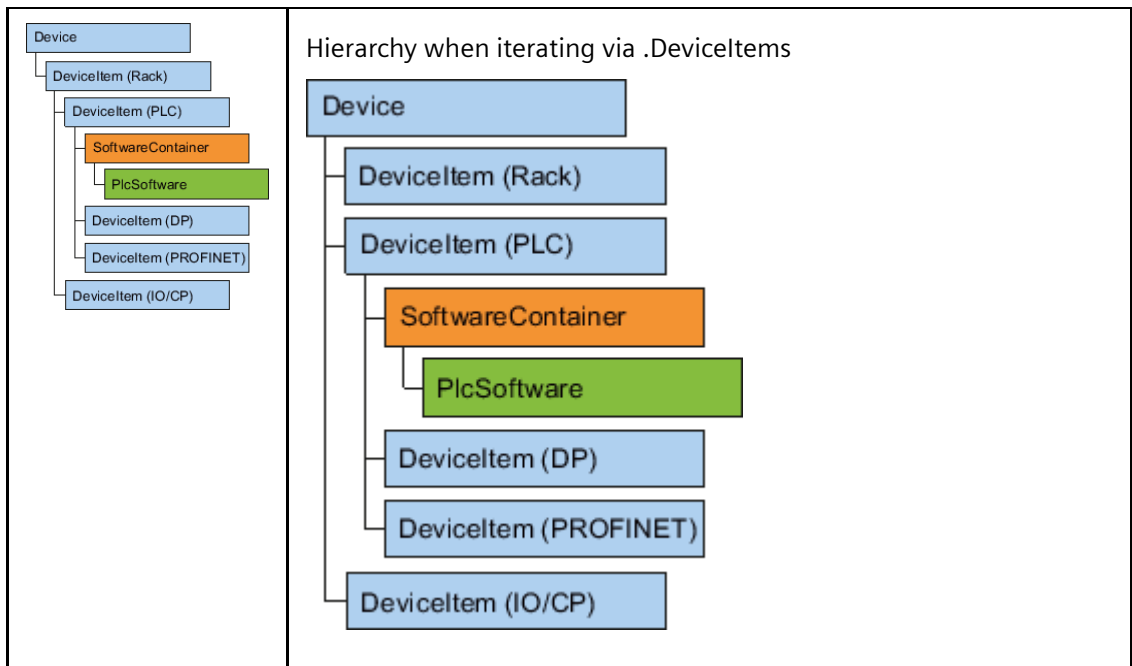
The parent-child relation between device item objects is a purely logical relation in the object model. A child cannot exist without its parents.

- If a submodule is modeled as part of a module (child), the submodule cannot be removed without the module.
- A module or submodule that can be added and removed is a child of the device object

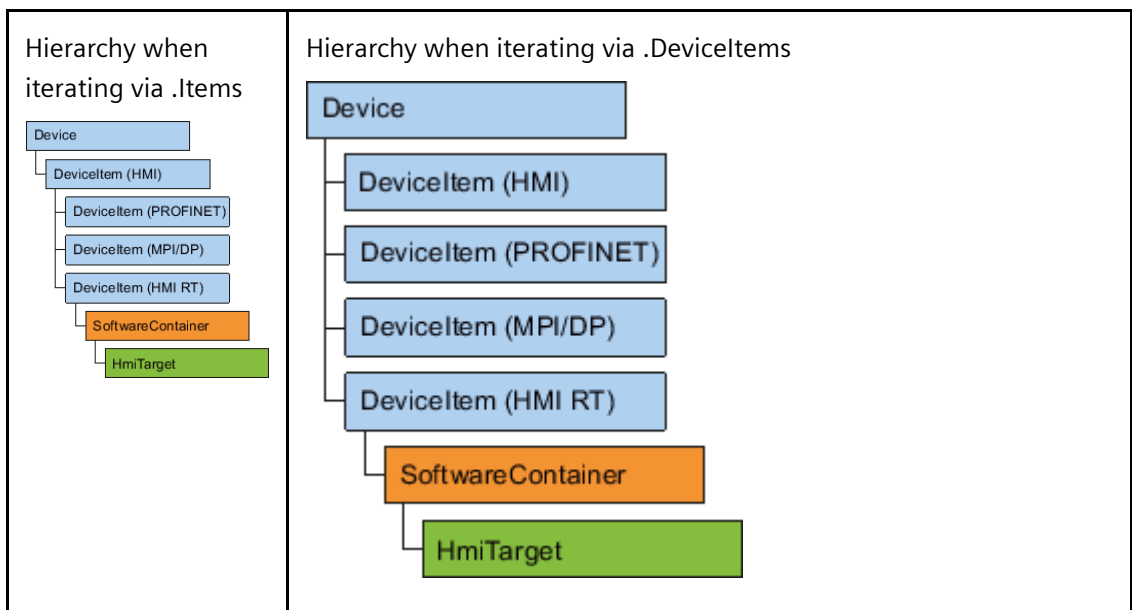
Hierarchy relationships of PLC devices

The diagrams below show the hierarchy relationship between devices, and device items of PLC and HMI devices.

Hierarchy when iterating via .Items	
-------------------------------------	--



Hierarchy relationships of HMI devices



See also

[TIA Portal Openness object model](#)

[Blocks and types of the TIA Portal Openness object model](#)

1.4.1.4 Object list

Introduction

The following tables show the available objects up to and including Runtime Advanced, and indicate whether these objects are supported by TIA Portal Openness.

Neither Runtime Professional nor device proxy files are supported by TIA Portal Openness in WinCC.

Objects

You can control the following project data depending on which HMI device you are using:

Screens

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Screen	Yes	Yes	Yes	Yes
Global screen	Yes	Yes	Yes	Yes
Templates	Yes	Yes	Yes	Yes
Permanent area	No	Yes	Yes	Yes
Pop-up screen	No	Yes	Yes	Yes
Slide-in screen	No	Yes	Yes	Yes

Screen objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Line	Yes	Yes	Yes	Yes
Polyline	No	Yes	Yes	Yes
Polygon	No	Yes	Yes	Yes
Ellipse	Yes	Yes	Yes	Yes
Ellipse segment	No	No	No	No
Circle segment	No	No	No	No
Elliptical arc	No	No	No	No

Camera view	No	No	No	No
Circular arc	No	No	No	No
Circle	Yes	Yes	Yes	Yes
PDF view	No	No	No	No
Rectangle	Yes	Yes	Yes	Yes
Connector	No	No	No	No
Text field	Yes	Yes	Yes	Yes
Graphic view	Yes	Yes	Yes	Yes
Pipe	No	No	No	No
Double T-piece	No	No	No	No
T-piece	No	No	No	No
Pipe bends	No	No	No	No
I/O field	Yes	Yes	Yes	Yes
Date/time field	Yes	Yes	Yes	Yes
Graphic I/O field	Yes	Yes	Yes	Yes
Editable text field	No	No	No	No
List box	No	No	No	No
Combo box	No	No	No	No
Button	Yes	Yes	Yes	Yes
Round button	No	No	No	No
Illuminated button	No	No	Yes	No

Switch	Yes	Yes	Yes	Yes
Symbolic I/O field	Yes	Yes	Yes	Yes
Key-operated switch	No	No	Yes	No
Bar	Yes	Yes	Yes	Yes
Symbol library	No	Yes	Yes	Yes
Slider	No	Yes	Yes	Yes
Scroll bar	No	No	No	No
Check box	No	No	No	No
Option buttons	No	No	No	No
Gauge	No	Yes	Yes	Yes
Clock	No	Yes	Yes	Yes
Memory space view	No	No	No	No
Function keys	Yes	Yes	Yes	Yes
Faceplate instances	No	Yes	Yes	Yes
Screen window	No	No	No	No
User view	Yes	Yes	Yes	Yes
HTML Browser	No	No	No	No
Print job/script diagnostics	No	No	No	No
Recipe view	No	No	No	No
Alarm view	No	No	No	No
Alarm indicator	No	No	No	No

Alarm window	No	No	No	No
Criteria analysis view	No	Yes ¹⁾	Yes	Yes
ProDiag overview	No	Yes ¹⁾	Yes	Yes
GRAPH overview	No	Yes ¹⁾	Yes	Yes
PLC code view	No	Yes ¹⁾	Yes	Yes
f(x) trend view	No	No	No	No
f(t) trend view	No	No	No	No
Table view	No	No	No	No
Value table	No	No	No	No
Media Player	No	No	No	No
Channel diagnostics	No	No	No	No
WLAN reception	No	No	No	No
Zone name	No	No	No	No
Zone signal	No	No	No	No
Effective range name	No	No	No	No
Effective range name (RFID)	No	No	No	No
Effective range signal	No	No	No	No
Charge condition	No	No	No	No
Handwheel	No	No	Yes	No
Help indicator	No	No	No	No
Sm@rtClient view	No	No	No	No

Status/Force	No	No	No	No
System diagnostic view	No	No	No	No
System diagnostic window	No	No	No	No

1) Only Mobile Panels with the device version greater than 12.0.0.0 support this screen object

Dynamic

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Display	Yes	Yes	Yes	Yes
Operability	No	Yes	Yes	Yes
Visibility	Yes	Yes	Yes	Yes
Movements	Yes	Yes	Yes	Yes

Additional objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Groups	Yes	Yes	Yes	Yes
Soft keys	Yes	Yes	Yes	Yes
Cycles	Yes	Yes	Yes	Yes
VB scripts	No	Yes	Yes	Yes
Function lists	Yes	Yes	Yes	Yes
Project graphics	Yes	Yes	Yes	Yes

Tags

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
--------	--------------	----------------	---------------	-------------

Multiplex tags	Yes	Yes	Yes	Yes
Arrays	Yes	Yes	Yes	Yes
User data types	Yes	Yes	Yes	Yes
Internal	No	Yes	Yes	Yes
Points of use of elementary data types	Yes	Yes	Yes	Yes
Points of use of user data types	Yes	Yes	Yes	Yes
Points of use of arrays	Yes	Yes	Yes	Yes

TIA Portal Openness also supports all value ranges which are supported by the communication drivers.

Connections

TIA Portal Openness supports non-integrated connections that are also supported by the respective HMI devices. You can find additional information in the online help for the TIA Portal under "Process visualization > Controller communication > Device-dependent".

Lists

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Text lists	Yes	Yes	Yes	Yes
Graphic lists	Yes	Yes	Yes	Yes

Texts

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multilingual texts	Yes	Yes	Yes	Yes
Formatted texts and their instances	No	Yes	Yes	Yes

1.4.2 General functions

1.4.2.1 Introduction

Overview

TIA Portal Openness supports a selection of functions for defined tasks that you can call outside the TIA Portal by means of the TIA Portal Openness API.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

You are provided with an overview of the typical programming steps in the sections below. You can learn how the individual code sections interact and how to integrate the respective functions into a complete program. You also get an overview of the code components that have to be adapted for each task.

Example program

The individual programming steps are explained using the "Creating API access in a console application" function as an example. You integrate the provided functions in this program code and adapt the respective code components for this task.

Functions

The section below lists the functions for defined tasks that you can call with TIA Portal Openness outside the TIA Portal.

See also

[Applications](#)

[Object list](#)

1.4.2.2 Information about installed TIA Portal Openness versions

Each installed TIA Portal version provides information about available, compatible TIA Portal Openness versions in the Windows registry. This enables TIA Portal Openness applications to automatically start or connect to matching TIA Portal instances.

The relevant TIA Portal Openness registry keys are in the following path in the Windows registry:

HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness

The registry key "Openness" contains the following subkeys and values:

- Version-independent subkey "AllowList" for the TIA Portal Openness firewall (only from TIA Portal V21 onwards)

- HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList
- See also [TIA Portal Openness firewall](#)
- Subkeys for each installed TIA Portal version: "xx.xx" (technical version number)
 - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx
 - The string value "PortalVersion" for a display string of that TIA Portal version (only from TIA Portal V21 onwards)
- Subkey "PublicAPI"
 - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI
- Subkeys for each provided TIA Portal Openness version in that specific TIA Portal version: "xx.xx.xx.xx" (technical version number)
 - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx
 - The string value "EngineeringVersion" for a display string of that TIA Portal Openness version
- Subkeys for each supported .NET target framework version in that specific TIA Portal Openness version using the Microsoft Target Framework Moniker (TFM) which is a standardized token format for specifying the target framework of a .NET library (only from TIA Portal V21 onwards): "netxxx"
 - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx\netxxx
 - The string value "AssemblyVersion" which the Microsoft CLR uses to load the exact version
 - The string value "PublicKeyToken" as part of the strong-named assembly to create a unique identity to prevent assembly conflicts
 - The string value "Siemens.Engineering.Base" pointing to the custom installation path

Note

Regardless which TIA Portal products are installed, the Windows registry only provides the installation path to the base assembly "Siemens.Engineering.Base".

See also

[Connecting to the TIA Portal](#)

1.4.2.3 TIA Portal Openness IntelliSense support

Application

The Intellisense support of TIA Portal Openness helps you at available attributes or methods via tooltip information. It could contain information about the number, names and types of the

required parameters. At the following example the bold parameter in the first line indicates the next parameter that is required as you type the function.

```
project = tiaPortal.Projects.OpenWithUpgrade(projectInfo);
```

 **Project** ProjectComposition.OpenWithUpgrade(FileInfo path)
Open Action with project update is necessary

You can manually invoke Parameter Info by clicking Edit IntelliSense/Parameter Info, typing CTRL+SHIFT+SPACE, or clicking the Parameter Info button on the editor toolbar.

1.4.2.4 Standard libraries

Insert the following namespace statements at the beginning of the relevant code example to ensure that the code examples work:

 Copies the following program code to clipboard.

```
using System;
using System.Collections.Generic;
using System.IO;
using Siemens.Engineering;
using Siemens.Engineering.Cax;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Compare;
using Siemens.Engineering.Download;
using Siemens.Engineering.Hmi;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.RuntimeScripting;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Extensions;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.HW.Utilities;
```

```

using Siemens.Engineering.Library;
using Siemens.Engineering.Library.MasterCopies;
using Siemens.Engineering.Library.Types;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.TechnologicalObjects;
using Siemens.Engineering.SW.TechnologicalObjects.Motion;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Upload;

```

1.4.2.5 Use of the code examples

Structure of the code-snippets

Each code-snippet in this documentation is implemented as a function without return value with an object reference as transfer parameter. Disposing of objects is omitted for the sake of readability. Objects of the TIA Portal are addressed by their name using the `Find` method.

 Copies the following program code to clipboard.

```

//Deletes a single screen from a user folder or a system
folder

private static void DeleteScreenFromFolder(HmiTarget
hmiTarget)
{
    //The screen "MyScreen" will be deleted if it is existing
in the folder "myScreenFolder".

    //If "myScreen" is stored in a subfolder of
"myScreenFolder" it will not be deleted.

    string screenName = "MyScreen";

    ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");

    ScreenComposition screens = folder.Screens;

    Screen screen = screens.Find(screenName);

```

```

        if (screen != null)
        {
            screen.Delete();
        }
    }
}

```

You need the following to execute this code-snippet:

- A WinCC project with an HMI device that includes a group with at least one screen.
- A function that instantiates the HMI device.

Note

When you specify directory paths, use the absolute directory path, for example, "C:/path/file.txt".

Relative directory paths are only allowed in the XML files for import and export, for example, "file.txt" or "C:/path01/.../path02/file.txt".

Example for execution of the code-snippet

Use the following example to execute the code-snippet "DeleteScreenFromFolder" as part of the "Hello TIA" example program:

 Copies the following program code to clipboard.

```

//In the sample program "Hello TIA" replace the function call
//"IterateThroughDevices(project)" by the following functions
calls:

    HmiTarget hmiTarget = GetTheFirstHmiTarget(project);
    DeleteScreenFromFolder(hmiTarget);

//Put the following function definitions before or after the
//function definition of "private static void
IterateThroughDevices(Project project)":
private static HmiTarget GetTheFirstHmiTarget(Project
project)
{
    if (project == null)
    {
        Console.WriteLine("Project cannot be null");
    }
}

```

```

        throw new ArgumentNullException("project");
    }

    foreach (Device device in project.Devices)
    {
        //This example looks for devices located directly in the
        project.

        //Devices which are stored in a subfolder of the project
        will not be affected by this example.

        {
            foreach (DeviceItem deviceItem in device.DeviceItems)
            {
                DeviceItem deviceItemToGetService = deviceItem as
                DeviceItem;

                SoftwareContainer container =
                deviceItemToGetService.GetService<SoftwareContainer>();

                if (container != null)
                {
                    HmiTarget hmi = container.Software as
                    HmiTarget;

                    if (hmi != null)
                    {
                        return hmi;
                    }
                }
            }
        }

        return null;
    }

    //Deletes a single screen from a user folder or a system
    folder

    private static void DeleteScreenFromFolder(HmiTarget
    hmiTarget)
    {
        string screenName = "MyScreen";
    }

```

```

        ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");

        ScreenComposition screens = folder.Screens;

        Screen screen = screens.Find(screenName);

        if (screen != null)
        {
            screen.Delete();
        }
    }
}

```

1.4.2.6 Example program

Application example: Creating API access in an application

The following program code shows basic API access in a Console application.

See also: [Connecting to the TIA Portal](#)

Note

Enabling the assembly reference property "Copy Local" is not supported. "Copy Local: True" means copying the references to Siemens.Engineering assemblies to the output directory of the application.

Remedy: Set "Copy Local: False" for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

 Copies the following program code to clipboard.

```

// Assembly reference: Siemens.Engineering.Base.dll
using Siemens.Engineering;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;

// Assembly reference: Siemens.Engineering.Step7.dll
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Units;

// System
using System;
using System.Collections.Generic;

```

```

using System.IO;
using System.Reflection;

namespace HelloOpenness
{
    internal static class Program
    {
        private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

        static Program()
        {
            // Register assembly resolver at the earliest stage in static
            Program constructor
            AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
        }

        private static Assembly OnAssemblyResolve(object sender,
            ResolveEventArgs args)
        {
            AssemblyName requestedAssemblyName = new
            AssemblyName(args.Name);

            string filePath = Path.Combine(OpennessFolder,
            string.Concat(requestedAssemblyName.Name, ".dll"));

            // Only respond to requests for TIA Portal Openness; must
            ignore missing files
            if
            (!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
            || !File.Exists(filePath))
            return null;

            Assembly loadedAssembly = Assembly.LoadFrom(filePath);

            // Check if details like version match the requested assembly
            if (requestedAssemblyName.FullName !=
            loadedAssembly.GetName().FullName)
            throw new FileNotFoundException("TIA Portal Openness version
            does not match", filePath);
        }
    }
}

```

```

return loadedAssembly;
}

public static void Main()
{
    Console.WriteLine("Please wait...");
    try
    {
        HelloOpennessApp app = new HelloOpennessApp();
        app.Run();
        Console.WriteLine("Done");
    }
    catch (Exception exception)
    {
        Console.WriteLine("Error");
        Console.WriteLine(exception);
    }
    Console.ReadLine();
}

internal class HelloOpennessApp
{
    private const string ProjectsPath = @"C:\Projects";

    public void Run()
    {
        using (TiaPortal portal = new
            TiaPortal(TiaPortalMode.WithUserInterface))
        {
            using (ExclusiveAccess exclusiveAccess =
                portal.ExclusiveAccess("Generator is working..."))
            {
                Project project = portal.Projects.Create(new
                    DirectoryInfo(ProjectsPath), "Project_1");
                Console.WriteLine($"Project has been created:
                    {project.Path.FullName}");

                Device station = project.Devices.CreateWithItem(
                    "OrderNumber:6ES7 516-3AP03-0AB0/V4.0", "PLC_1", "S7-1500
                    Station_1"
                );
            }
        }
    }
}

```

```

Console.WriteLine($"Device has been created:
{station.Name}");

PlcSoftware plc = GetPlcSoftware(station);
if (plc == null)
{
    Console.WriteLine("No PLC software found in the device.");
}
else
{
    PlcUnitProvider softwareUnits =
    plc.GetService<PlcUnitProvider>();
    if (softwareUnits == null)
    {
        Console.WriteLine("Software units are not supported for this
        PLC or firmware version.");
    }
    else
    {
        PlcUnit softwareUnit =
        softwareUnits.UnitGroup.Units.Create("Software unit_1");
        Console.WriteLine($"Software unit has been created:
        {softwareUnit.Name}");
    }
}

ICompilable compiler = station.GetService<ICompilable>();
if (compiler == null)
{
    Console.WriteLine("Compile service is not available for this
    device.");
}
else
{
    CompilerResult result = compiler.Compile();
    Console.WriteLine($"Compile: {result.WarningCount} warnings,
    {result.ErrorCount} errors.");
}

project.Save();
project.Close();

```

```

}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
    AssemblyName requestedAssemblyName = new
    AssemblyName(args.Name);

    string filePath = Path.Combine(OpennessFolder,
    string.Concat(requestedAssemblyName.Name, ".dll"));

    // Only respond to requests for TIA Portal Openness; must
    ignore missing files
    if
    (!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
    || !File.Exists(filePath))
    return null;

    Assembly loadedAssembly = Assembly.LoadFrom(filePath);

    // Check if details like version match the requested assembly
    if (requestedAssemblyName.FullName !=
    loadedAssembly.GetName().FullName)
    throw new FileNotFoundException("TIA Portal Openness version
    does not match", filePath);

    return loadedAssembly;
}

public static void Main()
{
    Console.WriteLine("Please wait...");
    try
    {
        HelloOpennessApp app = new HelloOpennessApp();
        app.Run();
        Console.WriteLine("Done");
    }
    catch (Exception exception)
    {
        Console.WriteLine("Error");
    }
}

```

```

Console.WriteLine(exception);
}
Console.ReadLine();
}
}

internal class HelloOpennessApp
{
private const string ProjectsPath = @"C:\Projects";

public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
using (ExclusiveAccess exclusiveAccess =
portal.ExclusiveAccess("Generator is working..."))
{
Project project = portal.Projects.Create(new
DirectoryInfo(ProjectsPath), "Project_1");
Console.WriteLine($"Project has been created:
{project.Path.FullName}");

Device station = project.Devices.CreateWithItem(
"OrderNumber:6ES7 516-3AP03-0AB0/V4.0", "PLC_1", "S7-1500
Station_1"
);
Console.WriteLine($"Device has been created:
{station.Name}");

PlcSoftware plc = GetPlcSoftware(station);
if (plc == null)
{
Console.WriteLine("No PLC software found in the device.");
}
else
{
PlcUnitProvider softwareUnits =
plc.GetService<PlcUnitProvider>();
if (softwareUnits == null)
{

```

```

Console.WriteLine("Software units are not supported for this
PLC or firmware version.");
}
else
{
PlcUnit softwareUnit =
softwareUnits.UnitGroup.Units.Create("Software unit_1");
Console.WriteLine($"Software unit has been created:
{softwareUnit.Name}");
}
}

ICompilable compiler = station.GetService<ICompilable>();
if (compiler == null)
{
Console.WriteLine("Compile service is not available for this
device.");
}
else
{
CompilerResult result = compiler.Compile();
Console.WriteLine($"Compile: {result.WarningCount} warnings,
{result.ErrorCount} errors.");
}

project.Save();
project.Close();
}
}
}

private static PlcSoftware GetPlcSoftware(HardwareObject
hardwareObject)
{
// Breadth-first traversal of a station
Queue<HardwareObject> queue = new Queue<HardwareObject>();
queue.Enqueue(hardwareObject);

while (queue.Count > 0)
{
// Check all modules at the current level

```

```

foreach (DeviceItem deviceItem in queue.Dequeue().Items)
{
    if (!deviceItem.IsPlugged) continue; // Skip if module is in
    parking lot

    if
    (deviceItem.Classification.HasFlag(DeviceItemClassifications.CPU)
    &&
    deviceItem.GetService<SoftwareContainer>().Software is
    PlcSoftware plcSoftware)
    return plcSoftware;

    // Enqueue all child modules for next level processing
    queue.Enqueue(deviceItem);
}

return null;
}
}
}

```

1.4.2.7 Programming steps

Overview

TIA Portal Openness requires the following programming steps for access by means of the TIA Portal Openness API:

1. Make the TIA Portal known in the development environment
2. Set up program access to the TIA Portal
3. Activate program access to the TIA Portal
4. Publish and start the TIA Portal
5. Open project
6. Execute commands
7. Save and close the project
8. Terminate the connection to the TIA Portal

Note

Permitted strings

Only certain characters are allowed in strings in the TIA Portal. All strings passed to the TIA Portal via the TIA Portal Openness application are subject to these rules. If you pass an invalid character in a parameter, an exception is thrown.

See also

[Example program](#)

[Use of the code examples](#)

1.4.2.8 Connecting to the TIA Portal

Introduction

TIA Portal Openness has the possibilities to

- get running TIA Portal processes and attach to TIA Portal instances
- start new TIA Portal instances, either with or without graphical user interface

Note

Since you always use TIA Portal Openness by loading Siemens.Engineering assemblies from a specific TIA Portal version, the returned processes are all from that same TIA Portal version. It is not supported to access different versions in the same AppDomain.

The app.config file and the AppDomain.CurrentDomain.AssemblyResolve event are two solutions for resolving the Siemens.Engineering assemblies during runtime.

Note

Resolving Siemens.Engineering assemblies is only needed for TIA Portal Openness applications, but not for TIA Portal Add-Ins.

Using the app.config file

Set "Copy Local: False" for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

By adding an application configuration file (app.config file) to a .NET project, you can customize how the Microsoft CLR locates and loads assemblies. The app.config file is deployed next to the executable and renamed to match the executable.

Note

The disadvantage of the app.config file is that it must contain the correct entries before the application starts. Changes cannot be made during program execution.

Example app.config file for .NET Framework 4.8:

 Copies the following program code to clipboard.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <startup>
    <supportedRuntime version="v4.0"
      sku=".NETFramework,Version=v4.8" />
  </startup>
  <runtime>
    <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
      <dependentAssembly>
        <assemblyIdentity name="Siemens.Engineering.Base"
          culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b" />
        <!-- Edit the following version and path according to your
          required TIA Portal Openness version -->
        <codeBase version="xx.xx.xx.xx"
          href="FILE:///C:\Program Files\Siemens\Automation\Portal
          Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Base.dll" />
      </dependentAssembly>
      <!-- Depending on your application, you may need to provide
        additional Siemens.Engineering assemblies -->
      <dependentAssembly>
        <assemblyIdentity name="Siemens.Engineering.Step7"
          culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b" />
        <!-- Edit the following version and path according to your
          required TIA Portal Openness version -->
        <codeBase version="xx.xx.xx.xx"
          href="FILE:///C:\Program Files\Siemens\Automation\Portal
          Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Step7.dll" />
      </dependentAssembly>
    </assemblyBinding>
  </runtime>
</configuration>
```

Using AppDomain.CurrentDomain.AssemblyResolve event

The assembly resolver must be registered at the earliest stage in the program.

Register the `AssemblyResolve` event before using any TIA Portal Openness types. This cannot happen in the same method. Place all TIA Portal Openness code in a separate method or separate class that is called only after event subscription.

Note: The .NET runtime will throw an exception if it encounters TIA Portal Openness types before the resolver is registered.

Important: This applies not only to method bodies but also to:

- Class properties with TIA Portal Openness types
- Method parameters with TIA Portal Openness types
- Return values with TIA Portal Openness types

Even when placed in separate methods, these will be resolved during class loading.

When you move your TIA Portal Openness code outside the `Program` class, the resolver will be active before that classes are evaluated during runtime.

Example of recommended application code structure:

 Copies the following program code to clipboard.

```
internal static class Program
{
    static Program()
    {
        // First register assembly resolver at the earliest stage in
        // static Program constructor
        AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
    }

    public static void Main()
    {
        // Then call method with TIA Portal Openness code in separate
        // class
        HelloOpennessApp app = new HelloOpennessApp();
        app.Run();
    }

    private static Assembly OnAssemblyResolve(object sender,
        ResolveEventArgs args)
    {
        // Code is shown in the next section
        // ...
    }
}
```

```

internal static class HelloOpennessApp
{
public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
// Do something with TIA Portal Openness
}
}
}

```

Example for a basic AssemblyResolve implementation where the installation path is given:

 Copies the following program code to clipboard.

```

private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);

```

```
return loadedAssembly;  
}
```

Note

These examples have demonstrated how to resolve multiple Siemens.Engineering assemblies using a robust and reliable approach. If your application architecture is very modular and your first class only accesses the TiaPortal class from Siemens.Engineering.Base.dll, it is sufficient to resolve only the Siemens.Engineering.Base.dll in your code. All subsequent Siemens.Engineering assemblies will then be automatically resolved by the Base library.

Note

Recommended way is using the TIA Portal Openness registry keys with the AppDomain.CurrentDomain.AssemblyResolve event.

See also: [Information about installed TIA Portal Openness versions](#)

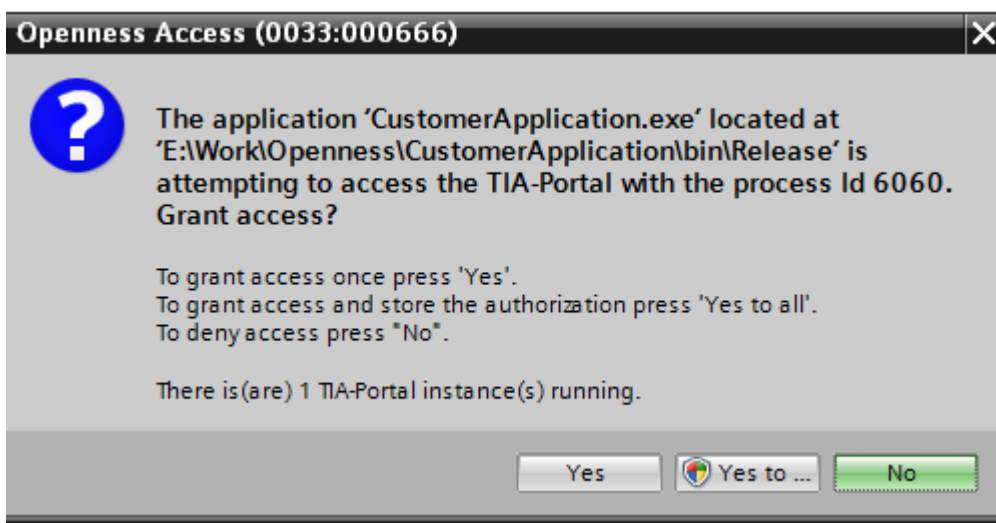
See also

[TIA Portal Openness firewall](#)

1.4.2.9 TIA Portal Openness firewall

TIA Portal Openness firewall prompt

When you try to connect to a running TIA Portal instance (includes starting a new TIA Portal instance) via TIA Portal Openness, TIA Portal will prompt you to accept or reject the connection like the following.



Note

Additionally, and beforehand, administrators must grant individual users the execution of any TIA Portal Openness application in general. This is managed by a local Windows group membership ("Siemens TIA Openness").

See also: [Adding users to the "Siemens TIA Openness" user group](#)

Allow connection to the TIA Portal once

Button "Yes": Users can allow individual TIA Portal Openness applications access to a specific (currently active) TIA Portal instance temporarily.

The next time the TIA Portal Openness application tries to connect to a TIA Portal instance, the firewall prompt will be shown again.

Addition of an AllowList entry by connecting to TIA Portal

Button "Yes to all": Administrators can allow individual TIA Portal Openness applications access to every TIA Portal instance permanently. This is managed by an automatically created Windows registry entry.

Note

If the file path, the modification date or the file content (file hash) of the TIA Portal Openness application changes, the application will be identified as a different one and the firewall prompt will be shown again.

Addition of an AllowList entry without using TIA Portal

Administrators can roll out a custom Windows registry file to automatically create an AllowList entry:

 Copies the following program code to clipboard.

```
Windows Registry Editor Version 5.00
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custom]
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custom]
"Path"="C:\\Program Files\\CustomerApplication\\bin\\CustomerApplication"
"DateModified"="2025/06/10 15:09:44.406"
"FileHash"="0rXRKUCNzMWHOMFrT52OwXzqJef10ran4UykTeBraaY="
```

The following example shows how you can calculate the values programmatically:

 Copies the following program code to clipboard.

```
using System;
using System.Globalization;
```

```

using System.IO;
using System.Security.Cryptography;

// Usage:
// CalculateValues(new FileInfo("C:\\Program
Files\\CustomerApplication\\bin\\CustomerApplication.exe"));

public static void CalculateValues(FileInfo
opennessApplicationInfo)
{
    using (FileStream stream =
opennessApplicationInfo.OpenRead())
    {
        HashAlgorithm hashAlgorithm = SHA256.Create();

        using (hashAlgorithm)
        {
            byte[] computedHash = hashAlgorithm.ComputeHash(stream);
            DateTime lastWriteTimeUtc =
opennessApplicationInfo.LastWriteTimeUtc;

            // This is how the path should be represented
            string path = opennessApplicationInfo.FullName;

            // This is how the date modified should be formatted
            string dateModified = lastWriteTimeUtc.ToString("yyyy/MM/dd
HH:mm:ss.fff", CultureInfo.InvariantCulture);

            // This is how the file hash should be represented
            string fileHash =
System.Convert.ToBase64String(computedHash);
        }
    }
}

```

See also: [Information about installed TIA Portal Openness versions](#)

1.4.2.10 Event handlers

Event handlers in TIA Portal Openness application

An instance of the TIA Portal provides the following events to which you can react with an event handler in a TIA Portal Openness application. You can access the attributes of notifications and

define the responses accordingly.

Event	Response
Disposed	Use this event to respond to the closing of the TIA Portal with a TIA Portal Openness application.
Notification	Use this event to respond to notifications of the TIA Portal with a TIA Portal Openness application. Notifications require only an acknowledgment, e.g. "OK".
Confirmation	Use this event to respond to confirmations of the TIA Portal with a TIA Portal Openness application. Confirmations always require a decision, e.g. "Do you want to save the project?".

Program code

Modify the following program code to register event handlers in a TIA Portal Openness application:

 Copies the following program code to clipboard.

```
//Register event handler for Disposed-Event

....

    tiaPortal.Disposed +=TiaPortal_Disposed;

....

private static void TiaPortal_Disposed(object sender,
EventArgs e)
{
    ....
}

//Register event handler for Notification-Event

....

    tiaPortal.Notification += TiaPortal_Notification;

....

private static void TiaPortal_Notification(object sender,
NotificationEventArgs e)
{
    ....
}
```

```

}

//Register event handler for Confirmation-Event

....

    tiaPortal.Confirmation += TiaPortal_Confirmation;

....

private static void TiaPortal_Confirmation(object sender,
ConfirmationEventArgs e)
{
    ....
}

```

Attributes of TIA Portal notifications

TIA Portal notifications have the following attributes:

Attribute	Description
Caption	Returns the name of the confirmation.
DetailText	Returns the detail text of the confirmation.
Icon	Returns the icon of the confirmation.
IsHandled	Returns the confirmation or specifies if it is still pending.
Text	Returns the text of the confirmation.

Attributes of confirmations

Confirmations have the following attributes:

Attribute	Description
Caption	Returns the name of the confirmation.
Choices	Returns the option to acknowledge the confirmation.
DetailText	Returns the detail text of the confirmation.

Icon	Returns the icon of the confirmation.
IsHandled	Returns the confirmation or specifies if it is still pending.
Result	Returns the result of the acknowledgment or specifies it.
Text	Returns the text of the confirmation.

See also

[Program-controlled acknowledgement of dialogs with system events](#)

1.4.2.11 Program-controlled acknowledgement of dialogs with system events

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)
- Event handlers are registered.
See [Connecting to the TIA Portal](#)

Application

When you operate the TIA Portal with the user interface, dialogs with system events are displayed for some program sequences. You decide how you want to proceed based on these system events.

When the TIA Portal is accessed with a TIA Portal Openness application, these system events must be acknowledged by means of corresponding ".NET" events.

The permitted confirmations are contained in the **Choices** list:

- Abort
- Cancel
- Ignore
- No
- NoToAll
- None
- OK

- Retry
- Yes
- YesToAll

The value of `ConfirmationEventArgs.Result` must be one of the above-mentioned entries. Otherwise, an exception is thrown.

Program code

Modify the following program code to notify the project engineer about executed actions of a TIA Portal Openness application:

 Copies the following program code to clipboard.

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
```

```

Namespace ConfirmDialogsWithSystemMessagesProgrammatically
{
class internal program()
{
private static void Main(string[] args)
{
// To respond to a confirmation event
using (TiaPortal tiaPortal = new TiaPortal());
{
tiaPortal.Confirmation += TiaPortalConfirmation(object
sender, ConfirmationEventArgs e);
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
private void TiaPortalConfirmation(object sender,
ConfirmationEventArgs e)
{
...
}
//Handles notifications
using (TiaPortal tiaPortal = new TiaPortal())
{
tiaPortal.Notification += Notification;
try
{

```

```
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
}
}
```

1.4.2.12 Terminating the connection to the TIA Portal

Introduction

If you started the TIA Portal instance without a user interface and if your application is the only TIA Portal Openness client attached to the TIA Portal, you can close the TIA Portal instance with the TIA Portal Openness application. Otherwise, you disconnect the TIA Portal Openness application from the TIA Portal instance.

Use the `IDisposable.Dispose()` method to separate or close the active instance of the TIA Portal.

You can use the `IDisposable.Dispose()` method as follows:

- With a using statement.
- Surround the object description with a try-finally block and call the `IDisposable.Dispose()` method within the finally block.

You can no longer access the TIA Portal when you close the active instance of the TIA Portal.

Note

When a configuration engineer closes the TIA Portal instance despite ongoing access of a TIA Portal Openness application, an exception of the class "NonRecoverableException" is thrown in the TIA Portal Openness application on the next API access. You can subscribe to the dispose event to get a call when the TIA Portal is closed.

Program code

Modify the following program code to separate or close the connection to the TIA Portal:

 Copies the following program code to clipboard.

```
// Add code to dispose the application if the application is
still instantiated

if (tiaPortal != null)
{
    tiaPortal.Dispose();
}
```

See also

[Event handlers](#)

1.4.2.13 Diagnostic interfaces on TIA Portal

Application

You can retrieve certain diagnostic information from running instances of TIA Portal via a static method. The diagnostic interface is implemented on the `TiaPortalProcess` object, which can be retrieved for any currently running instance of the TIA Portal.

The diagnostic interface is not blocking, so you can retrieve the `TiaPortalProcess` object and access its members, regardless of if the TIA Portal is busy or not. The diagnostic interface includes the following Members:

Class `TiaPortalProcess`

Member	Type	Function
<code>AcquisitionTime</code>	<code>DateTime</code>	The time when the <code>TiaPortalProcess</code> object was acquired. Since the <code>TiaPortalProcess</code> object represents a completely static snapshot of the state of the TIA Portal at a given point in time, the information it contains may become outdated.
<code>Attach</code>	<code>TiaPortal</code>	Attaches to the given <code>TiaPortalProcess</code> , it returns a <code>TiaPortal</code> instance.
<code>AttachedSessions</code>	<code>ICollection<TiaPortalSession></code>	A collection of all other sessions currently attached

		to the same TIA Portal. This collection can be empty. Each session is represented by a TiaPortalSession object.
Attaching	EventHandler<AttachingEventArgs>	This event enables an application to approve any attempts to attach to the TIA Portal. When another application attempts to attach to the TIA Portal, the subscribers of this event are notified and given 10 seconds to approve the attachment. If any subscriber ignores this event or does not respond in time, it is understood to be denying the other application permission to attach. Crashed applications, being unable to respond to this event and cannot cause an application to be denied permission to attach.
Dispose	void	Closes the associated TIA Portal instance.
Id	int	The Process ID of the TIA Portal.
InstalledSoftware	IList<TiaPortalProduct>	A collection of all the products currently installed as part of the TIA Portal. Each product is represented by a TiaPortalProduct object, which is described below.

Mode	TiaPortalMode	The mode in which the TIA Portal was started. The current values are WithUserInterface and WithoutUserInterface.
Path	FileInfo	The path to the executable of the TIA Portal.
ProjectPath	FileInfo	The path to the project which is currently open in the TIA Portal. If no project is open, this attribute will be null.

Class TiaPortalSession

Member	Type	Function
AccessLevel	TiaPortalAccessLevel	The level access of the session. It is represented as a flags enum, where multiple levels of access are possible. TiaPortalAccessLevel is described in detail below.
AttachTime	DateTime	The time when the connection to the TIA Portal was established.
Id	int	The Id of the current session.
IsActive	bool	Returns "true" if the TIA Portal is currently processing a call from the running session.
Dispose	void	Severs the process's connection to the TIA Portal. This method does not kill the process itself in the way that

		System.Diagnostics.Process. Kill would do. The application whose connection is terminated will still get a disposed event, but there is no other indication of why the connection was terminated.
ProcessId	int	The process ID of the attached process.
ProcessPath	FileInfo	The path to the executable of the attached process.
TrustAuthority	TiaPortalTrustAuthority	Indicates if the current session was started by a process that was signed, and if it is a TIA Portal Openness certificate or not. TrustAuthority is a flags enum and is described below.
UtilizationTime	TimeSpan	The period of time the process has spent actively using the TIA Portal. Combined with the AttachTime attribute, this could be used to determine usage percentages or similar data.
Version	string	The version of TIA Portal Openness to which the session is attached to.

Enum TiaPortalAccessLevel

Enum Value	Function
None	This is not a valid value. It is included because TiaPortalAccessLevel is a flags enum which needs an appropriate "zero value" to represent no flags being set, but it will never

	appear in actual use because no session can be started that has no access.
Published	The session has access to published functionality.
Modify	The session has modify access.

Enum TiaPortalTrustAuthority

Enum Value	Function
None	The main module of the attached process is not signed with a certificate.
Signed	The main module is signed with a certificate, which is not a TIA Portal Openness certificate.
Certified	The main module is signed with a TIA Portal Openness certificate.
CertifiedWithExpiration	The main module is signed with a TIA Portal Openness certificate that will become invalid at the end of its lifetime.

Class TiaPortalProduct

Member	Type	Function
Name	string	The name of the product (e.g. STEP 7 Professional).
Options	ICollection<TiaPortalProduct>	A collection of all optional packages that belong to the connected TIA Portal, represented as TiaPortalProduct objects. If an option package itself has option packages, this nesting could continue.

Version	string	The version string of the product.
---------	--------	------------------------------------

The following code snippet provides an example of how to use the diagnostic Interface to query information and of how to use them in your application.

 Copies the following program code to clipboard.

```
public void TiaPortalDiagnostics()
{
    IList<TiaPortalProcess> tiaPortalProcesses =
TiaPortal.GetProcesses();

    foreach (TiaPortalProcess tiaPortalProcess in tiaPortalProcesses)
    {
        Console.WriteLine("Process ID: {0}", tiaPortalProcess.Id);
        Console.WriteLine("Path: {0}", tiaPortalProcess.Path);
        Console.WriteLine("Project: {0}",
tiaPortalProcess.ProjectPath);
        Console.WriteLine("Timestamp: {0}",
tiaPortalProcess.AcquisitionTime);
        Console.WriteLine("UI Mode: {0}", tiaPortalProcess.Mode);
        //See method body below.
        Console.WriteLine("Installed Software:");
        EnumerateInstalledProducts(tiaPortalProcess.InstalledSoftware);
        Console.WriteLine("Attached Openness Applications:");
        foreach (TiaPortalSession session in
tiaPortalProcess.AttachedSessions)
        {
            Console.WriteLine("Process: {0}", session.ProcessPath);
            Console.WriteLine("Process ID: {0}", session.ProcessId);
            DateTime attachTime = session.AttachTime;
            TimeSpan timeSpentAttached = DateTime.Now - attachTime;
            TimeSpan utilizationTime = session.UtilizationTime;
            long percentageTimeUsed = (utilizationTime.Ticks /
```

```

timeSpentAttached.Ticks) * 100;

        Console.WriteLine("AttachTime: {0}", attachTime);

        Console.WriteLine("Utilization Time: {0}",
utilizationTime);

        Console.WriteLine("Time spent attached: {0}",
timeSpentAttached);

        Console.WriteLine("Percentage of attached time
spent using TIA Portal: {0}", percentageTimeUsed);

        Console.WriteLine("AccessLevel: {0}",
session.AccessLevel);

        Console.WriteLine("TrustAuthority: {0}",
session.TrustAuthority);

        if ((session.TrustAuthority &
TiaPortalTrustAuthority.Certified) !=
TiaPortalTrustAuthority.Certified)
        {
            Console.WriteLine("TrustAuthority doesn't
match required level, attempting to terminate connection to
TIA Portal.");

            session.Dispose();
        }
    }
}

public void
EnumerateInstalledProducts(IEnumerable<TiaPortalProduct>
products)
{
    foreach (TiaPortalProduct product in products)
    {
        Console.WriteLine("Name: {0}", product.Name);
        Console.WriteLine("Version: {0}", product.Version);
        //recursively enumerate all option packages
        Console.WriteLine("Option Packages \n:");
    }
}

```

```

        EnumerateInstalledProducts(product.Options);
    }
}

```

Security Relevant Information

Because of the fact that no connection to the TIA Portal is needed to use the diagnostics interface, it's possible to write a Windows service that uses the attaching event to check any application attempting to attach to a TIA Portal, e.g. only applications that begin with your company's name are allowed to attach. Another option might be to always grant access, but write information about attaching processes to a log. The following program code is an example event handler to check incoming connections:

 Copies the following program code to clipboard.

```

public void OnAttaching(object sender, AttachingEventArgs e)
{
    string name =
Path.GetFileNameWithoutExtension(e.ProcessPath);

    TiaPortalAccessLevel requestedAccessLevel = e.AccessLevel
& TiaPortalAccessLevel.Published;

    TiaPortalTrustAuthority certificateStatus =
e.TrustAuthority &TiaPortalTrustAuthority.Certified;

    if (requestedAccessLevel ==
TiaPortalAccessLevel.Published &&

        certificateStatus == TiaPortalTrustAuthority.Certified
&&

        name.StartsWith("SampleCustomerName"))
    {
        e.GrantAccess();
    }
}

```

1.4.2.14 Exclusive access

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

See [Connecting to the TIA Portal](#)

- A project is open.
See [Opening a project](#)

Application

The "TIA Portal" class provides the method "ExclusiveAccess(String text)" to establish an exclusive access to an attached TIA Portal process. The usage of an exclusive access is highly recommended even if it is not mandatory.

Use "ExclusiveAccess" in an "using" statement to ensure it is disposed attribute even when exceptions occur or the application is shutdown.

Note

Any attempt to create a second exclusive access within the scope of an open exclusive access will result in a recoverable exception being raised.

Modify the following example to get "ExclusiveAccess" to an instance:

 Copies the following program code to clipboard.

```
...  
[assembly: AssemblyTitle("MyApplication")]  
// This will be used for the exclusive access dialog when  
present....  
TiaPortal tiaPortal = ...;  
using (ExclusiveAccess exclusiveAccess =  
tiaPortal.ExclusiveAccess("My Activity"))  
{  
    ...  
}
```

After acquiring an "ExclusiveAccess" instance for a given TIA Portal process a dialog will be displayed. This dialog will display the message provided during instantiation. In addition the following information of the client application will be displayed:

- the assembly title of the manifest data if available; otherwise, the process name
- the process ID
- the SID (Openness Session ID of the client application's TiaPortal instance)

Note

There can be multiple sessions active for a given TIA Portal Openness client application because there can be multiple instances of TiaPortal each associated

with the same TIA Portal process.

The client application can also update the displayed content of the exclusive access dialog by setting the "Text" attribute with new values. Modify the following program code to invoke this behavior:

 Copies the following program code to clipboard.

```
exclusiveAccess = ...;

...

exclusiveAccess.Text = "My Activity Phase 1";

...

exclusiveAccess.Text = "My Activity Phase 2";

...

exclusiveAccess.Text = String.Empty; // or null;

...
```

You can request that the exclusive access could be cancelled by selecting the "Cancel" button. Modify the following program code to invoke this behavior:

 Copies the following program code to clipboard.

```
exclusiveAccess = ...;

...

if (exclusiveAccess.IsCancellationRequested)
{
    // stop your activity

    ...
}
else
{
    // continue your activity

    ...
}

...
```

1.4.2.15 Dynamic behaviours

Dynamic behaviors in Openness

The Openness users can determine objects and their attributes during the run time. They also are able to invoke actions and navigate to related objects bidirectionally in the hierarchy. In order to support dynamic behaviors the following interfaces are provided and implemented by Openness objects:

IEngineeringInstance

The interface that allows up navigation in the Object Model.

Properties:

- IEngineeringObject Parent - Parent object of an instance.

IEngineeringCompositionOrObject

The interface is implemented by both engineering objects and compositions.

IEngineeringObject

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance:

Methods:

- IList<EngineeringCreationInfo> GetCreationInfos(string compositionName) - Gets the list of composition infos available for the object.
- IEngineeringObject Create(string compositionName, Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated compositionName within the parameters
- object GetAttribute(string name) - Gets an attribute with the given name.
- IList<EngineeringAttributeInfo> GetAttributeInfos() - Returns a collection of EngineeringAttributeInfo objects describing the different attributes on this object.
- IList<object> GetAttributes(IEnumerable<string> names) - Gets a list of attributes for the given names
- IEngineeringCompositionOrObject GetComposition(string name) - Gets an IEngineeringCompositionOrObject with the given name.
- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.
- void SetAttribute(string name, object value) - Sets an attribute with the given name to the given value

- `void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes)` - Sets the attributes with the given names to the given values as indicated in attributes.
- `EngineeringObjectHandle GetHandle()` - Gets the object's unique handle.
- `void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler)` - Sets the attributes with the given names to the given values as indicated in attribute key value pair. If any error occurs (such as, attribute name/value is invalid), a callback call will be triggered so that you can decide whether to ignore and continue with setting value to the next attribute or abort setting the values to the remaining attributes.

The `SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler)` is supported by the following Engineering objects:

- `Siemens.Engineering.HW.Features.NetworkInterface`
- `Siemens.Engineering.HW.Features.NetworkPort`
- `Siemens.Engineering.HW.Features.PlcAccessLevelProvider`
- `Siemens.Engineering.HW.Address`
- `Siemens.Engineering.HW.Channel`
- `Siemens.Engineering.HW.Device`
- `Siemens.Engineering.HW.DeviceImpl`
- `Siemens.Engineering.HW.DeviceItem`
- `Siemens.Engineering.HW.DeviceItemImpl`
- `Siemens.Engineering.HW.HardwareObject`
- `Siemens.Engineering.HW.IoConnector`
- `Siemens.Engineering.HW.IoController`
- `Siemens.Engineering.HW.IoSystem`
- `Siemens.Engineering.HW.MrpDomain`
- `Siemens.Engineering.HW.Node`
- `Siemens.Engineering.HW.Subnet`
- `Siemens.Engineering.HW.SyncDomain`
- `Siemens.Engineering.HW.TransferArea`

If any other Engineering object is used, the method behaves similar to the method `void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes)` which has no callback.

- `GetAttributes(Siemens.Engineering.AttributeAccessOption)` - Get all attributes of an engineering object filtered by access mode with a single call. You can use the flag enum `Siemens.Engineering.AttributeAccessOptions` as a parameter for bulk API `GetAttributes` on an Engineering object to filter attributes based on access modes. The below options are available under `AttributeAccessOptions`:
 - None: No Attributes will be retrieved with this option. Empty list will be returned if this option is used in `GetAttributes` API.

- ReadOnly: Attributes which have only read access.
- WriteOnly: Attributes which have only write access. An exception of type Siemens.Engineering.EngineeringNotSupported Exception will be thrown If there is at-least one write only attribute on an engineering object and user invokes GetAttributes with AttributeAccessOptions.WriteOnly option.
- ReadWrite: Attributes which have read and write access.

Program code: SetAttributes(IEnumerable<KeyValuePair<string, object>>) method

 Copies the following program code to clipboard.

```
...
...

List<KeyValuePair<string, object>> attributeKeyValuePair =
new List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("Name", "DeviceItemName")
};
deviceItem.SetAttributes(attributeKeyValuePair,
(AttributeConfiguration attributeConfiguration) =>
{
//This callback delegate will be invoked when there is an
error while executing the SetAttributes API,
//reason could be either due to the attribute having invalid
attribute name, invalid value,
//invalid type of the value or when tried to set the value to
a readonly attribute
switch (attributeConfiguration)
{
case AttributeReadOnly attributeReadOnly:
attributeReadOnly.CurrentSelection =
AttributeChoiceSelection.Ignore;
break;

case AttributeNameUnsupported attributeNameUnsupported:
//Attribute name is not supported
attributeNameUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore;
```

```

// Or AttributeChoiceSelection.Abort
break;

case AttributeTypeUnsupported attributeTypeUnsupported:
//The the type of supplied value is not supported
attributeTypeUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;

case AttributeValueUnsupported attributeValueUnsupported:
//The value is not supported
attributeValueUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;
}
}
);

```

Program code: GetAttributes(Siemens.Engineering.AttributeAccessOption)

 Copies the following program code to clipboard.

```

private void GetAllAttributes(ProjectBase project)
{
    DeviceItem deviceItem = project.Devices[0].Items[0];
    // If there is no readonly attribute available in
    DeviceItem
    // then the returned list will be empty.
    IReadOnlyList<KeyValuePair<string, object>>
attributeNameValuePairs =
deviceItem.GetAttributes(AttributeAccessOptions.ReadOnly);
    foreach(KeyValuePair<string, object> pair in
attributeNameValuePairs)
    {
        Console.WriteLine("Attribute name: {0}, value: {1}",
pair.Key, pair.Value);
    }
}

```

```

    }
}

```

IEngineeringRoot

This interface is implemented by TiaPortal objects and indicates that the user is at the top object in the hierarchy. IEngineeringRoot is derived from IEngineeringObject, IEngineeringCompositionOrObject and IEngineeringInstance.

Methods:

- IEngineeringObject GetObject(EngineeringObjectHandle objectHandle) - Gets the object from a handle

IEngineeringComposition

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance. In addition to the properties and methods described in [Working with compositions](#), the following methods are supported.

Methods:

- IEngineeringObject Create(Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated in parameters..
- IList<EngineeringCreationInfo> GetCreationInfos() - Gets the collection of EngineeringCreationInfo objects describing the different CreateInfos on this object.
- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.

IEngineeringObjectAssociation

This interface is implemented by the collection of associated engineering objects. This interface is derived from IEngineeringAssociation and IEngineeringInstance. See [Working with associations](#)

IEngineeringServiceProvider

The interface is implemented by engineering objects that can provide service(s).

Methods:

- T GetService<T>() where T : class, IEngineeringService - Gets an instance of type T.
- IList<EngineeringServiceInfo> GetServiceInfos() - Returns a collection of EngineeringServiceInfo objects describing the different services on this object.

IEngineeringService

The interface representing an engineering service that is provided by IEngineeringServiceProvider.

EngineeringObjectHandle

The structure that provides a unique object handle.

See also

[Working with compositions](#)

[Working with associations](#)

1.4.2.16 Working with associations

Accessing associations

An association describes the relationship between two or more objects at type level.

TIA Portal Openness supports access to associations via index and via "foreach" loops. Direct access, for example via string name, is not supported.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadOnly`
- `IEngineeringObject Parent`
- `retType this [ int index ] { get; }`

Methods

TIA Portal Openness supports the following methods:

- `int IndexOf ( type )`: Returns the index in the association for a transferred instance.
- `bool Contains ( type )`: Determines whether the transferred instance is contained in the association.
- `IEnumerator GetEnumerator <retType>()`: Employed within "foreach" loops to access an object.
- `void Add ( type )`¹: Adds the transferred instance to the association.
- `void Remove ( type )`¹: Removes the transferred instance from the association.

¹: Not supported by all associations.

1.4.2.17 Working with compositions

Accessing compositions

A composition is the special case of an association. A composition expresses a semantic relationship of two objects, of which one is part of the other.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadOnly`
- `IEngineeringObject Parent`
- `retType this [int index] {get;}`: Indexed access to an object of the composition.

This type of access should only be used in a targeted manner, as each indexed access operation exceeds process boundaries.

Methods

TIA Portal Openness supports the following methods:

- `retType Create (id, ...)`: Creates a new instance and adds this instance to the composition.
The signature of the method depends on the way in which the instance is created. This method is not supported by all compositions.
- `type Find (id, ...)`: Scans a composition for the instance with the transferred ID.
The search is not recursive. The signature of the method depends on the way in which the instance is searched for. This method is not supported by all compositions.
- `IEnumerator GetEnumerator<retType> ()`: Employed within "foreach" loops to access an object.
- `Delete (type)`¹: Deletes the instance specified by the current object reference.
- `int IndexOf (type)`: Returns the index in the composition for a transferred instance.
- `bool Contains (type)`: Determines whether the transferred instance is contained in the composition.
- `void Import(string path, ImportOptions importOptions)`¹: Used for each composition that contains importable types.

Each import signature includes a configuration parameter of the type "ImportOptions" ("None", "Overwrite") by which the user controls the import behavior.

¹: Not supported by all compositions.

1.4.2.18 Verifying object equality

Application

As user of a TIA Portal Openness API, you can check that objects are the same with program code:

- You check whether two object references are the same with the operator "==".
- Use the `System.Object.Equals()` method to check if both objects are really identical with regard to the TIA Portal.

Program code

Modify the following program code to check for object reference types:

 Copies the following program code to clipboard.

```
...  
  
//Composition  
DeviceComposition sameCompA = project.Devices;  
DeviceComposition sameCompB = project.Devices;  
if (sameCompA.Equals(sameCompB))  
{  
    Console.WriteLine("sameCompA is equal to sameCompB");  
}  
if (!(sameCompA == sameCompB))  
{  
    Console.WriteLine("sameCompA is not reference equal to  
sameCompB");  
}  
DeviceComposition sameCompAsA = sameCompA;  
if (sameCompAsA.Equals(sameCompA))  
{  
    Console.WriteLine("sameCompAsA is equal to sameCompA");  
}  
if (sameCompAsA == sameCompA)  
{
```

```

        Console.WriteLine("sameCompAsA is reference equal to
sameCompA");
    }

    MultiLingualGraphicComposition notSameComp =
project.Graphics;

    if (!sameCompA.Equals(notSameComp))

    {

        Console.WriteLine("sameCompA is not equal to
notSameComp");
    }

```

1.4.2.19 Read operations for attributes

Group operations and standard read operations for attributes

TIA Portal Openness supports access to attributes via the following methods which are available at the object level:

- Group operation for read access
- Standard read operations

Program code for group operations

 Copies the following program code to clipboard.

```

//Exercise GetAttributes and GetAttributeNames
//get all available attributes for a device,
//then get the names for those attributes, then display the
results.

private static void DynamicTest(Project project)
{
    Device device = project.Devices[0];

    IList<string> attributeNames = new List<string>();

    IList<EngineeringAttributeInfo> attributes =
((IEngineeringObject) device).GetAttributeInfos();

    foreach (EngineeringAttributeInfo
engineeringAttributeInfo in attributes)
    {

```

```

        string name = engineeringAttributeInfo.Name;
        attributeNames.Add(name);
    }

    IList<object> values =
((IEngineeringObject) device).GetAttributes(attributeNames);
    for (int i = 0; i < attributes.Count; i++)
    {
        Console.WriteLine("attribute name: " +
attributeNames[i] + " value: " + values[i]);
    }
}

```

Group operation for read access

This method is available for any object:

```
public abstract IList<object> GetAttributes(IEnumerable<string> names);
```

Standard read operations

The following operations are available:

- Retrieve the names of available attributes:
Use the method [GetAttributeInfos\(\)](#) on an `IEngineeringObject`.

- Generic method for reading an attribute

```
public abstract object GetAttribute(string name);
```

Note

Dynamic attributes are not shown in IntelliSense because their availability depends on the status of the object instance.

1.4.2.20 Transaction handling

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Operation

A persistence (project, library, etc.) opened within an associated TIA Portal process can be modified by a TIA Portal Openness client application. You can produce this modification from a single operation or by a series of operations. Depending on the activity, it is reasonable to group these operations into a single undo unit for more logical workflows. Additionally there are performance advantages provided by grouping operations into a single undo unit. To support this, the "ExclusiveAccess" class provides the method "Transaction(ITransactionSupport persistence, string undoDescription)". The invocation of this method results in the instantiation of a new disposable object of type "Transaction". You have to provide a description of the transaction's contents (the text attribute cannot be null or Empty). While this instance has not been disposed, all client application operations will be grouped into a in a single undo unit within the associated TIA Portal process.

Modify the following program code to acquire a "Transaction" instance:

 Copies the following program code to clipboard.

```
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
    ...
}
```

Note

Use a "using" statement to instantiate a "Transaction" to ensure it is disposed properly even when exceptions occur, thus rolling back the transaction.

Consistent commit or rollback

The use of a "Transaction" within a client application helps you to ensure that there is a predictable way to commit or rollback a set of modifications. Your client application must decide whether or not to commit their modifications to a persistence. To do this your application must request that the modifications within the scope of an open transaction be committed when the transaction is disposed by invoking the 'Transaction.CommitOnDispose()' method. If this method is never invoked in the code flow, the modifications within the scope of the open transaction will automatically be rolled back when it is disposed.

Note

Any call to `Transaction.CommitOnDispose()` made after a `Siemens.Engineering.TargetInvocationException` has occurred within an open transaction will result in a recoverable exception being thrown.

If an exception occur after making the request, all modifications within the scope of an open transaction will still be rolled back on its disposal.

There are two properties available to determine the committal state of a given transaction:

Property	Data type	Description
<code>Transaction.CanCommit</code>	boolean	Indicates if a prior call to <code>Transaction.CommitOnDispose()</code> will be honored, or if a subsequent call to <code>Transaction.CommitOnDispose()</code> will result in a recoverable exception
<code>Transaction.CommitRequested</code>	boolean	Indicates if a call to <code>Transaction.CommitOnDispose()</code> has been made by the user within the given transaction

Modify the following program code to create a single undo unit in the attached TIA Portal containing two "Create" modifications:

 Copies the following program code to clipboard.

```
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
// Create a single undo unit with name "My Operation" in
// attached TIA Portal
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation")
{
project.DeviceGroups.Create("My Group 1");
project.DeviceGroups.Create("My Group 2");
transaction.CommitOnDispose();
}
```

Modify the following program code to create no undo unit:

 Copies the following program code to clipboard.

```

ExclusiveAccess exclusiveAccess = ...;

Project project = ...;

using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 2");
}

```

Modify the following program code to create no undo unit in the attached TIA Portal. This is due to exception raised by the second attempt to create a folder "My Folder 1" even though the request to commit on disposal was made before the exception was raised:

 Copies the following program code to clipboard.

```

ExclusiveAccess exclusiveAccess = ...;

Project project = ...;

using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 2");
transaction.CommitOnDispose();
project.DeviceGroups.Create("My Folder 1");
}

```

Modify the following program code to create no undo unit in the attached TIA Portal. This is due to exception raised by the second attempt to create a folder "My Folder 1" even though the try/catch block surrounds the call that caused the exception to be raised:

 Copies the following program code to clipboard.

```

ExclusiveAccess exclusiveAccess = ...;

Project project = ...;

using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
project.DeviceGroups.Create("My Folder 1");
}

```

```

project.DeviceGroups.Create("My Folder 2");
transaction.CommitOnDispose();
try
{
project.DeviceGroups.Create("My Folder 1");
}
catch (Siemens.Engineering.TargetInvocationException ex)
{
...
}
}

```

Modify the following program code to create undo unit created in the attached TIA Portal. This is due to the invocation of Transaction.CommitOnDispose() after a Siemens.Engineering.TargetInvocationException is raised prior in the code stream even though a try/catch block surrounds the call that caused the exception to be raised:

 Copies the following program code to clipboard.

```

ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
{
try
{
project.DeviceGroups.Create("My Folder 1");
project.DeviceGroups.Create("My Folder 1");
}
catch (Siemens.Engineering.TargetInvocationException ex)
{
...
}
transaction.CommitOnDispose();
}

```

Modify the following program code to have a transaction rolled back . This is due to the `Transaction.CommitOnDispose()` never being executed if the execution of any prior statement raises an exception within the scope of the given transaction.

 Copies the following program code to clipboard.

```
ExclusiveAccess exclusiveAccess = ...;
Project project = ...;
try
{
    using (Transaction transaction =
exclusiveAccess.Transaction(project, "My Operation"))
    {
        project.DeviceGroups.Create("My Folder 1");
        project.DeviceGroups.Create("My Folder 1");
        transaction.CommitOnDispose();
    }
}
catch (Exception ex)
{
    ...
}
```

Restrictions

The following actions are not allowed inside of a transaction. Calling these will result in a recoverable exception:

- Compile
- Go Online
- Go Offline
- ProjectText Import
- ProjectText Export
- Open Global Library
- Close Global Library
- Project Create
- Project Open

- Project OpenWithUpgrade
- Project Save
- Project SaveAs
- Project Close
- Set value in a TIA Portal Setting

Undo behavior

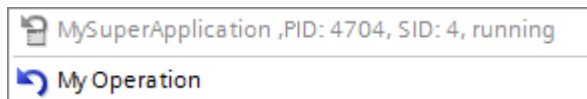
Actions performed by a TIA Portal Openness client application can result in undo units within the attached TIA Portal process. Each of these undo entries will be grouped under a location entry. This location entry will compose the following information from the client application:

- the assembly title from the manifest data if available; otherwise, the process name
- the process ID
- the SID (Openness Session ID of the client application's TiaPortal instance)
- optionally an indication that the client process is still running

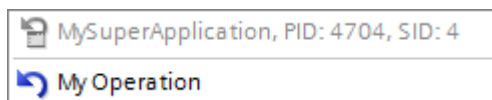
These entries will be one of the following two kinds:

1. The operations that are gathered into one undo transaction as a result of using a "Transaction" have the description as provided by the client application when the "Transaction" was instantiated.

- Undo entry for a running client application:

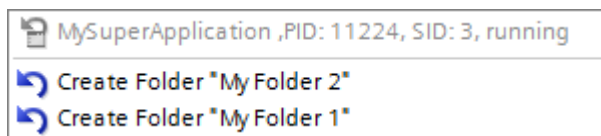


- Undo entry for a stopped client application:

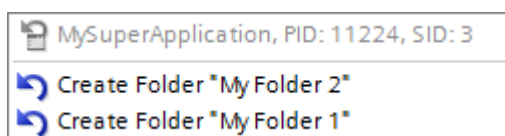


2. The operations that are executed individually have individual undo entries describing the operation as defined in the respective command meta data.

- Undo entry for a running client application:



- Undo entry for a stopped client application:



1.4.2.21 Creating a DirectoryInfo/FileInfo object

Application

The instances of `DirectoryInfo` and `FileInfo` classes have to contain an absolute path. Otherwise the methods using the `DirectoryInfo` or `FileInfo` objects will lead to an exception.

Program code

Modify the following program code to create a `DirectoryInfo` or a `FileInfo` object.

 Copies the following program code to clipboard.

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
```

```

namespace CreateObjectDirectoryInfoFileInfo
{
    internal class Program
    {
        private static void Main(string[] args)
        {
            RunTiaPortal();

            //..
            //Create a DirectoryInfo object
            string directoryPath = @"D:\Test\Project 1";

            DirectoryInfo directoryInfo = new
            DirectoryInfo(directoryPath);

            //Create a FileInfo object
            //please adapt the path and the extension apx to the
            installed version of TIA Portal
            string fileName = @"D:\Test\Project 1\Project 1.apx";
            FileInfo fileInfo = new FileInfo(fileName);

            //...
        }

        private static void RunTiaPortal()
        {
            Console.WriteLine("Starting TIA Portal");
            using (TiaPortal tiaPortal = new
            TiaPortal(TiaPortalMode.WithUserInterface))
            {
                Console.WriteLine("TIA Portal has started");
                ProjectComposition projects = tiaPortal.Projects;
                Console.WriteLine("Opening Project...");

                // please adapt the path and the extension apx to the
                installed version of TIA Portal
                FileInfo projectPath = new
                FileInfo(@"C:\Demo\AnyCompanyProject.apx");
            }
        }
    }
}

```

```

//edit the path according to your project
Project project = null;
try
{
project = projects.OpenWithUpgrade(projectPath);
}
catch (Exception)
{
Console.WriteLine(String.Format("Could not open project {0}",
projectPath.FullName));
Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
return;
}

Console.WriteLine(String.Format("Project {0} is open",
project.Path.FullName));
IterateThroughDevices(project);
project.Close();

Console.WriteLine("Demo complete hit enter to exit");
Console.ReadLine();
}
}

private static void IterateThroughDevices(Project project)
{
if (project == null)
{
Console.WriteLine("Project cannot be null");
return;
}

Console.WriteLine(String.Format("Iterate through {0}
device(s)", project.Devices.Count));
foreach (Device device in project.Devices)

```

```

{
    Console.WriteLine(String.Format("Device: \"{0}\".",
    device.Name));
}

Console.WriteLine();

}

}

}

```

1.4.2.22 Self-description support for attributes, navigators, actions, and services

Application

In TIA Portal Openness each `IEngineeringServiceProvider` of the TIA Portal Openness API describes its capabilities to potential calls.

Self-description Support on `IEngineeringObject`

Method Name	Return values
<code>GetCompositionInfos</code>	Returns a collection of <code>EngineeringCompositionInfo</code> objects describing the different compositions of these objects. <code>EngineeringCompositionInfo</code> is described below.
<code>GetAttributeInfos</code>	Returns a collection of <code>EngineeringAttributeInfo</code> objects describing the different attributes of these objects. <code>EngineeringAttributeInfo</code> is described below.
<code>GetCreationInfos(string compositionName)</code>	Returns a collection of <code>EngineeringCreationInfo</code> objects describing the different creation methods for items on a given composition of this object. <code>EngineeringCreationInfo</code> is described below.

GetInvocationInfos	Returns a collection of EngineeringInvocationInfo objects describing the different actions of these objects. EngineeringInvocationInfo is described below.
--------------------	--

Self-description Support on IEngineeringAssociation

Method Name	Return values
GetInvocationInfos()	Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object. EngineeringInvocationInfo is described below.

Self-description Support on IEngineeringComposition

Method Name	Return values
GetCreationInfos()	Returns a collection of EngineeringCreationInfo objects describing the different creation methods for items on this composition. EngineeringCreationInfo is described below.
GetInvocationInfos()	Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object. EngineeringInvocationInfo is described below.

Self-description Support on IEngineeringServiceProvider

Method Name	Return values
GetServiceInfos	Returns a collection of EngineeringServiceInfo objects describing the different services of these objects. EngineeringServiceInfo is described below.

Class EngineeringCompositionInfo

Attribute Name	Return values
Name	The name of the composition
Type	The type of the composition.

Class EngineeringAttributeInfo

Attribute Name	Return values
AccessMode	The level of access supported by the attribute. This attribute is combinable and is described in detail below.
Name	The name of the attribute.
CreateRelevance	The level to which the attribute is create relevant.
SupportedTypes	The potentially supported types of the attribute.

Class EngineeringInvocationInfo

Attribute Name	Return values
Name	The name of the action.
ParameterInfos	A collection of EngineeringInvocationParameterInfo objects describing any parameters that the action might require. EngineeringInvocationParameterInfo is described below.
Type	The return type of the action.

Class EngineeringServiceInfo

Attribute Name	Return values
----------------	---------------

Type	The type of the service as a System.Type object.
------	--

Enum EngineeringAttributeAccessMode

Enum Value	Return values
None	This is not a valid option.
Read	The attribute can be read.
ReadWrite	The attribute can be read and written
Write	The attribute can be written.

Enum EngineeringCreateRelevance

Enum Value	Return values
None	The attribute is not create relevant.
Mandatory	The attribute is required for the creation of an object.
Relevant	The attribute is optional for the creation of an object.

Class EngineeringInvocationParameterInfo

Attribute Name	Return values
Name	The name of the parameter.
Type	The type of the parameter as a System.Type object

Class EngineeringCreationInfo

Attribute Name	Return values
Type	The Type property returns a System.Type object

ParameterInfos	The ParameterInfos property returns a collection of EngineeringCreationParameterInfo objects
----------------	--

Class EngineeringCreationParameterInfo

Attribute Name	Return values
Name	The name of the parameter in question

Program code

AccessMode is a flags enum and its values can be combined like the following program code:

 Copies the following program code to clipboard.

```
EngineeringAttributeAccessMode value =
EngineeringAttributeAccessMode.Read|EngineeringAttributeAccessMode.Write
```

Modify the following program code to find all attributes of an IEngineeringObject and to do changes on the access mode of those attributes.

 Copies the following program code to clipboard.

```
...
IEngineeringObject engineeringObject = ...;
IList<EngineeringAttributeInfo> attributeInfos =
engineeringObject.GetAttributeInfos();
foreach(EngineeringAttributeInfo attributeInfo in attributeInfos)
{
    switch (attributeInfo.AccessMode)
    {
        case EngineeringAttributeAccessMode.Read:
            ...
            break;
        case EngineeringAttributeAccessMode.Write:
            ...
            break;
        case
EngineeringAttributeAccessMode.Read|EngineeringAttributeAccessMode.Write
```

```

        ...
        break;
    }
}
...

```

Modify the following program code of how Create and GetCreationInfos are intended to be used together.

 Copies the following program code to clipboard.

```

...
Project project = tiaPortal.Projects.Open(projectPath);
IEngineeringComposition deviceGroupComposition =
project.DeviceGroups;
EngineeringCreationInfo deviceGroupCreationInfo =
deviceGroupComposition.GetCreationInfos()[0];
EngineeringCreationParameterInfo
deviceGroupCreationParameterInfo =
deviceGroupCreationInfo.ParameterInfos[0];
string deviceGroupCreationParameterName =
deviceGroupCreationParameterInfo.Name;
string groupName = "Example";
IDictionary<string, object> createParameters = new
Dictionary<string, object>
{
    {deviceGroupCreationParameterName, groupName }
};
IEngineeringObject group =
deviceGroupComposition.Create(deviceGroupCreationInfo.Type,
createParameters);
...

```

1.4.2.23 Setting attributes of Blocks, DBs & UDTs

Introduction

It is possible to write and read (general) attributes of the Blocks (written in any programming language), DBs and UDTs via the Openness API.

TechnologicalObjects are internally DBs, so this feature is also valid for TOs.

The same checking rules are to be applied on Name by using SetAttribute of the Openness API on block, DB (TO) or UDT which are applied in the GUI.

In case of rule violation an appropriate user exception will be thrown and the attribute value won't be changed.

This extension of the API doesn't affect the XML export / import.

General Attributes

Legend

- X: The attribute is relevant, write to xml, accessible with API
- -: The attribute is not accessible, the attribute does not exist

Attributes	Datatype	Availability via the API		
		CodeBlock	DB	UDT
AutoNumber	Boolean	X (xml); RW (API)	X (xml); RW (API)	-
CodeModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	-
CompileDate	DateTime	X (xml); R (API)	X (xml); R (API)	-
CreationDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
HeaderAuthor	String	X (xml); R (API)	X (xml); R (API)	-
HeaderFamily	String	X (xml); R (API)	X (xml); R (API)	-
HeaderName	String	X (xml); R (API)	X (xml); R (API)	-
HeaderVersion	System.Version	X (xml); R (API)	X (xml); R (API)	-
InstanceOfName	String	-	X (xml); R (API)	-

Interface	String	X (xml); - (API)	X (xml); - (API)	X (xml); - (API)
InterfaceModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
IsConsistent	Boolean	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
IsKnowHowProtected	Boolean	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
MemoryLayout (sofar: Optimization)	MemoryLayout	X (xml); R (API)	X (xml); R (API)	-
ModifiedDate	DateTime	X (xml); R (API)	X (xml); R (API)	X (xml); R (API)
Name	String	X (xml); RW (API)	X (xml); RW (API)	X (xml); RW (API)
Number	Int32	X (xml); RW (API)	X (xml); RW (API)	-
ParameterModified	DateTime	X (xml); R (API)	X (xml); R (API)	-
ProgrammingLanguage	ProgrammingLanguage	X (xml); R (API)	X (xml); R (API)	-
SecondaryType	String	X (xml); R (API)	-	-
StructureModified	DateTime	X (xml); R (API)	X (xml); R (API)	-

1.4.2.24 Notes on performance of TIA Portal Openness

Root objects

You can specify several root objects in import files.

Example: You can create several text lists in an XML file instead of one text list for each XML file.

TIA Portal Openness functions

The first call of a TIA Portal Openness function can take longer than any subsequent calls of the TIA Portal Openness function.

Example: If you perform multiple configuration data exports one after the other, the first export can take longer than the subsequent exports.

1.4.3 Functions for projects and project data

1.4.3.1 Opening a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- The project to be opened is not open in any other instance of the TIA Portal.

Note

Undo of project upgrade

If you undo an upgrade of a project to V14SP1 after you have connected it to TIA Portal Openness, conflicts will occur.

Introduction

Use the `Projects.Open` method to open a project. Enter a path to the desired project in the `Projects.Open` method.

The `Projects.Open` method only accesses projects that were created with the current version of TIA Portal or which have been upgraded to the current version. If you access a project of a previous version with the `Projects.Open` method, an exception will be returned. Use the `OpenWithUpgrade` method, to open projects which have been made with previous versions of TIA Portal.

Program code

Modify the following program code to open a project:

 Copies the following program code to clipboard.

```
private static Project ProjectOpen(string projectPath)
{
    try
    {
        // Refer Connecting to the TIA Portal section
    }
}
```

```

        TiaPortal tiaPortal = new TiaPortal();

        Project project
=tiaPortal.Projects.Open(projectPath);

        if (project != null)
        {
            Console.WriteLine("Project Name: " +
project.Name);

            Console.WriteLine("Project Version: " +
project.Version);

        }

        return project
    }

    catch (Exception ex)
    {

        Console.WriteLine($"Exception of type
{ex.GetType()} occurred: {ex.Message}");

        return null;

    }

}

```

Opening a UMAC protected project

You can also open a UMAC protected project. The overload of Open function takes an additional parameter of type UmacDelegate. This additional parameter allows the caller to specify a handler to be used during UMAC authentication. The new UmacDelegate is implemented with a method containing one parameter of type UmacCredentials. The UmacCredentials has two properties, 'Name' of type string, and 'Type' of type UmacUserType; and one method SetPassword with one parameter of type SecureString. The use of UmacUserType.Project indicates a UMAC scope of project whereas the use of UmacUserType.Global indicates a UMAC scope of application (i.e. controlled by a UMAC server).

Program code

 Copies the following program code to clipboard.

```

public struct UmacCredentialsParameters
{

    public UmacUserType Type;

```

```

        public string Name;

        public SecureString Password;
    }

    private static void UMACProtectedProjectOpen(string
projectFilePath,
UmacCredentialsParameters umacParameters)
    {
        try
        {
            void MyUmacDelegate(UmacCredentials
umacCredentials)
            {
                umacCredentials.Type = umacParameters.Type;
umacCredentials.Name = umacParameters.Name;

                umacCredentials.SetPassword(umacParameters.Password);
            }

            // Refer Connecting to the TIA Portal section
            TiaPortal tiaPortal = new TiaPortal();

            Project project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath), MyUmacDelegate);

            if (project != null)
            {
                try
                {
                    Console.WriteLine(project.Name);
Console.WriteLine(project.Version);
                }

                finally
                {
                    project.Close();
                }
            }
        }
    }

```

```

    }

    catch (Exception ex)
    {
        Console.WriteLine(ex.ToString());
    }
}

```

Opening a UMAC protected project using new authentication

Before TIA Portal Openness V17, You can open the protected project by passing the user name and password through Umac delegate in Open Project API.

With TIA Portal Openness V17, UMAC have introduced new authentication mechanism by which you will able to open a protected project. The following new authentication mechanisms are Desktop Single Sign-On (SSO) and Anonymous User.

TIA Portal Openness V17 supports the following authentication mechanisms. .The Openness implementation is independent of settings in TIA Portal.

- Desktop Single Sign-On (SSO)
- Anonymous User
- Interactive log on
- Credential (Existing From TIA Portal V15.1)

In order to accommodate the additional authentication methods, Openness have introduced a new event name "Authentication" in TIA Portal object. The changes are introduced as a part of V17 Engineering. The Event "Authentication" when registered by the user in the Openness script, will be executed when open project is called. The Event "Authentication" will be executed only for Open Protected Project.

 Copies the following program code to clipboard.

```

private static void OnAuthentication(object sender,
AuthenticationEventArgs e)
{
    // Refer Connecting to the TIA Portal section
    var tiaPortal = new TiaPortal();

    tiaPortal.Authentication += OnAuthentication;

    e.AuthenticationTypeProvider =
AuthenticationTypeProvider.Credentials;
}

```

You are able to use the Additional Authentication Methods using the AuthenticationEventArgs event args. Using AuthenticationTypeProvider, the mode of authentication is provided. From the above example, the credential mode is set.

Below are the values for AuthenticationTypeProvider.

- AuthenticationTypeProvider.DesktopSso : Open project with Single sign on user, no password asked
- AuthenticationTypeProvider.Anonymous : Open project with anonymous user, no password asked
- AuthenticationTypeProvider.Interactive : Open project with Interactive logon, User name and password to be given in user interface
- AuthenticationTypeProvider.Credentials : Open Project with User name and password given.

The event is applicable for opening the project in multiuser user environment. The above authentication methods are applicable when a protected project is opened in local session or server view.

If there are exceptions for example, a single sign-on session or an invalid user name / password scenarios are handled by EngineeringTargetInvocationException.

Opening multiple projects

You can open one primary project and multiple secondary projects at a time in an instance of the TIA Portal. You can open a project as a primary or a secondary project. If a project is opened as primary, then this project is represented in the project navigation if the Openness application is attached to a TIA Portal and project is opened as secondary, then this project is not reflected in the user interface. Secondary projects are always opened as read-only. A user with read and write privileges to UMAC projected project will have read-only privileges when project is opened as secondary. A primary project does not need to be open in order to open a secondary project.

Any opened projects can be enumerated by using the ProjectComposition available on the TiaPortal instance. The order of the projects in the composition will be determined by the order in which the projects were opened. If a project is closed, the index of all projects will be recalculated.

Program code

 Copies the following program code to clipboard.

```
private static void OpeningMultipleProjects()
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    Project project1 = tiaPortal.Projects.Open(new
    FileInfo(@"D:\Project_1\Project_1.apXX"), null,
    ProjectOpenMode.Primary);
```

```

        Project project3 = tiaPortal.Projects.Open(new
FileInfo(@"D:\Project_3\Project_3.apXX"), null,
ProjectOpenMode.Secondary);

        bool isPrimary = project3.IsPrimary;
    }

```

Opening projects created with previous versions

Use the `OpenWithUpgrade` method to open a project which was created with the previous version of TIA Portal. The method will create a new, upgraded project and open it.

If you access a project created with an elder version than the previous, an exception will be returned.

Note

If you access a project created with the current version the project will just be opened.

Program code

Modify the following program code to open a project via the `OpenWithUpgrade` method:

 Copies the following program code to clipboard.

```

private static void ProjectOpenWithUpgrade()
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    Project project = tiaPortal.Projects.OpenWithUpgrade(new
FileInfo(@"D:\Some\Path\Here\Project.apXX"));

    if (project != null)
    {
        try
        {
            Console.WriteLine("Project Name: " + project.Name);
        }
        finally
        {
            project.Close();
        }
    }
}

```

```

    }

}
}

```

Program code for UMAC protected project

You can also open a UMAC protected which has been created with a previous version of TIA Portal. An overload function of `OpenWithUpgrade` takes an additional parameter of type `UmacDelegate`. `OpenWithUpgrade` is also supported for secondary projects.

 Copies the following program code to clipboard.

```

private static void UMACProtectedProject(UmacDelegate
MyUmacDelegate)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    Siemens.Engineering.Project project =
    tiaPortal.Projects.OpenWithUpgrade(new
        FileInfo(@"D:\Project_1\Project.apXX"), MyUmacDelegate);
}

```

1.4.3.2 Creating a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)

Introduction

Projects can be created via TIA Portal Openness API

- by calling the `Create` method on `ProjectComposition`
- by calling the `Create` method on `IEngineeringComposition`

ProjectComposition.Create

Modify the following program code:

 Copies the following program code to clipboard.

```

private static void ProjectCreate()

```

```

{
    TiaPortal tiaPortal = new TiaPortal();

    ProjectComposition projectComposition =
tiaPortal.Projects;

    DirectoryInfo targetDirectory = new
DirectoryInfo(@"D:\TiaProjects");

    // Create a project with name MyProject

    Project project =
projectComposition.Create(targetDirectory, "MyProject");
}

```

According to this example

- a folder "D:\TiaProjects\MyProject" will be created.
- a project file "D:\TiaProjects\MyProject\MyProject.aPXX" will be created.

Note

About parameter targetDirectory

The parameter targetDirectory can also represent an UNC (Universal Naming Convention) path, therefore a project can also be created on a network shared drive.

EngineeringComposition.Create

Modify the following program code:

 Copies the following program code to clipboard.

```

public struct ProjectCreateParameters
{
    public DirectoryInfo TargetDirectory;
    public string Name;
    public string Author;
    public string Comment;
}

private static Project
ProjectAttributeCreate(ProjectCreateParameters parameters)
{

```

```

try
{
    TiaPortal tiaPortal = new TiaPortal();

    ProjectComposition projectComposition =
tiaPortal.Projects;

    // Prepare the create parameters

    var createParameters = new List<KeyValuePair<string,
object>>

    {
        new KeyValuePair<string, object>("TargetDirectory",
parameters.TargetDirectory), // Mandatory

        new KeyValuePair<string, object>("Name", parameters.Name)
// Mandatory

    };

    if (!string.IsNullOrEmpty(parameters.Author))
    {
        createParameters.Add(new KeyValuePair<string, object>
("Author", parameters.Author)); // Optional

        if (!string.IsNullOrEmpty(parameters .Comment))
        {
            createParameters.Add(new KeyValuePair<string, object>
("Comment", parameters.Comment)); // Optional

        }

        // Create a project with both mandatory and optional
parameters

        return ((IEngineeringComposition)projectComposition)
.Create(typeof(Project), createParameters) as Project;

    }

    catch (Exception ex)
    {
        Console.WriteLine($"Exception of type {ex.GetType()}
occurred: {ex.Message}");

        return null;
    }
}

```

```

    }
}

```

According to this example:

- a folder "D:\TiaProjects\MyProject" will be created.
- a project file "D:\TiaProjects\MyProject\MyProject.aPXX" will be created with project attributes Author as "Bob" and Comment as "This project was created with openness".

Parameters for creating project with optional project attributes

Parameter	Data Type	Is Mandatory	Description
Author	String	No	Author of a project.
Comment	String	No	Comment for the project.
Name	String	Yes	Name of a project,
TargetDirectory	DirectoryInfo	Yes	Directory that will contain the created project folder.

1.4.3.3 Accessing general settings of the TIA Portal

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to access general settings of the TIA Portal:

- Current user interface language
- "Search in project" option to create the search index needed for searching within a project.

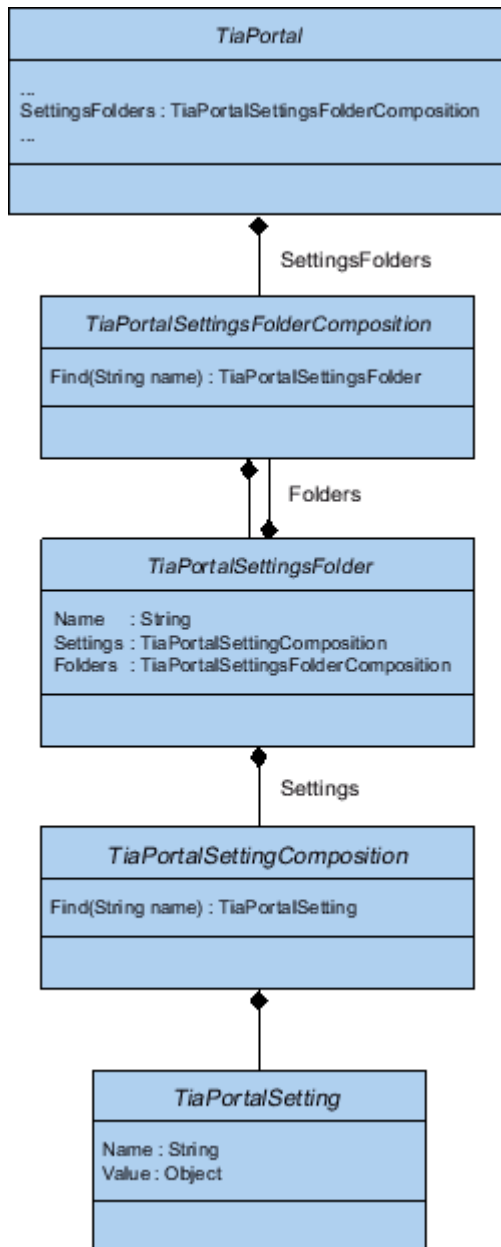
The following table shows the details of the accessible settings in the "General" section of the TIA Portal settings. The `TiaPortalSettingsFolder` instance will have the name "General".

Setting name	Data type	Writeable	Description
--------------	-----------	-----------	-------------

"SearchInProject"	System.Boolean	r/w	Enables or disables the creation of the search index needed for searching within a project.
"UserInterfaceLanguage"	System.CultureInfo	r/w	Indicates the active user interface language of the TIA Portal or the specification of the active user interface language.

The access to these settings is provided via the `TiaPortalSettingsFolder` class. The `TiaPortalSettingsFolder` class will be accessible via the `Settings` attribute on the `TiaPortal` class.

The following figure shows the specific settings in TIA Portal Openness:



Program code: Search in project

Modify the following program code to activate/deactivate the "Search in project" option.

 Copies the following program code to clipboard.

```

private static void SetSearchInPoject()
{
    TiaPortal tiaPortal = new TiaPortal();

    TiaPortalSettingsFolder generalSettingsFolder =
    tiaPortal.SettingsFolders.Find("General");

    TiaPortalSetting searchSetting =
  
```

```

        generalSettingsFolder.Settings.Find("SearchInProject");
        if ((bool)searchSetting.Value)
        {
            searchSetting.Value = false;
        }
    }
}

```

Program code: User interface language

Modify the following program code to access the current user interface language.

 Copies the following program code to clipboard.

```

private static void SetUILanguage()
{
    TiaPortal tiaPortal = new TiaPortal();
    TiaPortalSettingsFolder generalSettingsFolder =
        tiaPortal.SettingsFolders.Find("General");
    TiaPortalSetting UILanguageSetting =
        generalSettingsFolder.
            Settings.Find("UserInterfaceLanguage");
    if ((CultureInfo)UILanguageSetting.Value) !=
        CultureInfo.GetCultureInfo("de-DE")
    {
        UILanguageSetting.Value =
            CultureInfo.GetCultureInfo("de-DE");
    }
}

```

See also

[Opening a project](#)

1.4.3.4 Archiving and Retrieving a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a Project](#)
- A project is save
See [Saving a Project](#)

Introduction

You can use the TIA Portal Openness to archive the opened and saved project state prior to further any changes and then later you can retrieve the archived project.

Archiving a project

You can archive a TIA Portal project using TIA Openness API available on Siemens.Engineering.Project object.

 Copies the following program code to clipboard.

```
public void Archive(System.IO.DirectoryInfo targetDirectory,
string targetName, Siemens.Engineering.ProjectArchivationMode
archivationMode)
```

'targetName' is the name of the file created may it be archived or non archived. This file may or may not contain any file extensions. For ProjectArchivationMode value as Compressed and DiscardRestorableDataAndCompressed, the archived file name is as it is provided by you. If you don't provide any extension or extension apart from "zap15_1" or "zap14" etc., then this archived project could not be retrieved from TIA Portal outside the Openness API. If you don't not choose Compressed or DiscardRestorableDataAndCompressed the result will be the default file structure of a project of the current version.

Note

You must have saved the project before calling Archiving API. In case the project contains any unsaved changes, archive would throw an EngineeringTargetException.

Project ArchivationMode

The **ProjectArchivationMode** enumeration have four values.

ProjectArchivationMode	Description
None	• No special action are taken with the orginial files. Mode is similiar to a "save as" operation.

	• No compressed zip file is created in this mode. • The difference with SaveAs in this case is that the Archive will not change the persistence location to the new Archived folder whereas SaveAs does that
DiscardRestorableData	• The file storage stores the project in an internal data file and this file grows whenever a project data modification happens. In the case of DiscardRestorableData mode this data file is reorganized (only latest version of the objects are stored and history is removed from the file) and Intermediate data, the files of the IM directory and tmp directory are not copied to the archive location. • No compressed zip file is created in this mode.
Compressed	• The tmp project folder structure, created by Archiving, is compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.
DiscardRestorableDataAndCompressed	• In the tmp project folder structure, created by Archiving, the restorable data is discarded and then compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

Program code: Archiving a project

Modify the following program code to access the archive project without an extension:

 Copies the following program code to clipboard.

```
public void ArchivingProjectWithoutExtension()
{
```

```

        var tiaPortal = new
TiaPortal (TiaPortalMode.WithoutUserInterface);

        var projectFilePath = @"E:\Sample1\Sample1.ap15_1";

        var project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath));

        var archiveDirectory = @"E:\Archive";

        project.Archive(new DirectoryInfo(archiveDirectory),
            "ArchiveProjectNameWithoutExtension",
            ProjectArchivationMode.Compressed);
    }

```

Modify the following program code to access the archive project with an extension as '.archive':

 Copies the following program code to clipboard.

```

public void ArchivingProjectWithExtension()
{
    var tiaPortal = new
TiaPortal (TiaPortalMode.WithoutUserInterface);

    var projectFilePath = @"E:\Sample1\Sample1.ap15_1";

    var project = tiaPortal.Projects.Open(new
FileInfo(projectFilePath));

    var archiveDirectory = @"E:\Archive";

    project.Archive(new DirectoryInfo (archiveDirectory),
        "ArchiveProjectName.archive",
        ProjectArchivationMode.Compressed);
}

```

Note

You are free to use any extension. The behavior of Archive will be same with/without extension.

Retrieving a project

You can use the Openness API to retrieve an archived TIA Portal project. You can only retrieve a compressed archive. For Archived project with 'ProjectArchivationMode.None' or 'ProjectArchivationMode.DiscardRestorableData' enumeration value cannot be retrieved by this API. This API is available on the "Siemens.Engineering.ProjectComposition" object.

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project  
Retrieve(System.IO.FileInfo sourcePath,  
System.IO.DirectoryInfo targetDirectory)
```

For retrieving a UMAC protected project an overloaded API with the UmacDelegate is provided for getting the credentials, then the following API definition is used

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project  
Retrieve(System.IO.FileInfo sourcePath,  
System.IO.DirectoryInfo targetDirectory,  
Siemens.Engineering.UmacDelegate umacDelegate)
```

Retrieve also supports ProjectOpenMode that can be specified as "Primary" or "Secondary", then the following API definition is used

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project  
Retrieve(System.IO.FileInfo sourcePath,  
System.IO.DirectoryInfo  
targetDirectory, Siemens.Engineering.UmacDelegate  
umacDelegate, Siemens.Engineering.ProjectOpenMode  
projectOpenMode)
```

Note

umacDelegate could be passed as null if the project is not protected

If the archived project is of a previous TIA Portal version then you have to call the RetrieveWithUpgrade API, then the following API definition is used

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project  
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,  
System.IO.DirectoryInfo targetDirectory);
```

If the archived project is of a previous TIA Portal version and UMAC protected then another overloaded API with UmacDelegate is available, then the following API definition is used.

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory,
Siemens.Engineering.UmacDelegate umacDelegate)
```

If the archived project is of a previous TIA portal version and project open mode also needs to be specified, then the following API definition is used.

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Project
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo
targetDirectory, Siemens.Engineering.UmacDelegate
umacDelegate, Siemens.Engineering.ProjectOpenMode
projectOpenMode)
```

Note

umacDelegate could be passed as null if the project is not protected. If the project is protected then the user credentials of an admin user is required to perform the RetrieveWithUpgrade action

Program code: Retrieving a project

Modify the following program code to access the retrieve project:

 Copies the following program code to clipboard.

```
public void RetrieveProject()
{
    var archivePath = @"E:\Archive\Sample1.zap15_1";
    var retrievedProjectsDirectory = @"E:\RetrievedProjects";
    var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);
    tiaPortal.Projects.Retrieve(new FileInfo(archivePath),
new DirectoryInfo(retrievedProjectsDirectory));
}
```

See also

[Connecting to the TIA Portal](#)

[Opening a project](#)

Saving a project

1.4.3.5 Accessing read-only TIA Portal project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

Using TIA Portal Openness, you can perform select operations while working with a read-only TIA Portal project. You can have access to read-only project, but you will not be able to use full set of features that are available to a user with read-write access. For example, a user with read-only credentials can use Openness to open a UMAC protected project as described in [Opening a project](#). This functionality does not include Reference projects.

The list of Openness features that are available to you while accessing a read-only project can be categorized into two sets of features - Inherent and Enabled non-modifying actions:

Inherent functionality

- GetAttribute(s) or using the getter for any attribute on any accessible object
- GetComposition on any accessible object
- GetService on any accessible object
- Find actions on any accessible object
- Navigation on any accessible object
- Determining the existence of accessible objects and accessing those objects in compositions and associations
- System.Object methods on any accessible object

Enabled non-modifying actions

- Project.Close (...)
- PlcBlock.ShowInEditor ()
- CaxProvider.Export (Device,...)
- CaxProvider.Export (Project,...)

See also

[Opening a project](#)

1.4.3.6 Accessing languages

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

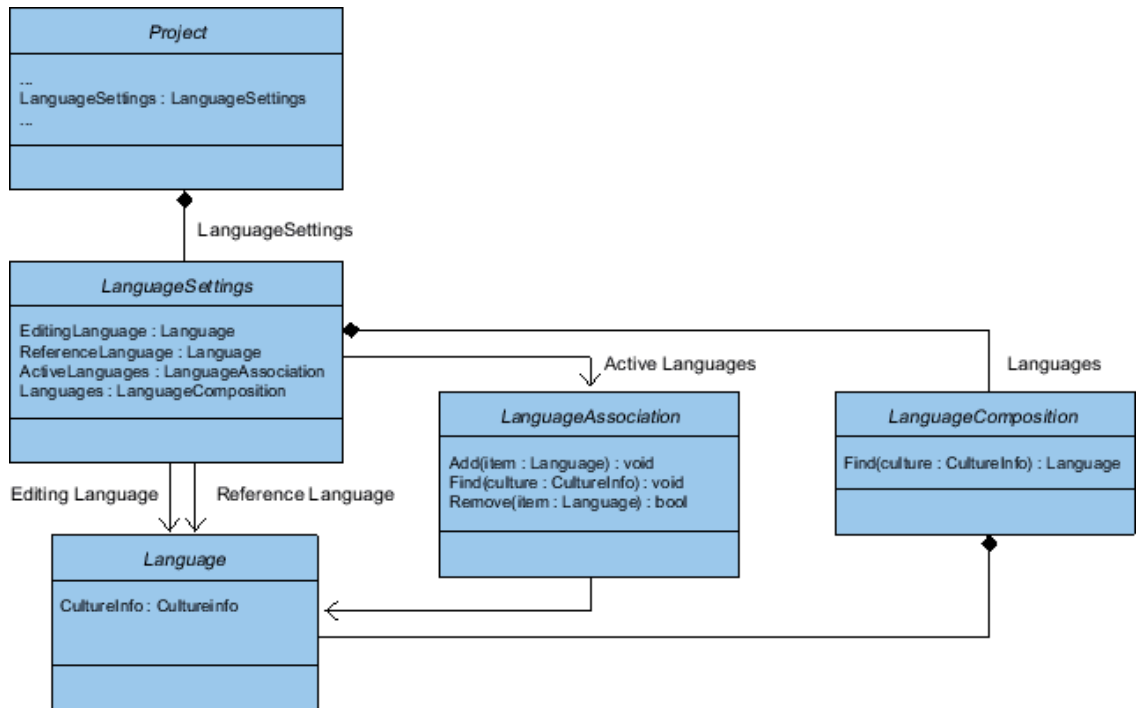
Introduction

In the TIA Portal you can set and manage the project language in the Editor "Project languages".

TIA Portal Openness supports the following access to the project languages:

- Iterating through supported languages.
- Searching through the collection of supported languages by `System.Globalization.CultureInfo`.
- Accessing individual languages. Each Language object will contain a single read-only attribute `Culture` of type `System.Globalization.CultureInfo`.
- Accessing a collection of active languages.
- Searching through the collection of active languages by `System.Globalization.CultureInfo`.
- Adding a language to a collection of active languages.
- Removing a language from a collection of active languages.
- Setting an editing language.
- Setting a reference language.

The functionalities are provided by the `LanguageSettings` object. The following figure shows model, which is provided by TIA Portal Openness:



Program code: Setting languages

Modify the following program code to set a language. If you set an inactive language via TIA Portal Openness, the language will be added to the active languages collection.

 Copies the following program code to clipboard.

```

private static void SettingLanguages()
{
    TiaPortal tiaPortal = new TiaPortal();
    Project project = tiaPortal.Projects.
    Open(new FileInfo(@"D:\Project_3\Project_2.apXX"));
    LanguageSettings languageSettings =
project.LanguageSettings;
    LanguageComposition supportedLanguages =
languageSettings.Languages;
    LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;
    Language supportedGermanLanguage = supportedLanguages.
Find(CultureInfo.GetCultureInfo("de-DE"));
    activeLanguages.Add(supportedGermanLanguage);
}
  
```

```

        languageSettings.EditingLanguage =
supportedGermanLanguage;

        languageSettings.ReferenceLanguage =
supportedGermanLanguage;
    }

```

Program code: Deactivating an active language

Modify the following program code to deactivate an active language. If you deactivate a language which is used as reference or editing language the language selected will be consistent with the behavior in the user interface.

 Copies the following program code to clipboard.

```

private static LanguageSettings
DeactivatingLanguages(TiaPortal tiaPortal, string
projectPath)
{
    try
    {
        Project project = tiaPortal.Projects.Open(new
FileInfo(projectPath));

        LanguageSettings languageSettings =
project.LanguageSettings;

        LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;

        Language activeGermanLanguage = activeLanguages.
Find(CultureInfo.GetCultureInfo("de-DE"));

        activeLanguages.Remove(activeGermanLanguage);

        return languageSettings;
    }

    catch(Exception ex)
    {
        Console.WriteLine($"Exception of type {ex.GetType()}
occurred: {ex.Message}");

        return null;
    }
}

```

```
}
```

See also

[Hierarchy of hardware objects of the object model](#)

1.4.3.7 Determining the object structure and attributes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Application

You can determine the navigation structure through the object hierarchy with the `IEngineeringObject` interface. The result is returned as a list:

- Child objects
- Child compositions
- All attributes

Signature

Use the `GetAttributeInfos` method to determine attributes.

```
IList<EngineeringAttributeInfo> IEngineeringObject.GetAttributeInfos();
```

Program code: Determining objects or compositions

Use the following program code to display all composition names:

 [Copies the following program code to clipboard.](#)

```
private static void
DisplayCompositionInfos(IEngineeringObject obj)
{
    IList<EngineeringCompositionInfo> compositionInfos =
obj.GetCompositionInfos();

    foreach (EngineeringCompositionInfo compositionInfo in
compositionInfos)
    {
```

```

        Console.WriteLine (compositionInfo.Name);
    }
}

```

Modify the following program code if you know the return value:

 Copies the following program code to clipboard.

```

private static DeviceItemComposition
GetDeviceItemComposition(Device device)
{
    IEngineeringCompositionOrObject composition =
    ((IEngineeringObject) device).GetComposition("DeviceItems");

    DeviceItemComposition deviceItemComposition =
    (DeviceItemComposition)composition;

    return deviceItemComposition;
}

```

Program code: Determining attributes

Modify the following program code to return attributes of an object with specific access rights in a list:

 Copies the following program code to clipboard.

```

private static string
DisplayAttributenInfos(IEngineeringObject obj)
{
    IList<EngineeringAttributeInfo> attributeInfos =
obj.GetAttributeInfos();

    foreach (EngineeringAttributeInfo attributeInfo in
attributeInfos)
    {
        Console.WriteLine("Attribute: {0} - AccessMode {1} ",
attributeInfo.Name,
attributeInfo.AccessMode);

        switch (attributeInfo.AccessMode)
        {

```

```

        case EngineeringAttributeAccessMode.Read:
Console.WriteLine

        ("Attribute: {0} Read Access", attributeInfo.Name);

        break;

        case EngineeringAttributeAccessMode.Write:
Console.WriteLine

        ("Attribute: {0} Write Access", attributeInfo.Name);

        break;

        case EngineeringAttributeAccessMode.Read |
EngineeringAttributeAccessMode.Write:

        Console.WriteLine("Attribute: {0} - Read and Write
Access",attributeInfo.Name);

        break;

    }

}

Object nameAttribute = obj.GetAttribute("Name");

string name = nameAttribute?.ToString();

return name;

}

```

1.4.3.8 Access software target

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Program code

Modify the following programm code to make a software target available:

 [Copies the following program code to clipboard.](#)

```

private static void AccessSoftwareTarget(DeviceItem
deviceItem)

```

```

{
    SoftwareContainer softwareContainer =
    ((IEngineeringServiceProvider)deviceItem).
    GetService<SoftwareContainer>();
    if (softwareContainer != null)
    {
        Software software = softwareContainer.Software;
    }
}

```

Modify the following program code to access the software attributes:

 Copies the following program code to clipboard.

```

private static void AccessSoftwareAttribute(DeviceItem
deviceItem)
{
    SoftwareContainer softwareContainer =
    ((IEngineeringServiceProvider)deviceItem).
    GetService<SoftwareContainer>();
    if (softwareContainer != null)
    {
        PlcSoftware software = softwareContainer.Software as
PlcSoftware;
        string name = software.Name;
    }
}

```

1.4.3.9 Accessing and enumerating multilingual texts

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

Multilingual texts in the TIA Portal are e. g. Project.Comment, PlcTag.Comment etc. In TIA Portal Openness, the multilingual texts are represented by the `MultilingualText` object. A `MultilingualText` object is composed of `MultilingualTextItemComposition`.

`MultilingualTextItemComposition` supports the following `Find` method:

- `Find(<language: Siemens.Engineering.Language>):MultilingualTextItem`

Each `MultilingualTextItem` provides the following attributes:

Attribute name	Data type	Writ-able	Description
Language	Siemens.Engineering.Language	r/o	Language of this item.
Text	System.String	r/w	Text provided for this language.

Program code: Set multilingual text

 Copies the following program code to clipboard.

```
private static void SettingMultilingualText(Project project)
{
    Language englishLanguage =
project.LanguageSettings.Languages.
    Find(new CultureInfo("en-US"));
    MultilingualText comment = project.Comment;
    MultilingualTextItemComposition mltItemComposition =
comment.Items;
    MultilingualTextItem englishComment =
mltItemComposition.Find(englishLanguage);
    englishComment.Text = "English comment";
}
```

Program code: Set multilingual text for devices

Modify the following program code to set the multilingual text for devices and device items:

 Copies the following program code to clipboard.

```
private static void SettingMultilingualTextDevice(Device
device, Project project)
```

```

{
    var mltObject = device.GetAttribute("CommentML");
    MultilingualText multilingualText =
(MultilingualText)mltObject;
    if (multilingualText != null)
    {
        Language englishLanguage =
project.LanguageSettings.Languages.
        Find(new CultureInfo("en-US"));
        MultilingualTextItem multilingualTextItem =
multilingualText.Items.
        Find(englishLanguage);
        if (multilingualTextItem != null)
        {
            multilingualTextItem.Text = "comment";
        }
    }
}

```

1.4.3.10 Updating project properties

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to update the simulation support and virtual PLC support property of a project in order to create a plc program. The setting for simulation support and virtual PLC support property is available for virtual SINUMERIK controller and SIMATIC PLCs simulated with PLCSIM Advanced .

The following attribute is supported to access the "simulation support" and "Virtual PLC support" property of a TIA Portal project. The API is available on the "Siemens.Engineering.Project" object.

Attribute name	Data type	Access	Description
IsSimulationDuringBlockCompilationEnabled	System.Boolean	Read / Write	To indicate whether support for simulation during block compilation is enabled for the project
IsVirtualPlcDuringBlockCompilationEnabled	System.Boolean	Read / Write	To indicate whether support for virtual plc during block compilation is enabled for project

Program code

Modify the following program code to access the attribute value:

 Copies the following program code to clipboard.

```
private static void UpdateProjectProperty(Project project)
{
    // Read the attribute value
    var attributeValue =
project.IsSimulationDuringBlockCompilationEnabled;
    var attributeValueForVirtualPLC =
project.IsVirtualPlcDuringBlockCompilationEnabled;
    //Write the attribute value to false
    project.IsSimulationDuringBlockCompilationEnabled =
false;
    //Write the attribute value to True
    project.IsVirtualPlcDuringBlockCompilationEnabled = true;
}
```

See also

[Opening a project](#)

1.4.3.11 Read project related attributes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

See [Connecting to the TIA Portal](#)

- A project is open.

See [Opening a project](#)

Introduction

By using this function you can get project related attributes from the TIA Portal Openness API. The provided information contains project attributes, project history and products utilized by the project.

Project attributes

The project attributes provide the following information:

Attribute name	Data type	Writeable	Description
Author	System.String	r/o	The author of the project
Comment	Siemens.Engineering.MultilingualText	r/o	The comment of the project
Copyright	System.String	r/o	The copyright statement of the project
CreationTime	System.DateTime	r/o	The time when project has been created
Family	System.String	r/o	The family of the project
IsModified	System.Boolean	r/o	Returns true if the project has been modified
LanguageSettings	Siemens.Engineering.LanguageSettings	r/o	Handles project languages
LastModified	System.DateTime	r/o	The time when project was last modified
LastModifiedBy	System.String	r/o	Who made the last modification
Name	System.String	r/o	The name of the project

Path	System.IO.FileInfo	r/o	The absolute path of the project
Size	System.Int64	r/o	The size of the project in KB
Version	System.String	r/o	The version of the project

Modify the following program code to access project related attributes:

 Copies the following program code to clipboard.

```
private static void AccessProjectAttributes(Project project)
{
    string author = project.Author;
    string name = project.Name;
    DateTime creationTime = project.CreationTime;
    DateTime modificationTime = project.LastModified;
    string lastModifiedBy = project.LastModifiedBy;
    string version = project.Version;
    MultilingualText comment = project.Comment;
    string copyright = project.Copyright;
    string family = project.Family;
    Int64 size = project.Size;
    LanguageSettings languageSettings =
project.LanguageSettings;
}
```

Modify the following program code to enumerate the project languages:

 Copies the following program code to clipboard.

```
private static void AccessProjectLanguage(Project project)
{
    LanguageComposition languages =
project.LanguageSettings.Languages;
    foreach (Language language in languages)
```

```

    {
        CultureInfo lang = language.Culture;
    }
}

```

Modify the following program code to get comment text:

 Copies the following program code to clipboard.

```

private static void RetrieveCommentText(Project project)
{
    Language english =
project.LanguageSettings.ActiveLanguages.
    Find(CultureInfo.GetCultureInfo("en-US"));
    MultilingualText projectComment = project.Comment;
    MultilingualTextItem textItem =
project.Comment.Items.Find(english);
    string text = textItem.Text;
}

```

Project history

The project history is a composition of `HistoryEntry` objects, which contain the following information:

Attribute name	Data type	Writeable	Description
Text	System.String	r/o	The event description
DateTime	System.DateTime	r/o	The time when the event was occurred

Modify the following program code to enumerate through `HistoryEntries` and access their attributes:

 Copies the following program code to clipboard.

```

private static void AccessHistoryEntryAttributes(Project
project)
{

```

```

        HistoryEntryComposition historyEntryComposition =
project.HistoryEntries;

        foreach (HistoryEntry historyEntry in
historyEntryComposition)
        {
            string entryText = historyEntry.Text;
            DateTime entryTime = historyEntry.DateTime;
        }
    }
}

```

Note

The text attribute of `HistoryEntry` contains a string in the same language as UI. If a TIA Portal Openness application is attached to a TIA Portal with no UI, the string is always in English

Used Products

The object `UsedProduct` includes the following information:

Attribute name	Data type	Writeable	Description
Name	System.String	r/o	The name of the product used
Version	System.String	r/o	The version of the product

Modify the following program code to enumerate through `UsedProduct` and access the attributes.

 Copies the following program code to clipboard.

```

private static void AccessUsedProduct(Project project)
{
    UsedProductComposition usedProductComposition =
project.UsedProducts;

    foreach (UsedProduct usedProduct in
usedProductComposition)
    {

```

```

        string productName = usedProduct.Name;

        string productVersion = usedProduct.Version;
    }
}

```

1.4.3.12 Deleting project graphics

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Program code

Modify the following program code to delete a project graphics:

 Copies the following program code to clipboard.

```

//Deletes a single project graphic entry
public static void DeletesSingleProjectGraphicEntry(Project
project)
{
    MultiLingualGraphicComposition graphicsAggregation =
project.Graphics;

    MultiLingualGraphic graphic =
graphicsAggregation.Find("Graphic XYZ");

    graphic.Delete();
}

```

1.4.3.13 Compiling a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)
- All devices are "Offline".

Application

The API interface supports the compilation of devices and program blocks. The compilation result is returned as an object. Depending on type of the object HW or SW or HW/SW compilation will be provided. The following object types are supported:

- Device - HW & SW
- Device with failsafe CPU - SW with switched-off F-activation property
- DeviceItem - HW
- CodeBlock - SW
- DataBlock - SW
- HmiTarget - SW
- PlcSoftware - SW
- PlcType - SW
- PlcBlockSystemGroup - SW
- PlcBlockUserGroup - SW
- PlcTypeSystemGroup - SW
- PlcTypeUserGroup - SW

Note

Time stamp format

All time stamps are in UTC. If you want to see the local time you can use `DateTime.ToLocalTime()`.

Note

If a password has been assigned to the safety program, you are required to a log on to the safety program to compile software changes. Otherwise, the process is canceled and an exception is triggered.

Signature

Use the `ICompilable` method for compilation.

```
ICompilable compileService =  
iEngineeringServiceProvider.GetService<ICompilable>();  
CompilerResult result = compileService.Compile();
```

Note

All devices must be "Offline" before you start compiling.

Program code

Modify the following program code to compile the software changes of an object of the type HmiTarget:

 Copies the following program code to clipboard.

```
public static void CompileHmiTarget(HmiTarget hmiTarget)
{
    ICompilable compileService =
hmiTarget.GetService<ICompilable>();
    CompilerResult result = compileService.Compile();
}
```

Modify the following program code to compile the software changes of an object of the type PlcSoftware:

 Copies the following program code to clipboard.

```
public static void CompilePlcSoftware(PlcSoftware
plcSoftware)
{
    ICompilable compileService =
plcSoftware.GetService<ICompilable>();
    CompilerResult result = compileService.Compile();
}
```

Modify the following program code to compile the software changes of an object of the type CodeBlock:

 Copies the following program code to clipboard.

```
public static void CompileCodeBlock(PlcSoftware plcSoftware)
{
    CodeBlock block =
plcSoftware.BlockGroup.Blocks.Find("MyCodeBlock") as
CodeBlock;
    if (block != null)
    {
```

```

        ICompilable compileService =
block.GetService<ICompilable>();

        CompilerResult result = compileService.Compile();

    }

}

```

Modify the following program code to evaluate the compilation result:

 Copies the following program code to clipboard.

```

private void WriteCompilerResults(CompilerResult result)
{
    Console.WriteLine("State:" + result.State);

    Console.WriteLine("Warning Count:" +
result.WarningCount);

    Console.WriteLine("Error Count:" + result.ErrorCount);

    RecursivelyWriteMessages(result.Messages);
}

private void
RecursivelyWriteMessages(CompilerResultMessageComposition
messages, string indent = "")
{
    indent += "\t";

    foreach (CompilerResultMessage message in messages)
    {
        Console.WriteLine(indent + "Path: " + message.Path);

        Console.WriteLine(indent + "DateTime: " +
message.DateTime);

        Console.WriteLine(indent + "State: " + message.State);

        Console.WriteLine(indent + "Description: " +
message.Description);

        Console.WriteLine(indent + "Warning Count: " +
message.WarningCount);

        Console.WriteLine(indent + "Error Count: " +
message.ErrorCount);

        RecursivelyWriteMessages(message.Messages, indent);
    }
}

```

```
}  
  
}
```

1.4.3.14 Modifying project in UMAC

Introduction

You can use the TIA Portal Openness to modify project in user management and access right. With this Function Right , When a project is protected, "Modify Project via Openness API" can be assigned to a role which in turn can be assigned to user.

There are certain behavior changes in TIA Portal Openness when you are having / without having this function right in a protected project and affected when a protect project is opened with a particular user credentials. The changes for Openness API is applicable only on protected project and on project which is opened with ProjectOpenMode as primary.The ProjectOpenMode as secondary is not considered.

Changes in behaviors with function right

- When you are having "Modify project via Openness API" function right along with Project Read Write rights , you are given with full access to Openness API i.e you can perform any operation using Openness API.
- When you are not having "Modify project via Openness API" function right along with Project Read Write rights / Project Read rights, you are entitled to Read only operations in Openness API. The examples of read only operations are export, navigation to a device etc. In this scenario, if you want to do an operation like import which is a data oriented operation (Change of project data in TIA portal) , then you would get an exception depending on the function rights assigned to you.
- When there are user interface related Openness API's example (ShowInEditor), the user interface related function rights does not depend on "Modify project via Openness API" , it depends on GUI function right which should be handled by the respective GUI function right. Below API's are the exceptions from point 1 and 2 . Openness API will bypass the Function Right "Modify project via Openness API" and the API's are allowed irrespective of Function Right "Modify project via Openness API" set / not set.
 - project.ShowHwEditor(View.Network)
 - project.ShowHwEditor(View.Topology)
 - device.ShowInEditor(View.Device)
 - plcBlock.ShowInEditor()
 - plcType.ShowInEditor()
 - plcTagTable.ShowInEditor()
 - plcForceTable.ShowInEditor()
 - plcWatchTable.ShowInEditor()

1.4.3.15 Saving a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

To save a project

- Use the `Save()` method to save a project
- Use the `SaveAs()` method to save a project with a different name or in a different directory

Program code

Modify the following program code open and save a project:

 Copies the following program code to clipboard.

```
private static void SaveProject(TiaPortal tiaPortal, Project
project)
{
    //Use the code in the try block to open and save a project
    try
    {
        //please adapt the path and the extension apx
        //to the installed version of TIA Portal
        project = tiaPortal.Projects.Open(new
        FileInfo(@"Some\Path\MyProject.apx"));
        //begin of code for further implementation
        //...
        //end of code'
        project.Save();
    }
    //Use the code in the final block to close a project
    finally
    {
```

```

        if (project != null)
            project.Close();
    }
}

```

Modify the following program code save a project with a different name or in a different location:

 Copies the following program code to clipboard.

```

private static void SaveProjectWithDiffName()
{
    TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);

    FileInfo fileInfoExistingProject = new
FileInfo(@"D:\SampleProjects\SampleProject.apXX");

    DirectoryInfo dirInfoSaveAsProject = new
DirectoryInfo(@"D:\SampleProjects\SampleProjectSaveAs");

    Project sampleProject =
portal.Projects.Open(fileInfoExistingProject );

    sampleProject.SaveAs(dirInfoSaveAsProject);
}

```

1.4.3.16 Closing a project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Program code

Modify the following program code to close a project:

 Copies the following program code to clipboard.

```

public static void CloseProject(Project project)
{

```

```
project.Close();
}
```

1.4.3.17 Export/Import system diagnostics settings

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to export and import system diagnostics settings for a TIA Portal project. The format for export and import is .dat file.

The following service and class are used to export and import the setting for system diagnostics.

Name	Description
SystemdiagnosticsSettingsDataProvider	Service class to provide a method to export and import
SystemdiagnosticsSettingsExportImportResults	To return the result of Export or Import operation

The SystemdiagnosticsSettingsExportImportResultState ENUM has following values:

ENUM	Description	Value
SystemdiagnosticsSettingsExportImportResults	Returns the result whether the file is saved/settings are changed successfully or not.	Error
		Success
		Warning

Program code: Export system diagnostics settings

Modify the following program code to export settings for system diagnostics:

 [Copies the following program code to clipboard.](#)

```

public static void ExportSystemDiagnosticSettings(Project
project)
{
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsDataProvider settingsProvider =
project.
    GetService<Siemens.Engineering.HW.Systemdiagnostics.
    Settings.SystemdiagnosticsSettingsDataProvider>();
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsExportImportResult exportResult
= settingsProvider.
    Export(new FileInfo(@"D:\Temp\SysDiagSettings.dat"));
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsExportImportResultState
resultState = exportResult.State;
}

```

Program code: Import system diagnostics settings

Modify the following program code to import settings for system diagnostics:

 Copies the following program code to clipboard.

```

public static void ImportSystemDignosticSettings(Project
project)
{
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsDataProvider settingsProvider =
project.GetService<Siemens.Engineering.HW.Systemdiagnostics.
    Settings.SystemdiagnosticsSettingsDataProvider>();
    Siemens.Engineering.HW.Systemdiagnostics.Settings.
    SystemdiagnosticsSettingsExportImportResult importResult =
settingsProvider.Import(new
FileInfo(@"D:\Temp\SysDiagSettings.dat"));
    Siemens.Engineering.HW.Systemdiagnostics.Settings.

```

```
SystemdiagnosticsSettingsExportImportResultState  
resultState = importResult.State;  
}
```

1.4.3.18 Identifying cross session object for Hardware & Software

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to uniquely identify engineering objects.

To uniquely identify the engineering objects, you can use the ObjectIdentifierProvider service defined under the Siemens.Engineering namespace.

The TIA Portal Openness supports the following object types for unique identification:

- Siemens.Engineering.HW.Device
- Siemens.Engineering.HW.DeviceItem
- Siemens.Engineering.SW.Blocks.CodeBlock
- Siemens.Engineering.SW.Blocks.DataBlock
- Siemens.Engineering.SW.Blocks.FB
- Siemens.Engineering.SW.Blocks.FC
- Siemens.Engineering.SW.Blocks.OB
- Siemens.Engineering.SW.Blocks.GlobalDB
- Siemens.Engineering.SW.Blocks.InstanceDB
- Siemens.Engineering.SW.Blocks.ArrayDB
- Siemens.Engineering.SW.Tags;
- Siemens.Engineering.SW.Units;
- Siemens.Engineering.SW.TechnologicalObjects.TechnologicalInstanceDB
- Siemens.Engineering.SW.Types.PlcStruct

Method

The ObjectIdentifierProvider service provides the following method details through the TIA Portal Openness:

Method name	Data type	Description
GetIdentifier()	string	To retrieve unique id for particular engineering object
Find()	IEngineeringObject	To retrieve back engineering object from unique id.

Exception

You can encounter the exception in the following cases:

- If you have invoked GetIdentifier method with null as argument.
- If you have invoked GetIdentifier method with disposed object
- If you have invoked Find method with null as argument.

Program code

To access an engineering object from unique id modify the following program code:

 Copies the following program code to clipboard.

```
private static void IdentifyCrossSectionObject(TiaPortal
tiaPortal)
{
    Project project = tiaPortal.Projects.Open(projectPath);
    ObjectIdentifierProvider objectIdentifierProvider =
project.GetService<ObjectIdentifierProvider>();
    //Get the ObjectIdentifierProvider service
    Device device = project.Devices.First();
    string deviceUniqueId =
objectIdentifierProvider.GetIdentifier(device);
    //Fetch unique id for device object
    Device device =
objectIdentifierProvider.Find(deviceUniqueId);
    //Find device object using unique id
    DeviceItem deviceItem =
device.DeviceItems.FirstOrDefault();
```

```

        string deviceItemUniqueId =
objectIdentifierProvider.GetIdentifier(deviceItem);

        //Fetch unique id for deviceItem object

        DeviceItem deviceItem =
objectIdentifierProvider.Find(deviceItemUniqueId);

        //Find deviceItem object using unique id
    }

```

To access unique Identify engineering objects with exception handling modify the following program code:

 Copies the following program code to clipboard.

```

private static void IdentifyCrossSectionObject(TiaPortal
tiaPortal)
{
    Project project = tiaPortal.Projects.Open(projectPath);
    ObjectIdentifierProvider objectIdentifierProvider =
project.
    GetService<ObjectIdentifierProvider>();
    try
    {
        objectIdentifierProvider.GetIdentifier(null);
        //Invoke GetIdentifier null
    }
    catch (EngineeringTargetInvocationException)
    {
        // new use case and new behavior.
    }
    Device device = project.Devices.First();
    device.Delete();
    try
    {
        objectIdentifierProvider.GetIdentifier(device);
        //Invoke GetIdentifier with disposed object
    }
}

```

```

    }

    catch (EngineeringObjectDisposedException)
    {
        // new use case and new behavior.
    }

    try
    {
        objectIdentifierProvider.Find(null);
        //Invoke Find with null
    }

    catch (EngineeringTargetInvocationException)
    {
        // new use case and new behavior.
    }

}

```

1.4.3.19 Managing exception for missing component in project

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Introduction

With the TIA Portal Openness v18, You will get a new Siemens.Engineering.MissingProducts exception when you attempting to open a project using any one of the following APIs in a TIA Portal where some of the products or packages are missing.

- Project Open(FileInfo path)
- Project Open(FileInfo path, UmacDelegate umacDelegate)
- Project Open(FileInfo path, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project OpenWithUpgrade(FileInfo path)
- Project OpenWithUpgrade(FileInfo path, UmacDelegate umacDelegate)

- Project OpenWithUpgrade(FileInfo path, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate)
- Project Retrieve(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate)
- Project RetrieveWithUpgrade(FileInfo sourcePath, DirectoryInfo targetDirectory, UmacDelegate umacDelegate, ProjectOpenMode projectOpenMode)

Before TIA Portal Openness v18, you can get a Siemens.Engineering.EngineeringTargetException exception.

Program code

 Copies the following program code to clipboard.

```
private static void
ManagingExceptionForMissingComponent (ProjectPath
s_ProjectPath)
{
    var projectPath = new FileInfo(s_ProjectPath);
    try
    {
        var project = tiaPortal.Projects.Open(projectPath);
    }
    catch (Siemens.Engineering.MissingProductsException ex)
    {
        Console.WriteLine(ex);
    }
}
```

1.4.3.20 Accessing multilingual support

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to select and enable their preferred project languages. A new service 'Load to PLC' is added to enable the project language in TIA Portal Openness.

LoadToPLC

Method name	Description
GetAttribute(string attributeName)	To get load to plc.
SetAttribute(string attributeName, bool true)	To enable LoadToPlc for a project language.

Note

Exception for SetAttribute() Action:

- Invalid Configuration: EngineeringTargetException is thrown for invalid configurations.
- Exceeding Language Limit: ConfigOpennessUserException is thrown if more than 20 languages are set to LoadToPlc.

Program code: Load to PLC

To get load to PLC modify the following program code:

 Copies the following program code to clipboard.

```
private void GetLoadToPLC(Project project)
{
    // 1. Example to use 'GetAttribute'
    var project = CreatTestProject();// Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
    plc.GetAttribute("AdvancedWebserverLanguageContainer") as
```

```

TableData;

    var rowOrder = 2;

    var index = rowOrder-1;

    //zeroth index based, index 1 represent 2nd row.
    var tableData = multilingualSupportTable.Rows[index];

    //StructuredDataComposition
    if (tableData != null)
    {
        var loadToPlc =
        (bool)tableData.GetAttribute("LoadToPlc");//StructuredData,
    }
}

```

To set load to PLC modify the following program code:

 Copies the following program code to clipboard.

```

private void SetLoadToPLC(Project project)
{
    // 2. Example to use 'SetAttribute'
    var project = CreatTestProject();// Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "",
    ""); // Create Device
    var multilingualSupportTable =
    plc.GetAttribute("AdvancedWebserverLanguageContainer") as
    TableData;

    var tableData = multilingualSupportTable.Rows[index];
    //StructuredDataComposition

    var rowOrder = 2; //Position of language is 2.
    var index = rowOrder-1; // zeroth index based,
    if (tableData != null)
    {
        tableData.SetAttribute("LoadToPlc", true);
        //StructuredData, project language is enabled.
    }
}

```

Language Position

In the new MultilingualSupport, the structured data is based on the project language, making the language position immutable and representing the project language row number.

Note

Exception:

- Setting an invalid language position throws an `EngineeringTargetInvocationException`.

Active

Use Generic APIs to set or get the Active status of a project language.

- Argument Validation for `SetAttribute()`
 - **LoadToPlc Dependency:** `ConfigOpennessUserException` is thrown if Active is set to true while LoadToPlc is false.

- **Language Limit:** Only 3 languages can be set to Active; exceeding this limit throws ConfigOpennessUserException.

 Copies the following program code to clipboard.

```
private void GetActiveStatusProjectLanguage()
{
    // 1. Example to use 'GetAttribute'
    var project = CreatTestProject();
    // Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;

    var rowOrder = 2;
    var index = rowOrder-1;
    //zeroth index based, index 1 represent 2nd row.
    var tableData = multilingualSupportTable.Rows[index];
    //StructuredDataComposition
    if (tableData != null)
    {
        var active = (bool)tableData.GetAttribute("Active");
        // get the active status for a project language
    }

    // 2. Example to use 'SetAttribute'
    var project = CreatTestProject();
    // Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "", "");
    // Create Device
    var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;

    var rowOrder = 2;
```

```

        //Position of language is 2.
        var index = rowOrder-1;
        // zeroth index based,
        var tableData = multilingualSupportTable.Rows[index];
        //StructuredDataComposition
        if (tableData != null)
        {
            tableData.SetAttribute("LoadToPlc", true);
            //First need to set the loadToPlc as true
            tableData.SetAttribute("Active", true);
            //set the project language active as true
        }

```

Display Language

Display languages (e.g., German, English) can be set using the Generic Set/Get API.

- **Exception for SetAttribute()**

- **Active Dependency:** ConfigOpennessUserException is thrown if a display language is enabled while Active is false

 Copies the following program code to clipboard.

```

// 1. Example to use 'GetAttribute'
var project = CreatTestProject();// Create project
var plc = PlugDevice(project, plcTypeIdentifier, "",
"");// Create Device
var multilingualSupportTable =
plc.GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;
var rowOrder = 2;
var index = rowOrder-1; //zeroth index based, index 1
represent 2nd row.
var tableData = multilingualSupportTable.Rows[index];
//StructuredDataComposition
if (tableData != null)
{

```

```

        var german= (bool)tableData.GetAttribute("German");

        // get the display language is enabled for a given row or
structured data
    }

    // 2. Example to use 'SetAttribute'

    var project = CreatTestProject();// Create project
    var plc = PlugDevice(project, plcTypeIdentifier, "",
    ""); // Create Device

    var multilingualSupportTable = plc.
    GetAttribute("AdvancedWebserverLanguageContainer") as
TableData;

    var rowOrder = 2; //Position of language is 2.
    var index = rowOrder-1; // zeroth index based,
    var tableData = multilingualSupportTable.Rows[index];
//StructuredDataComposition
    if (tableData != null)
    {

        tableData.SetAttribute("LoadToPlc", true);//set the
loadToPlc as true

        tableData.SetAttribute("Active", true);//set the active as
truth

        tableData.SetAttribute("German", true);//then only we can
set German as true

    }

```

Creation/Deletion of Structured Data Instance

To add new structured data, a new project language must be added. Directly adding a row to the structured data table is restricted, as these rows are derived from the project languages.

1.4.4 Functions for accessing Teamcenter Gateway

1.4.4.1 Connecting to the Teamcenter Gateway

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Application

You can use the TIA Portal Openness to establish a connection with Teamcenter via TcGateway.

Parameter

Parameter name	Data type	Mandatory	Description
userName	System.String	Yes	Specifies the username in Teamcenter
userPassword	System.SecurityString	Yes	Specifies the password of user in Teamcenter
serverURL	System.String	Yes	Specifies the fully qualified URL of Teamcenter server
teamcenterInstance	System.String	Yes	Specifies the instance of Teamcenter server
userRole	System.String	No	Specifies the user role defined in Teamcenter.
userGroup	System.String	No	Specifies the group to which the user belongs in Teamcenter.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown, if any of the above mandatory parameter is not passed.

A successful connection returns a encrypted TcGatewayConnectionInfo object. In case the connection is not successful, an engineering exception of type TcGatewayException is thrown containing the reason for failure.

Data type	Description
TcGatewayConnectionInfo	The TcGatewayConnection Info object has to be mandatorily passed in all the TcGateway openness APIs. It is used in other TcGateway Openness APIs to validate the source of the call and to validate the active Teamcenter session. The TcGatewayConnectionInfo provides the following connection details: • SessionToken (active connection session token) • Group (Teamcenter user group) • Role (Teamcenter user role)

Program code

 Copies the following program code to clipboard.

```
private void ConnectToTeamcenter(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService
    method.
    TeamcenterConnectionProvider tcGatewayProvider =
    tiaPortal.GetService<TeamcenterConnectionProvider>();
    // Replace with actual username
    string userName = "yourUserName";
```

```

// Replace with actual role
string userRole = "yourRole";
// Replace with actual group
string userGroup = "yourGroup";
// Replace with actual Teamcenter server URL
string serverURL = "yourServerURL";
// Replace with actual instance
string teamcenterInstance = "yourInstance";

//Establish a connection to the Teamcenter using provided
credentials

TcGatewayConnectionInfo connectionInfo =
tcGatewayProvider.Connect(userName, password, userGroup,
    userRole, serverURL, teamcenterInstance);

// Represents the group information retrieved from the
Teamcenter connection

string group = connectionInfo.Group;

// Represents the role information retrieved from the
Teamcenter connection

string role = connectionInfo.Role;
}

```

1.4.4.2 Disconnecting to the Teamcenter Gateway

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Application

You can use the TIA Portal Openness to disconnect the connection between the Teamcenter server and TIA Portal using TcGateway.

Parameter

Parameter	Data type	Mandatory	Description
tcGatewayConnection Info	TcGatewayCon nectionInfo	Yes	Returns the TcGatewayConnection Info object during Connect / ConnectSSO API call.

Program code

 Copies the following program code to clipboard.

```
private void DisconnectToTeamCenter(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService
    method.
    TeamcenterConnectionProvider tcGatewayProvider =
    tiaPortal.GetService<TeamcenterConnectionProvider>();
    //Replace with username
    string userName = "yourUserName";
    // Replace with user role
    string userRole = "yourUserRole";
    //Replace with user group
    string userGroup = "yourUserGroup";
    //Teamcenter server URL
    string serverURL = "yourServerURL";
    //Replace with Teamcenter Instance
    string teamcenterInstance = "yourInstance";
    //Establish a connection to Teamcenter using provided
    credentials
    TcGatewayConnectionInfo tcgConnectionInfo =
```

```

        tcGatewayProvider.Connect(userName,
password,serverURL,teamcenterInstance, userRole, userGroup);

        // Disconnect to Teamcenter

        tcGatewayProvider.Disconnect(tcgConnectionInfo);
    }

```

See also

[Connecting to the Teamcenter Gateway](#)

1.4.4.3 Connecting to the ConnectSSO

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Application

You can use the TIA Portal Openness to call the ConnectSSO() if an active SSO session is available with below required parameters.

Parameter	Data type	Mandatory	Description
hostURL	System.String	Yes	Specifies the fully qualified Teamcenter server host with port number.
instance	System.String	Yes	Specifies the Teamcenter server instance.
loginURL	System.String	Yes	Specifies the URL for the Login Service configured as part of Teamcenter Security Services.
applicationID	System.String	Yes	Specifies the Application ID

			configured in Teamcenter Application registry via Identity service
--	--	--	---

A successful connection returns an encrypted TcGatewayConnectionInfo object. In case the connection is not successful, an appropriate exception is thrown.

Note

If no active SSO session is available, you will be prompted to enter Teamcenter credentials in browser to create SSO session. The API supports secure card based login via PKI.

Program code

 Copies the following program code to clipboard.

```
//Program code: ConnectSSO using TcGatewayConnectionInfo
private void ConnectingToTheConnectSSO()
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieves the Teamcenter Gateway connection provider
    // from the TiaPortal instance using the GetService method.
    TeamcenterConnectionProvider tcGatewayProvider =
    tiaPortal.GetService<TeamcenterConnectionProvider>();
    // Replace with actual values
    string hostUrl = "host_url";
    string teamcenterInstance = "teamcenter_instance";
    string loginURL = "teamcenter SSO_login_service_url";
    String applicationId = "teamcenter_SSO_application";
    // Connect to Teamcenter SSO
    TcGatewayConnectionInfo connectionInfo =
    tcGatewayProvider.ConnectSSO(hostUrl,teamcenterInstance,loginURL,applicationId);
    // Retrieve connection details
    string group = connectionInfo.Group;
    string role = connectionInfo.Role;
```

```
}
```

1.4.4.4 Saving Project/Global library in Teamcenter

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to save a project/global library via the TcGatewayWorkflowProvider service. This service will be available at Project / Global Library composition.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnection Info	TcGatewayConnection Info	Yes	Returns the tcGatewayConn ectionInfo during connection to Teamcenter. The tcGatewayConn ectionInfo is passed to match with the active connection's tcGatewayConn ectionInfo .

localCacheOption	LocalCacheOption(Enum)	Yes	TcGateway LocalCacheOption to overwrite the TIA project/ Global library in Teamcenter cache.
------------------	------------------------	-----	---

Datatype	Description
Siemens.Engineering.TeamcenterGateway.ItemInfo	The API returns the following ItemInfo object after the project or global library is successfully saved to the Teamcenter: • ItemId [System.String] • RevisionId [System.String] • ItemName [System.String] • ItemType [Siemens.Engineering.TeamcenterGateway.ItemType(Enum)]

Program code

 Copies the following program code to clipboard.

```
//Program code: Saving Project/global library in Teamcenter
using Save()

private void SaveProjectInTeamCenter(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {
        // Define user-specific credentials and connection
details.
```

```

string userName = "name";
string userRole = "role";
string userGroup = "group";
string serverURL = "teamcenter_server_url";
string teamcenterInstance = "instance";

// Connect to Teamcenter using the provided credentials
TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);

// Checkout a specific dataset from Teamcenter for
editing.

tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

// Access the project composition in the TiaPortal
instance

ProjectComposition projectComposition =
tiaPortal.Projects;

// Define the project path for the dataset checked out
earlier

// or download it using the TcGateway API.
string projectPath = "project_path";

// Open the project using the specified project path.
Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));

// Create a device group within the project.
tcgProject.DeviceGroups.Create("Group_1");

// Save the project locally in the TC cache (optional
step).

tcgProject.Save();

// Access the TcGatewayWorkflowProvider from the project
// instance using GetService()

TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();

```

```

        // Choose an appropriate local cache option
        // and save the changes made to the project.
        LocalCacheOption cacheOption =
LocalCacheOption.Overwrite;

        // Choose the appropriate option

        ItemInfo response =
tcgWorkflowProvider.Save(connectionInfo, cacheOption);

        // Check in the dataset back to Teamcenter after making
changes.

        tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }

    catch (TcGatewayException ex)
    {
        // Handle exception or log error
    }
}

```

1.4.4.5 Saving Project/Global library with proxy object

Requirement

- The TIA Portal application is connected to the TIA Portal Openness
See [Connection to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to archive the Project / Global library and then save the archive object to the Teamcenter dataset along with proxy objects (e.g. Plc, Program blocks, UDTs, other supported types).

Once the project / Global Library is successfully saved to Teamcenter, the API return ItemInfo object containing ItemId, RevisionId, ItemName and ItemType details.

Parameter

Parameter name	Data type	Is Mandatory	Description
----------------	-----------	--------------	-------------

tcGatewayConnectio nInfo	TcGatewayConnecti onInfo	Yes	Returns the tcGatewayConnectio nInfo during connection to Teamcenter. The tcGatewayConnectio nInfo has passed has to match with the active connection's tcGatewayConnectio nInfo
localCacheOption	LocalCacheOption(E num)	Yes	TcGateway LocalCacheOption to overwrite TIA project/ Global library in TC cache.

Program code:

 Copies the following program code to clipboard.

```
//Program code: Saving project/ global library in Teamcenter
using SaveWithProxyObject()

private void SaveProjectWithProxyObject(SecureString password
)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {
        // Define user-specific credentials and connection
details.

        string userName = "your_username";

        string userRole = "your_userrole";
```

```

string userGroup = "your_usergroup";
string serverURL = "serverURL";
string teamcenterInstance = "teamcenterInstance";
// Connect to Teamcenter using the provided credentials
TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect
    (userName, password, userGroup, userRole, serverURL,
teamcenterInstance);

// Checkout a specific dataset from Teamcenter for
editing.

tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

// Access the project composition in the TiaPortal
instance

ProjectComposition projectComposition =
tiaPortal.Projects;

// Define theProject path for the checkout dataset
// in above step or download project using TcGateway Load
API

string projectPath = "projectPath";

// Open the project using the specified project path.
Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));

// Create a device group within the project.

TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();

// Save the project with proxy object, choose appropriate
LocalCacheOption

ItemInfo responseWithOverwrite =
tcgWorkflowProvider.SaveWithProxyObject(connectionInfo,
LocalCacheOption.Overwrite);

//OR

ItemInfo responseWithoutOverwrite =
tcgWorkflowProvider.SaveWithProxyObject(connectionInfo,
LocalCacheOption.DoNotOverwrite);

```

```

        // Check in the dataset after operations
        tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
    }
    catch (TcGatewayException ex)
    {
        // Handle TcGatewayException or perform other operations
    }
}

```

See also

[Connecting to the TIA Portal](#)

1.4.4.6 Saving Project/Global library with new item

Requirement

- Connecting to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo for successful connection to the Teamcenter. The tcGatewayConnectionInfo has passed and must match with the active connection's tcGatewayConnectionInfo .
itemId	System.String	Yes	Specifies the item id to which project needs to be saved.

revisionId	System.String	Yes	Specifies the revision id to which project needs to be saved.
localCacheOption	Siemens.Engineering.TeamcenterGateway.LocalCacheOption(Enum)	Yes	Specifies the overwrite local cache project contents if local cache or dataset version of project mismatches with server project version

Program code

 Copies the following program code to clipboard.

```
//Program code: Saving project/global library in Teamcenter
using SaveToItem( )

private void SavingProjectWithSaveToItem(SecureString
password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    // Retrieve TcGatewayConnectionProvider from
    GetService()using tiaPortal instance

    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();

    // Retrieve TcGatewayLockProvider from GetService() using
tiaPortal instance

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {

        // Define user-specific credentials and connection
details

        string userName = "your_userName";
```

```

    string userRole = "your_role";
    string userGroup = "your_group";
    String serverURL = "serverURL";
    String teamcenterInstance = "teamcenterInstance";

    // Connect to Teamcenter using the provided credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup,

    userRole, serverURL, teamcenterInstance);

    tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

    ProjectComposition projectComposition =
tiaPortal.Projects;

    // Define Project path for the checkout dataset in above
step

    // or download project using TcGateway Download API
    String projectPath = "projectPath";

    // Open the project using the specified project path
    Project tcgProject = projectComposition.Open(new
FileInfo(projectPath));

    // Access the TcGatewayWorkflowProvider from the project
    // instance using GetService()

    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();

    // Choose an appropriate local cache option
    // and save the changes made to the project.

    ItemInfo response =
tcgWorkflowProvider.SaveToItem(connectionInfo, "000495", "A",
LocalCacheOption.Overwrite);

    // Check in the dataset back to Teamcenter after making
changes.

    tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
}

```

```

        catch (TcGatewayException ex)
        {
            // Handle exception or log error or other operations
        }
    }
}

```

1.4.4.7 Accessing custom attributes in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Intriduction

You can use the TIA Portal Openness to get and set the custom attributes value in the Teamcenter Gateway.

Parameter

The Teamcenter itemtype refers to T4TiaLibrary,T4TiaProject and subtypes which are present in Teamcenter Datamodel for which custom attributes are mapped in Teamcenter.

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.Teamcenter.Gateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo is passed and has to match with the active connection's tcGatewayConnectionInfo .
itemType	System.String	Yes	Specifies the Teamcenter itemType

After successful execution, the API returns collection of custom attributes if available in the Teamcenter for given itemtype.

Data type	Description
-----------	-------------

<code>IList<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty></code>	Specifies the list of Teamcenter custom property for item or revision. • Name [System.String] • MappingLevel [System.int] • ListOfValueInfo [Siemens.Engineering.TeamcenterGateway.TcPropertyListOfValueInfo] - LowerLimit [System.String] - UpperLimit [System.String] - Values [System.Collections.Generic.IEnumerable<System.String>] • DataType [Siemens.Engineering.TeamcenterGateway.MappedCustomAttributeType(Enum)]
--	--

The Call SetValue API to set value on custom attributes fetched using GetTeamcenterCustomAttributes API along with pass error callback delegate.

Parameter name	Data type	Is Mandatory	Description
value	System.String	Yes	Specifies the value for custom attribute
errorCallback	Siemens.Engineering.TeamcenterGateway.ErrorCallback	Yes	Delegate to handle error which are raised during API invocation/actions.

Program code

 Copies the following program code to clipboard.

```
private void AccessCustomAttributes(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    var tcGatewayConnectionProvider =
    tiaPortal.GetService<TeamcenterConnectionProvider>();
```

```

    var tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {
        // Define user-specific credentials information
        string userName = string.Empty;
        string userRole = "your_role";
        string userGroup = "your_group";
        string serverURL = "serverURL";
        string teamcenterInstance = "teamcenterInstance";
        // Delegate for item details
        ItemDetailsDelegate itemDetailsDelegate = ItemDetailsForCreation;
        void ItemDetailsForCreation(ItemDetails itemDetails)
        {
            itemDetails.ItemId = "...";
            itemDetails.ItemName = "Project30";
            itemDetails.RevisionId = "...";
            itemDetails.TeamcenterItemType = "T4TiaProject";
            itemDetails.Comment = "Project Comments.";
        }

        // Connect to Teamcenter using provided credentials
        TcGatewayConnectionInfo connectionInfo =
tiaPortal.GetService<TcGatewayConnectionProvider>().Connect(userName, password, userGroup,
        userRole, serverURL, teamcenterInstance);

        // Open project or global library in TIA Portal
        var project = tiaPortal.Projects.Open(new
FileInfo(@"~\UserData\Documents\Automation\Project30\Project30.ap18"));

        var tcGatewayWorkflowProviderForProject =
project.GetService<TcGatewayWorkflowProvider>();

        //OR

        var library = tiaPortal.GlobalLibraries.Open(new
FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
OpenMode.ReadWrite);

```

```

// Get the workflow provider for the project or library
var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();

// Get the custom attributes from Teamcenter Project
var attributes = tcGatewayWorkflowProviderForProject.
GetTeamcenterCustomAttributes(connectionInfo,"T4TiaProject");

// Error callback for property value setting
ErrorCallback errorCallback =
PropertyValueSettingErrorCallback;

void PropertyValueSettingErrorCallback(string errorMessage)
{
Console.WriteLine(errorMessage);
}

// Loop through attributes and update values
foreach (var arr in attributes)
{
// Display values from ListOfValueInfo if available
if (arr.ListOfValueInfo != null)
{
foreach (var l in arr.ListOfValueInfo.Values)
{
Console.WriteLine(l);
}
}

switch (arr.DataType)
{
case MappedCustomAttributeType.String:
arr.SetValue("23232323232", errorCallback);
break;

case MappedCustomAttributeType.Char:
arr.SetValue("d", errorCallback);
break;
}
}

```

```

        case MappedCustomAttributeType.Integer:
            arr.SetValue("232", errorCallback);
            break;

        case MappedCustomAttributeType.Float:
            arr.SetValue(23.4f.ToString(), errorCallback);
            break;
    }
}

catch (TcGatewayException ex)
{
    // Handle exception or log error or other operations or
    other operations
}
}

```

1.4.4.8 Saving Project to Teamcenter as new item

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A Project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to save a TIA Portal project in Teamcenter as a new item. The API returns itemInfo object containing the ItemId, RevisionId, ItemType, ItemName after successful creation of item/revision.

Parameter

Parameter name	Data type	Is Mandatory	Description
----------------	-----------	--------------	-------------

tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo has to match with the active connection's tcGatewayConnectionInfo .
itemDetailsDelegate	delegate	Yes	This delegate is defined by caller of API which contains information about the item to be created in Teamcenter: • ItemId [System.String] • RevisionId [System.String] • ItemName [System.String] (Required) • Comments [System.String] • TeamcenterFolder [System.String] • TeamcenterProject [System.String[]] • TeamcenterItemType [System.String] (Required)
customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>	No	Specifies the Item/Revision custom attributes which are saved

			along with Project / Global Library.
--	--	--	--------------------------------------

Program code

 Copies the following program code to clipboard.

```
private void SavingProjectToTeamcenterAsNewItem(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    TeamcenterConnectionProvider tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {
        // Define user specific credentials and connection details
        String userName = string.Empty;
        String userRole = "your_role";
        String userGroup = "your_userGroup";
        String serverURL = "serverURL";
        String teamcenterInstance = "teamcenterInstance";

        // Connect to Teamcenter using the provided credentials
        TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup, userRole, serverURL, teamcenterInstance);

        var project = tiaPortal.Projects.Open(new
FileInfo(@"D:\UserData\Documents\Automation\Project30\Project30.tia"),
OpenMode.ReadWrite);

        var library = tiaPortal.GlobalLibraries.Open(new
FileInfo(@"~\Documents\Automation\Library7\Library7.all8"),
OpenMode.ReadWrite);

        var tcGatewayWorkflowProvider = project.GetService<TcGatewayWorkflowProvider>();
    }
}
```

```

        var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();

        // Define item details
        ItemDetailsDelegate itemDetailsDelegate = (itemDetails) =>
        {
            itemDetails.ItemId = "...";
            itemDetails.ItemName = "Proj_Object";
itemDetails.RevisionId = "...";

            itemDetails.TeamcenterItemType = "T4TiaProject";
            itemDetails.Comment = "Prooject Comments.";
            itemDetails.TeamcenterFolder =
"Home\\TIA_Project\\ConveyourBelt";

            itemDetails.TeamcenterProject = new string[]
{"Project_TIA_1", "Project_TIA_2"};

        };

        // Define error callback
        ErrorCallback errorCallback = (errorMessage) =>
        {
            Console.WriteLine(errorMessage);
        };

        //Save item in Teamcenter with custom attributes.
        var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,
"T4TiaProject");

        foreach (var arr in attributes)
        {
            if (arr.ListOfValueInfo != null)
            {
                foreach (var l in arr.ListOfValueInfo?.Values)
                {
                    Console.WriteLine(l);
                }
            }
        }

```

```

        switch (arr.DataType)
        {
            case MappedCustomAttributeType.String:
                arr.SetValue("23232323232", errorCallback);
                break;
            case MappedCustomAttributeType.Char:
                arr.SetValue("d", errorCallback);
                break;
            case MappedCustomAttributeType.Integer:
                arr.SetValue("232", errorCallback);
                break;
            case MappedCustomAttributeType.Float:
                arr.SetValue(23.4f.ToString(), errorCallback);
                break;
        }

        var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, attributes);

        var itemId = createdItem.ItemId;
        var revisionId = createdItem.RevisionId;
        var itemName = createdItem.ItemName;
        var type = createdItem.ItemType;

        //Checkout created dataset to edit in TIA portal
        tcGatewayLockProvider.CheckoutDataset(connectionInfo,
itemId, revisionId, DatasetType.T4TiaProjectDataset, itemName);

        //Check in dataset after changes to project are
completed

        tcGatewayLockProvider.CheckinDataset(connectionInfo,
itemId, revisionId, DatasetType.T4TiaProjectDataset, itemName);
    }
}

catch (TcGatewayException ex)

```

```

    {
        // Handle exception
    }
}

```

1.4.4.9 Saving project using save to item with proxy object

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to to save a TIA Portal Project with proxy objects into Teamcenter as a new Item revision. The API returns Siemens.Engineering.TeamcenterGateway.ItemInfo.

Program code

 Copies the following program code to clipboard.

```

private static void
RevisionDetailsForCreation(revisionDetails
itemRevisionDetails)
{
    itemRevisionDetails.RevisionId = "B";
    itemRevisionDetails.Comment = "Project Comments for new
revision.";
}

private static void PropertyValueSettingErrorCallback(string
errorMessage)
{
    Console.WriteLine(errorMessage);
}

using Siemens.Engineering.TeamcenterGateway;
{

```

```

        var tcGatewayConnectionProvider= tiaPortal.GetService<TeamcenterConn
    );
    var tcGatewayLockProvider = tiaPortal.GetService<TcGatewayLockProvi
    try
    {
        String userName = ...//User name
        SecureString password = ...;
        //Secure string.
        // Client takes care of encrypting password and openness framework
        // decrypts and passes the decrypted value to the action;
        String userRole = null;//Role
        String userGroup = null//Group
        String serverURL = ...//Teamcenter server URL
        String teamcenterInstance=....//Instance
        RevisionDetailsDelegate revisionDetailsDelegate = RevisionDetailsFor
        TcGatewayConnectionInfo connectionInfo = tcGatewayConnectionProvider
        (userName, password, userGroup, userRole, serverURL,teamcenterInstan
        var project = tiaPortal.Projects.Open
        (new
FileInfo("D:\\UserData\\Documents\\Automation\\tc\\Project30\\Project30.a
        //OR
        //var library = tiaPortal.GlobalLibraries.Open
        (new FileInfo(@"~\Documents\Automation\Library7\Library7.al18"),
OpenMode.ReadWrite);
        //var tcGatewayWorkflowProvider = library.GetService<TcGatewayWorkf
        var tcGatewayWorkflowProvider = project.GetService<TcGatewayWorkflow
        //Save item in Teamcenter with custom attributes.
        var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,
        ErrorCallback errorCallback = PropertyValueSettingErrorCallback;
        foreach (var arr in attributes)
        {

```

```

        if (arr.ListOfValueInfo != null){
            foreach (var l in arr.ListOfValueInfo?.LovValues)
            {
                Console.WriteLine(l);
            }

            switch (arr.DataType)
            {
                case
MappedCustomAttributeType.String:arr.SetValue("23232323232",
errorCallback);
                break;

                case MappedCustomAttributeType.Char:
arr.SetValue("d", errorCallback);
                break;

                case MappedCustomAttributeType.Integer:
arr.SetValue("232", errorCallback);
                break;

                case MappedCustomAttributeType.Float:
arr.SetValue(23.4f.ToString(), errorCallback);
                break;
            }

            var createdItemRevision =
wf.SaveAsNewRevisionWithProxyObject
(connectionInfo, RevisionDetailsDelegate, attributes);
//OR
//var createdItemRevision =
wf.SaveAsNewRevisionWithProxyObject(connectionInfo,
RevisionDetailsDelegate, null);<-- optional
        }

        catch (TcGatewayException)
        {
            // other operations

```

```

    }
    }
}

```

1.4.4.10 Saving Project as new item with proxy object

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to save the TIA Portal project and Global library to the Team Center along with proxy object. The API returns itemInfo object details such as ItemId, RevisionId, ItemType, ItemName after successful creation of new item.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo is passed and should match with the active connection's tcGatewayConnectionInfo .
itemDetailsDelegate	delegate	Yes	This delegate is defined by caller of API which has information about item to be created in Teamcenter:

			• ItemId [System.String] • RevisionId [System.String] • ItemName [System.String] (Required) • Comments [System.String] • TeamcenterFolder [System.String] • TeamcenterProject [System.String[]] • TeamcenterItemType [System.String] (Required)
customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>	No	Specifies the Item/Revision custom attributes saved along with Project / Global Library.

Program code

 Copies the following program code to clipboard.

```
private void
SavingProjectAsNewItemWithProxyObject(SecureString password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    // Retrieve the TcGatewayConnectionProvider and
    TcGatewayLockProvider

    // from GetService() using tiaPortal instance

    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
```

```

        TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

        try
        {
            // Define user-specific credentials and connection

            String userName = string.Empty;
            String userRole = "your_role";
            String userGroup = string.Empty;
            String serverURL = string.Empty;
            String teamcenterInstance = string.Empty;

            // Connection to the Teamcenter using user's credential

            TcGatewayConnectionInfo connectionInfo =
tiaGatewayConnectionProvider.Connect(userName, password, userGroup,
            userRole, serverURL, teamcenterInstance);

            // Open the TIA Portal project file located at the specified path
            var project = tiaPortal.Projects.Open(new
            FileInfo(@"D:\UserData\Documents\Automation\Project30\Project30.ap18

            // Retrieve the workflow provider for the opened project to perform
            // Teamcenter-related operations

            var tcGatewayWorkflowProvider =
project.GetService<TcGatewayWorkflowProvider>();

            //OR

            var library = tiaPortal.GlobalLibraries.Open(new
            FileInfo(@"~\Documents\Automation\Library7\Library7.all8"),
            OpenMode.ReadWrite);

            var tcGatewayWorkflowProviderForLibrary =
library.GetService<TcGatewayWorkflowProvider>();

            ItemDetailsDelegate itemDetailsDelegate = (ItemDetails itemDetails)
            {

                itemDetails.ItemId = "...";
                itemDetails.ItemName = "Proj_Object";
                itemDetails.RevisionId = "...";
                itemDetails.TeamcenterItemType = "T4TiaProject";
            }
        }
    }
}

```

```

        itemDetails.Comment = "Project Comments.";

        itemDetails.TeamcenterFolder =
"Home\\TIA_Project\\ConveyourBelt";

        itemDetails.TeamcenterProject = new string[] { "Project_TIA_1",
"Project_TIA_2" };

    };

    // Save item in Teamcenter with custom attributes.

    // Retrieve custom attributes for a Teamcenter item of type
"T4TiaProject"

    // using the workflow provider

    var attributes =
tcGatewayWorkflowProvider.GetTeamcenterCustomAttributes(connectionInfo,
"T4TiaProject");

    // Set up an error callback delegate for handling property value
setting errors

    ErrorCallback errorCallback = (string errorMessage) =>
Console.WriteLine(errorMessage);

    // Iterate through each attribute in the retrieved custom
attributes

    foreach (var arr in attributes)
    {

        // Display values from ListOfValueInfo if available
        if (arr.ListOfValueInfo != null)
        {

            foreach (var l in arr.ListOfValueInfo?.Values)
            {

                Console.WriteLine(l);

            }

        }

        switch (arr.DataType)
        {

            case MappedCustomAttributeType.String:

                arr.SetValue("23232323232", errorCallback);

```

```

        break;

        case MappedCustomAttributeType.Char:
            arr.SetValue("d", errorCallback);
            break;

        case MappedCustomAttributeType.Integer:
            arr.SetValue("232", errorCallback);
            break;

        case MappedCustomAttributeType.Float:
            arr.SetValue(23.4f.ToString(), errorCallback);
            break;
    }

    //Save the Teamcenter item as a new item with the specified
attributes
    // and proxy object

    var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItemWithProxyObject(connectionInfo,
itemDetailsDelegate, attributes);

    //OR

    //< --optional

    /*var createdItem =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, null);*/

    var itemId = createdItem.ItemId;
    var revisionId = createdItem.RevisionId;
    var itemName = createdItem.ItemName;
    var type = createdItem.ItemType;

    //Checkout created dataset to edit in TIA portal

    tcGatewayLockProvider.CheckoutDataset(connectionInfo, itemId,
revisionId, DatasetType.T4TiaProjectDataset,
itemName);

    //Check in dataset after changes to project are completed

    tcGatewayLockProvider.CheckinDataset(connectionInfo, itemId,
revisionId, DatasetType.T4TiaProjectDataset,

```

```

        itemName);
    }
}

catch (TcGatewayException ex)
{
    // other operations
}
}

```

1.4.4.11 Saving project using save as new revision with proxy object

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use TIA Portal Openness to save the modified TIA project and its internal data as proxy objects into a specific item/revision and dataset within a Teamcenter project, including the proxy objects. This operation returns ItemInfo.

Program code

 Copies the following program code to clipboard.

```

using Siemens.Engineering.TeamcenterGateway;
private static void SaveProjectAsNewRevision()
{
    using (TiaPortal tiaPortal = new
    TiaPortal(TiaPortalMode.WithUserInterface))
    {
        var tcGatewayConnectionProvider=
        tiaPortal.GetService<TeamcenterConnectionProvider>();

        var tcGatewayLockProvider =
        tiaPortal.GetService<TcGatewayLockProvider>();

        try

```

```

{
    String userName = ...; //User name
    SecureString password = ...;
    //Secure string. Client takes care of encrypting password
    // and openness framework decrypts and passes the
    decrypted value to the action;
    String userRole = null; //Role
    String userGroup = null; //Group
    String serverURL = ...; //Teamcenter server URL
    String teamcenterInstance=....; //Instance
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.

    Connect(userName, password, userGroup, userRole,
serverURL,teamcenterInstance);

    tcGatewayLockProvider.CheckoutDataset(connectionInfo ,
    "000495","B",DatasetType.T4TiaProjectDataset,"Project30");

    ProjectComposition projectComposition =
tiaPortal.Projects;

    String projectPath = ...;
    // Project path for the checkout dataset in above step
    // or download project using TcGateway Load API
    Project tcgProject = projectComposition.Open(projectPath);
    TcGatewayWorkflowProvider tcgWorkflowProvider =
tcgProject.GetService<TcGatewayWorkflowProvider>();

    ItemInfo response =
tcgWorkflowProvider.SaveToItemWithProxyObject(connectionInfo ,
"000495","B",

    LocalCacheOption.OverwriteLocalCache);

    //OR if client doesn't want to overwrite the local cache,
    // use DoNotOverwriteLocalCache of LocalCacheOption enum
    //ItemInfo response =
tcgWorkflowProvider.SaveToItemWithProxyObject(connectionInfo ,
"000495","B",

```

```

        // LocalCacheOption.DoNotOverwriteLocalCache);

        tcGatewayLockProvider.CheckinDataset(connectionInfo
        ,"000495","B",DatasetType.T4TiaProjectDataset,"Project30");

    }

    catch(TcGatewayException)

    {

        // other operations

    }

}

}

```

1.4.4.12 Saving Project to Teamcenter as new revision

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Application

You can use the TIA Portal Openness to save the TIA Portal project and Global library in Teamcenter as new revision. The API returns the ItemInfo object such as ItemId, RevisionId, ItemName and ItemType details after successful creation of new revision.

Parameter

Parameter name	Data type	Is Mandatory	Description
----------------	-----------	--------------	-------------

tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo has passed to match with the active connection's tcGatewayConnectionInfo
revisionDetailsDelegate	RevisionDetailsDelegate	No	This delegate is defined by caller of API which has information about revision to be created in Teamcenter
customAttributes	IEnumerable<Siemens.Engineering.TeamcenterGateway.TeamcenterProperty>	No	Specifies the Item/Revision custom attributes saved along with Project / Global Library.

Program code

 Copies the following program code to clipboard.

```
private void
SavingProjectToTeamcenterAsNewRevision(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    var tcGatewayConnectionProvider =
tiaPortal.GetService<TeamcenterConnectionProvider>();
    var tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
    try
```

```

{
    String userName = "your_userName";//User name
    String userRole = "yourUserRole";//Role
    String userGroup = "yourUserGroup";//Group
    String serverURL = "serverURL";//Teamcenter server URL
    String teamcenterInstance =
"teamcenterInstance";//InstanceRevisionDetailsDelegate
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
    userGroup, userRole, serverURL, teamcenterInstance);
    var project = tiaPortal.Projects.Open(new
FileInfo(@"D:\UserData\Documents\Automation\Project30\Project30.ap18")),
    var tcGatewayWorkflowProvider =
project.GetService<TcGatewayWorkflowProvider>();
    ItemDetailsDelegate itemDetailsDelegate = ItemDetailsForCreation;
    void ItemDetailsForCreation(ItemDetails itemDetails)
    {
        itemDetails.ItemId = "...";
        itemDetails.ItemName = "Project30";
        itemDetails.RevisionId = "...";
        itemDetails.TeamcenterItemType = "T4TiaProject";
        itemDetails.Comment = "Project Comments.";
    }
    RevisionDetailsDelegate revisionDetailsDelegate =
RevisionDetailsForCreation;
    void RevisionDetailsForCreation(RevisionDetails revisionDetails)
    {
        revisionDetails.RevisionId = "B";
        revisionDetails.Comment = "Revised project";
    }
    var saveAsNewItemResult =
tcGatewayWorkflowProvider.SaveAsNewItem(connectionInfo,
itemDetailsDelegate, null);

```

```

        //Checkout created dataset to edit in TIA portal
        tcGatewayLockProvider.CheckoutDataset(connectionInfo,
        "000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

        var saveAsNewRevisionResult =
        tcGatewayWorkflowProvider.SaveAsNewRevision(connectionInfo,
        revisionDetailsDelegate, null);

        //Check in dataset after changes to project are complete
        tcGatewayLockProvider.CheckinDataset(connectionInfo,
        "000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

        //Checkout revised project/library
        tcGatewayLockProvider.CheckoutDataset(connectionInfo,
        "000495", "B", DatasetType.T4TiaProjectDataset, "Project30");

        //Check in revised dataset after changes to project are
        complete
        tcGatewayLockProvider.CheckinDataset(connectionInfo,
        "000495", "B", DatasetType.T4TiaProjectDataset, "Project30");
    }

    catch (TcGatewayException)
    {
        // other operations
    }
}

```

1.4.4.13 Downloading Project/Global library from Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to download and open the project and library from Teamcenter. The API returns starter file path to open the project or library in the TIA Portal.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo returned during connection to Teamcenter. The tcGatewayConnectionInfo has passed has to match with the active connection's tcGatewayConnectionInfo .
itemID	System.String	Yes	Specifies the Item ID of the TIA Project / Library.
revisionID	System.String	Yes	Specifies the Revision ID of the TIA Project / Library.
itemType	Siemens.Engineering.TeamcenterGateway.ItemType(Enum)	Yes	Item type of Project/Library in Teamcenter.
localCacheOption	Siemens.Engineering.TeamcenterGateway.LocalCacheOption(Enum)	Yes	Overwrite the contents of TcCache incase a version of the Project / Global Library being downloaded is already available. True means overwrite the content. Incase of False a exception is thrown if there is a version mismatch between TcCache and Teamcenter or if the contents have been modified.

Note

If any of the mandatory parameter is not passed, a Siemens.Engineering exception of type TcGatewayException is thrown.

Program code

 Copies the following program code to clipboard.

```
//Program code: Download project/library in Teamcenter
private void DownloadProjectLibrary(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    // Retrieve TcGatewayConnectionProvider and
    TcGatewayLockProvider from GetService()

    // using tiaPortal instance

    TeamcenterConnectionProvider tcGatewayConnectionProvider=
    tiaPortal.GetService<TeamcenterConnectionProvider>();

    TcGatewayLockProvider tcGatewayLockProvider =
    tiaPortal.GetService<TcGatewayLockProvider>();

    // Define user-specific credentials and connection
    string userName = "userName";
    string userRole = "userRole";
    string userGroup = "userGroup";
    string serverURL = "TeamcenterserverURL";
    string teamcenterInstance = "teamcenterInstance";

    try
    {
        // Connection to the Teamcenter using user's credential

        TcGatewayConnectionInfo connectionInfo =
        tcGatewayConnectionProvider.Connect(userName, password,
        userGroup, userRole, serverURL, teamcenterInstance);

        // Check out the dataset with specific parameters for
        editing in TIA Portal
    }
}
```

```

        tcGatewayLockProvider.CheckoutDataset(connectionInfo
, "000495", "A",
        DatasetType.T4TiaProjectDataset, "Project30");

        // Use TcGatewaySearchAndDownloadProvider to download the
dataset as a file for
        // local processing

        TcGatewaySearchAndDownloadProvider downloadFromTcProvider
= tiaPortal.GetService<TcGatewaySearchAndDownloadProvider>();

        FileInfo TIAObjectStarterFilePath =
downloadFromTcProvider.Download(connectionInfo, "000495", "A",
ItemType.Project, LocalCacheOption.Overwrite);

        var project =
tiaPortal.Projects.Open(TIAObjectStarterFilePath);

        //OR

        var library =
tiaPortal.GlobalLibraries.Open(TIAObjectStarterFilePath,
OpenMode.ReadWrite);

        //User can work on the Project.

        project.DeviceGroups.Create("Group_3");

        project.Save();

    }

    catch(TcGatewayException ex)

    {

        // other operations

    }

}

```

1.4.4.14 Searching Project / Global library from Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A Project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to search project and global library in Teamcenter. The Search API returns a list of TIA Portal objects from Teamcenter with search result containing the details such as ItemID and List of related RevisionIds.

Parameter

Parameter	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the Teamcenter connectionInfo after successful connection.
type	Siemens.Engineering.TeamcenterGateway.ItemType(Enum)	Yes	Specifies the Item type of project / library in Teamcenter.
TIAObjectName	System.String	No	Specifies the name of the TIA Project / Library. It provides a complete name or with wild card characters. [User should provide value in at least one of the fields; Item ID, Item Name, TIAObjectName]
itemID	System.String	No	Specifies the item ID of the TIA Project / Library. It Provides a complete Item Id or with wild card characters. [User should provide value in at least one of the fields; Item ID, Item Name, TIAObjectName]

itemName	System.String	No	Specifies the item name of the TIA Project / Library. Provide a complete Item Name or with wild card characters. [User should provide value in at least one of the fields; Item ID, Item Name, TIAObjectName]
revisionID	System.String	No	Specifies the revision ID of the TIA Project / Library. User shall either enter a complete revision ID or with wild card characters.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if the required mandatory parameters are not passed to search project / library in Teamcenter.

Program code

 Copies the following program code to clipboard.

```
//Program code: Search project/library in Teamcenter
private void SearchProjectInLibraryTeamCenter(SecureString
password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve TcGatewayConnectionProvider from
    GetService()using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
    string userName = "username";
```

```

string userRole = "userrole";
string userGroup = "usergroup";
string serverURL = "teamcenterserver_url";
string teamcenterInstance = "teamcenterInstance";

try
{
    // Connection to the Teamcenter using user's credentials
    TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup, userRole, serverURL, teamcenterInstance);

    // Search for items of type 'Project' starting with 'P*'
    TcGatewaySearchAndDownloadProvider searchAndLoadProvider
= tiaPortal.GetService<TcGatewaySearchAndDownloadProvider>();

    var items = searchAndLoadProvider.Search(connectionInfo,
ItemType.Project, "P*", "", "", "");

    foreach (var item in items)
    {
        var itemId = item.ItemId;
        var revisionIdList = item.RevisionId;
        // Display information about each revision of the item
        foreach (var revision in revisionIdList)
        {
            Console.WriteLine(itemId + "-" + revision);
        }
    }

    catch (TcGatewayException ex)
    {
        // other operations
    }
}

```

1.4.4.15 Checkout Project/Global library dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Opening a project](#)
- A Project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to perform checkout action on TIA Portal project and Global library in Teamcenter dataset.

Parameter

Parameter	Data type	Is Mandatory	Description
tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Returns the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo must be passed to match with the active connection's tcGatewayConnectionInfo .
itemID	System.String	Yes	Specifies the item ID of the TIA Project / Library in Teamcenter
revisionID	System.String	Yes	Specifies the revision ID of the TIA Project / Library in Teamcenter to which the dataset is linked.

datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes	Specifies the Teamcenter dataset type corresponding to either project / library to be checked out. Eg: T4TiaProjectDataset / T4TiaLibraryDataset.
datasetName	System.String	Yes	Specifies the Teamcenter dataset name corresponding to either project / library to be checked out.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if the required mandatory parameters are not passed to perform checkout action on project / library in Teamcenter.

Program code: Checkout dataset in Teamcenter

 Copies the following program code to clipboard.

```
//Program code: Checkout project/global library in Teamcenter
dataset

private void CheckoutProjectInTeamcenterDataset(SecureString
password)
{
    // Refer the Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    // Retrieve TcGatewayConnectionProvider and
TcGatewayLockProvider
    // from GetService()using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
```

```

        TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

        try
        {
            // Define user-specific credentials and connection
details

            string userName = "username";
            string userRole = "userrole";
            string userGroup = "usergroup";
            string serverURL = "serverURL";
            string teamcenterInstance = "teamServerInstance";

            // Connect to the Teamcenter using credentials

            TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
userGroup, userRole, serverURL, teamcenterInstance);

            // Checkout the dataset in Teamcenter

            tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

        }

        catch (TcGatewayException)

        {

            //User can perform saving of Changes of TIA Project to
the checked out dataset

            // or any other operations

        }

    }

```

1.4.4.16 Checkin Project/Global library dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to check in action on TIA Portal project and Global library in Teamcenter dataset.

Parameter

Parameter name	Data type	Is Mandatory	Description
----------------	-----------	--------------	-------------

tcGatewayConnectionInfo	Siemens.Engineering.TeamcenterGateway.TcGatewayConnectionInfo	Yes	Return the tcGatewayConnectionInfo during connection to Teamcenter. The tcGatewayConnectionInfo must be passed to match with the active connection's tcGatewayConnectionInfo .
itemID	System.String	Yes	Specifies the item ID of the TIA Portal Project / Library in Teamcenter
revisionID	System.String	Yes	Specifies the revision ID of the TIA Portal Project / Library in Teamcenter to which the dataset is linked.
datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes	Specifies the Teamcenter dataset type corresponding to either project / library to be checked in. Eg: T4TiaProjectDataset / T4TiaLibraryDataset.
datasetName	System.String	Yes	Specifies the Teamcenter dataset name corresponding to either project / library to be checked in.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if any of the above mandatory parameters is not passed for checkin of project / library to the Teamcenter dataset.

Program code

 Copies the following program code to clipboard.

```
//Program code: Checkin project/global library in Teamcenter
dataset

private void CheckinProject(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();
    {
        // Retrieve TcGatewayConnectionProvider and
        TcGatewayLockProvider
        // from GetService() using tiaPortal instance
        TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();
        TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();
        try
        {
            // Define user specific credentials and connection
            details
            string userName = "username";
            string userRole = "userole";
            string userGroup = "usergroup";
            string serverURL = "teamcenterserver_URL";
            string teamcenterInstance = "teamcenter_instance";
            // Connect to Teamcenter using provided credentials
            TcGatewayConnectionInfo connectionInfo =
            tcGatewayConnectionProvider.Connect(userName, password,
            userGroup, userRole, serverURL, teamcenterInstance);
```

```

        // User can perform save changes of TIA Project to the
checkedout dataset

        // using TcGateway Save APIs.

        tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

        // Checkin to release the lock on dataset

        tcGatewayLockProvider.CheckinDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

    }

    catch (TcGatewayException ex)

    {

        // other operations

    }

}

}

```

1.4.4.17 Cancel checkout dataset in Teamcenter

Requirement

- The TIA Portal Openness is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A Project is open
See [Opening a project](#)

Application

You can use the TIA Portal Openness to cancel checkout action on TIA Portal project and Global library dataset in Teamcenter. You can also use the `CancelCheckoutDataset()` to release the lock on dataset in Teamcenter.

Parameter

Parameter name	Data type	Is Mandatory	Description
tcGatewayConnectio nInfo	Siemens.Engineerin g.TeamcenterGatew ay.TcGatewayConne ctionInfo	Yes	Returns the tcGatewayConnectio nInfo during connection to

			Teamcenter. The tcGatewayConnectionInfo must be passed to match with the active connection's tcGatewayConnectionInfo .
itemID	System.String	Yes	Specifies the item ID of the TIA Project / Library in Teamcenter
revisionID	System.String	Yes	Specifies the revision ID of the TIA Project / Library in Teamcenter to which the dataset is linked.
datasetType	Siemens.Engineering.TeamcenterGateway.DatasetType	Yes	Specifies the Teamcenter dataset type corresponding to either project / library to be checked out. Eg: T4TiaProjectDataset / T4TiaLibraryDataset.
datasetName	System.String	Yes	Specifies the Teamcenter dataset name corresponding to either project / library to be checked out.

Note

A Siemens.Engineering exception of type TcGatewayException is thrown if any of the above mandatory parameters is not passed to cancel checkout action on project / library in Teamcenter.

Program code

 Copies the following program code to clipboard.

```
//Program code: Cancel checkout dataset in Teamcenter
private void CancelCheckoutDataset(SecureString password)
{
    // Refer Connecting to the TIA Portal section
    TiaPortal tiaPortal = new TiaPortal();

    // Retrieve the TcGatewayConnectionProvider and
    TcGatewayLockProvider

    // from GetService() using tiaPortal instance
    TeamcenterConnectionProvider tcGatewayConnectionProvider
= tiaPortal.GetService<TeamcenterConnectionProvider>();

    TcGatewayLockProvider tcGatewayLockProvider =
tiaPortal.GetService<TcGatewayLockProvider>();

    try
    {
        // Define user specific credentials and connection
details
        string userName = "your_username";
        string userRole = "your_userrole";
        string userGroup = "your_usergroup";//Group
        string serverURL = "teamcenterserverURL";
        string teamcenterInstance = "teamcenterInstance";
        // Connect to Teamcenter using provided credentials
        TcGatewayConnectionInfo connectionInfo =
tcGatewayConnectionProvider.Connect(userName, password,
        userGroup, userRole, serverURL, teamcenterInstance);

        //User can perform saving of Changes of TIA Project to
the checked out dataset

        //using TcGateway Save APIs.

        tcGatewayLockProvider.CheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");
```

```

        //Cancel checkout on dataset to release the lock on dataset
        and undo the changes

        //done on dataset

        tcGatewayLockProvider.CancelCheckoutDataset(connectionInfo,
"000495", "A", DatasetType.T4TiaProjectDataset, "Project30");

    }

    catch (TcGatewayException ex)
    {
        // other operations
    }
}

```

1.4.5 Functions for Connections

1.4.5.1 Configurable attributes of a port-to-port connection

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Opening a project](#)
- A project is open.
See [Opening a project](#)

Application

The attributes of a port interconnection are located at the port device item. The read and write access of attributes via TIA Portal Openness is the same as at the UI.

Port interface settings

The following attributes are provided for port interface settings:

Attribute name	Data type	Writeable	Access	Description
MediumAttachmentType	MediumAttachmentType	r/o	Dynamic attribute	-

CableName	CableName	r/w	Dynamic attribute	-
AlternativePartnerPorts	Boolean	r/w	Dynamic attribute	Only available if toolchanger functionality is supported, e.g. at CPU1516.
SignalDelaySelection	SignalDelaySelection	r/w	Dynamic attribute	-
CableLength	CableLength	r/w	Dynamic attribute	-
SignalDelayTime	Double	r/w	Dynamic attribute	-

The following ENUM values are provided for the attribute `MediumAttachmentType`:

Value	Description
<code>MediumAttachmentType.None</code>	Attachment type cannot be determined.
<code>MediumAttachmentType.Copper</code>	Attachment type is copper.
<code>MediumAttachmentType.FibreOptic</code>	Attachment type is fiber optic.

The following ENUM values are provided for the attribute `CableName`:

Value	Description
<code>CableName.None</code>	No cable name is specified
<code>CableName.FO_Standard_Cable_9</code>	FO standard cable GP (9 µm)
<code>CableName.Flexible_FO_Cable_9</code>	Flexible FO cable (9 µm)
<code>CableName.FO_Standard_Cable_GP_50</code>	FO standard cable GP (50 µm)
<code>CableName.FO_Trailing_Cable_GP</code>	FO trailing cable / GP

<code>CableName.FO_Ground_Cable</code>	FO ground cable
<code>CableName.FO_Standard_Cable_62_5</code>	FO standard cable (62.5 μm)
<code>CableName.Flexible_FO_Cable_62_5</code>	Flexible FO cable (62.5 μm)
<code>CableName.POF_Standard_Cable_GP</code>	POF standard cable GP
<code>CableName.POF_Trailing_Cable</code>	POF trailing cable
<code>CableName.PCF_Standard_Cable_GP</code>	PCF standard cable GP
<code>CableName.PCF_Trailing_Cable_GP</code>	PCF trailing cable / GP
<code>CableName.GI_POF_Standard_Cable</code>	GI-POF standard cable
<code>CableName.GI_POF_Trailing_Cable</code>	GI-POF trailing cable
<code>CableName.GI_PCF_Standard_Cable</code>	GI-PCF standard cable
<code>CableName.GI_PCF_Trailing_Cable</code>	GI-PCF trailing cable

The following ENUM values are provided for the attribute `SignalDelaySelection`:

Value	Description
<code>SignalDelaySelection.None</code>	-
<code>SignalDelaySelection.CableLength</code>	<code>CableLength</code> is used to define the signal delay.
<code>SignalDelaySelection.SignalDelayTime</code>	<code>SignalDelayTime</code> is used to define the signal delay.

The following ENUM values are provided for the attribute `CableLength`:

Value	Description
<code>CableLength.None</code>	Cable length is not specified.
<code>CableLength.Length20m</code>	Cable length is 20m.

<code>CableLength.Length50m</code>	Cable length is 50m.
<code>CableLength.Length100m</code>	Cable length is 100m.
<code>CableLength.Length1000m</code>	Cable length is 1000m.
<code>CableLength.Length3000m</code>	Cable length is 3000m.

Port options

The following attributes are provided for port options:

Attribute name	Data type	Write able	Access
<code>PortActivation</code>	bool	r/w	Dynamic attribute
<code>TransmissionRateAndDuplex</code>	<code>TransmissionRateAndDuplex</code>	r/w	Dynamic attribute
<code>PortMonitoring</code>	bool	r/w	Dynamic attribute
<code>TransmissionRateAutoNegotiation</code>	bool	r/w	Dynamic attribute
<code>EndOfDetectionOfAccessibleDevices</code>	bool	r/w	Dynamic attribute
<code>EndOfTopologyDiscovery</code>	bool	r/w	Dynamic attribute
<code>EndOfSyncDomain</code>	bool	r/w	Dynamic attribute

The following ENUM values are provided for the attribute `TransmissionRateAndDuplex`:

Value	Description
<code>TransmissionRateAndDuplex.None</code>	-
<code>TransmissionRateAndDuplexAutomatic</code>	Automatic

TransmissionRateAndDuplex.AUI10Mbps	10 Mbps AUI
TransmissionRateAndDuplex.TP10MbpsHalfDuplex	TP 10 Mbps half duplex
TransmissionRateAndDuplex.TP10MbpsFullDuplex	TP 10 Mbps full duplex
TransmissionRateAndDuplex.AsyncFiber10MbpsHalfDuplex	async fiber 10Mbit/s half duplex mode
TransmissionRateAndDuplex.AsyncFiber10MbpsFullDuplex	async fiber 10Mbit/s full duplex mode
TransmissionRateAndDuplex.TP100MbpsHalfDuplex	TP 100 Mbps half duplex
TransmissionRateAndDuplex.TP100MbpsFullDuplex	TP 100 Mbps full duplex
TransmissionRateAndDuplex.FO100MbpsFullDuplex	FO 100 Mbps full duplex
TransmissionRateAndDuplex.X1000MbpsFullDuplex	X1000 Mbps full Duplex
TransmissionRateAndDuplex.FO1000MbpsFullDuplexLD	FO 1000 Mbps full duplex LD
TransmissionRateAndDuplex.FO1000MbpsFullDuplex	FO 1000 Mbps full Duplex
TransmissionRateAndDuplex.TP1000MbpsFullDuplex	TP 1000 Mbps full duplex
TransmissionRateAndDuplex.FO10000MbpsFullDuplex	FO 10000 Mbps full Duplex
TransmissionRateAndDuplex.FO100MbpsFullDuplexLD	FO 100 Mbps full duplex LD

TransmissionRateAndDuplex.POFPCF100 MbpsFullDuplex	POF/PCF 100 Mbps full duplex
---	------------------------------

See also

[Connecting to the TIA Portal](#)

1.4.6 Functions on libraries

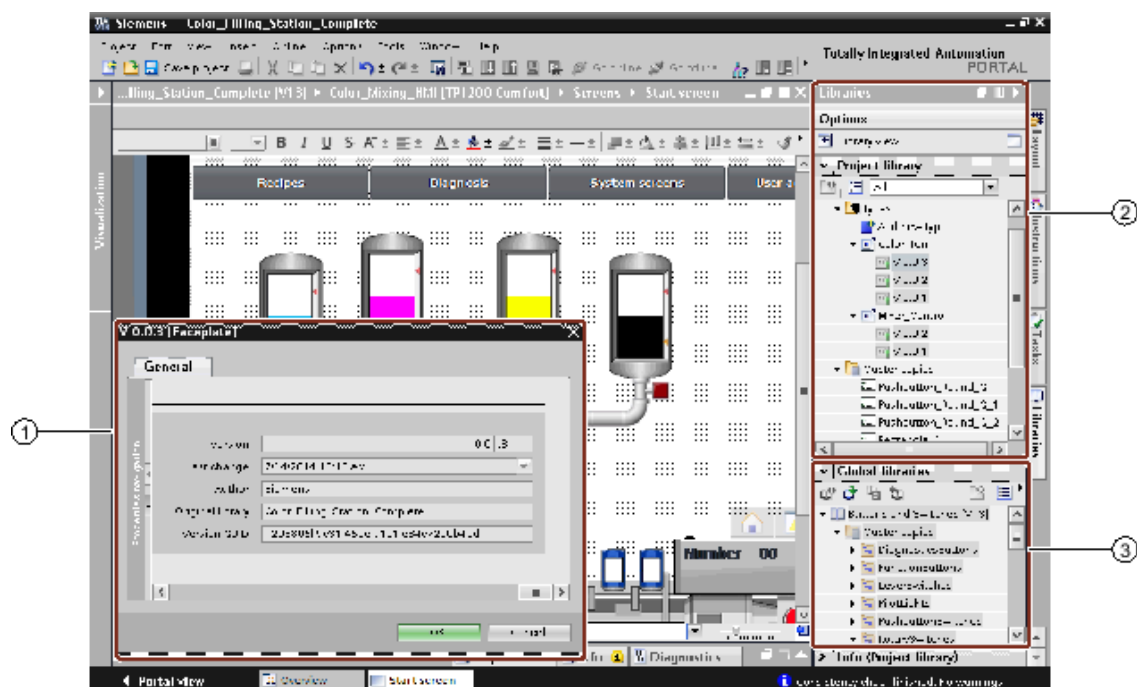
1.4.6.1 Functions for objects and instances

Accessing types and instances

You can use the TIA Portal Openness API interface to access types, type versions and master copies in the project library or global libraries. You can determine connections between type versions and instances. You can also update instances in the project and synchronize changes between a global library and the project library. The TIA Portal Openness API interface also supports the comparison of types versions and instances.

Functions for objects and instances

You have access to the following functions for types, type versions, master copies and instances with the TIA Portal Openness API interface:



- ① Display attributes of types, type versions, master copies and instances
- ② The following functions are available in the project library:

- Update instances of types
 - Instantiating type versions in the project
 - Navigate within the library group
 - Delete groups, types, type versions and master copies
- ③ The following functions are available in the global library:
- Update instances of types
 - Instantiate type version in the project
 - Navigate within the library group

1.4.6.2 Accessing global libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)

Application

Three types of Global Libraries are existing.

- System global library (Siemens.Engineering.Library.SystemGlobalLibrary): These global libraries are included as part of a TIA Portal installation and use the .asx file extension. All system global libraries are read only.
- Corporate global library (Siemens.Engineering.Library.CorporateGlobalLibrary): These global libraries have been chosen by an administrator to be preloaded when TIA Portal is started. All corporate global libraries are read only.
- User global library (Siemens.Engineering.Library.UserGlobalLibrary): These global libraries have been created by users of TIA Portal. User global libraries can be opened either as read only mode or readwrite mode.
If an user global library is already opened in a certain mode then the same user global library cannot be opened with another mode.
User global libraries from previous versions can be opened only in read only mode.

Global Libraries opened using TIA Portal Openness will also be added to the TIA Portal UI's global library collection, and seen in the TIA Portal UI if the UI is present.

Attributes

Attributes of GlobalLibrary:

Attribute	Data type	Description	Access
-----------	-----------	-------------	--------

Author	string	Specifies the author of the global library.	Read
Name	string	Specifies the name of the global library.	Read
IsModified	boolean	True if the contents of the global library has been modified.	Read
IsReadOnly	boolean	Specifies whether the global library is open to read only or not.	Read
Path	FileInfo	Specifies the path of the global library.	Read
LanguageSettings	LanguageSettings	Specifies the global library language settings	Read
Comment	MultilingualText	Specifies the comment of the global library.	Read
IsWriteProtected ¹	boolean	Specifies whether the global library is write protected or not	Read
LastModifiedBy ²	string	Specifies the user who modified the global library last time	Read
Size ³	long	Specifies the size of the global library in KB	Read
CreationTime ⁴	DateTime	Specifies the time at which the library was created	Read

LastModified ⁵	DateTime	Specifies the last time at which the global library was modified	Read
Version ⁶	string	Specifies the version of the global library	Read
Family ⁷	string	Specifies the global library family	Read
Copyright ⁸	string	Specifies the global library copyright information	Read
HistoryEntries ⁹	Siemens.Engineering.HistoryEntryComposition	Specifies the log of global library history events	Read
UsedProducts ¹⁰	Siemens.Engineering.UsedProductComposition	Specifies the composition of used products in the global library	Read

Note

The attribute name mentioned above with superscript is supported starting from TIA Portal Openness v20.

Attributes of GlobalLibraryInfo:

Attribute name	Return type	Description
IsReadOnly	boolean	True if the global library is read only.
IsOpen	boolean	True if the global library is already open.
LibraryType	GlobalLibraryType	Type of the global library: • System: System global library • Corporate: Corporate global library • User: User global library

Name	string	Specifies the name of the global library.
Path	FileInfo	Specifies the path of the global library.

Program code: Access properties of global library

Modify the following program code to access the properties of a global library :

 Copies the following program code to clipboard.

```
private static void
GetLibraryProperties(Siemens.Engineering.Library.GlobalLibrary
globalLibrary)
{
    string author = globalLibrary.Author;
    string name = globalLibrary.Name;
    bool isModified = globalLibrary.IsModified;
    bool isReadOnly = globalLibrary.IsReadOnly;
    bool isWriteProtected = globalLibrary.IsWriteProtected;
    string lastModifiedBy = globalLibrary.LastModifiedBy;
    long size = globalLibrary.Size;
    FileInfo path = globalLibrary.Path;
    DateTime creationTime = globalLibrary.CreationTime;
    DateTime lastModified = globalLibrary.LastModified;
    string version = globalLibrary.Version;
    string family = globalLibrary.Family;
    string copyright = globalLibrary.Copyright;
    Siemens.Engineering.HistoryEntryComposition historyEntries
= globalLibrary.HistoryEntries;
    Siemens.Engineering.LanguageSettings languageSettings =
globalLibrary.LanguageSettings;
    Siemens.Engineering.MultilingualText comment =
globalLibrary.Comment;
    Siemens.Engineering.UsedProductComposition usedProducts =
globalLibrary.UsedProducts;
```

```
}
```

Program code: Available global libraries

Modify the following program code to get informations about all available global libraries:

 [Copies the following program code to clipboard.](#)

```
TiaPortal tiaPortal = ...;

var availableLibraries =
tia.GlobalLibraries.GetGlobalLibraryInfos();

foreach (GlobalLibraryInfo info in availableLibraries)
{
    Console.WriteLine(" Library Name: {0}", info.Name);
    Console.WriteLine(" Library Path: {0}", info.Path);
    Console.WriteLine(" Library Type: {0}", info.LibraryType);
    Console.WriteLine(" Library IsOpen: {0}", info.IsOpen);
}
```

See also

[Accessing folders in a library](#)

1.4.6.3 Accessing library base type

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to access the base type classes of the library type object. These all have a direct base type of Siemens.Engineering.Library.Types.LibraryType

You can have access to the below base type classes:

- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryType

- Siemens.Engineering.Hmi.Screen.ScreenLibraryType
- Siemens.Engineering.Hmi.Screen.StyleLibraryType
- Siemens.Engineering.Hmi.Screen.StyleSheetLibraryType
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryType
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryType
- Siemens.Engineering.SW.Types.PlcTypeLibraryType

Program code

 Copies the following program code to clipboard.

```
TiaPortal tiaPortal = ...;

Project project = tia.Projects.First();
ProjectLibrary library = project.ProjectLibrary;

//Pre-V14SP1

LibraryType libraryType=
library.TypeFolder.Types.Find("VBFunction_1");

//V14SP1

VBScriptLibraryType libraryTypeAsVbScript = libraryType as
VBScriptLibraryType;
```

1.4.6.4 Accessing global library languages

Requirements

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A library is open
See [Opening libraries](#)

Application

You can use language setting navigator to access and manage the Global Library languages.

TIA Portal Openness supports the following access to the global library languages:

- Iterating through supported languages.
- Searching through the collection of supported languages by
`System.Globalization.CultureInfo`.
- Accessing individual languages. Each Language object will contain a single read-only attribute Culture of type `System.Globalization.CultureInfo`.

- Accessing a collection of active languages.
- Searching through the collection of active languages by `System.Globalization.CultureInfo`.
- Adding a language to a collection of active languages.
- Removing a language from a collection of active languages.
- Setting an editing language.
- Setting a reference language.

Attribute of global library languages

Global library languages provides the following attribute:

Attribute name	Data type	Writeable	Description
LanguageSettings	Siemens.Engineering.LanguageSettings	r/o	Handles global library languages

Program code: Enumerating global library language

Modify the following program code to enumerate the global library language:

 Copies the following program code to clipboard.

```

FileInfo m_GlobalLibraryPath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);
var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibraryPath,
OpenMode.ReadOnly);
LanguageSettings languageSettings =
globalLibrary.LanguageSettings;

```

Program code: Accessing language setting

Modify the following program code to access the language setting on global library:

 Copies the following program code to clipboard.

```

FileInfo m_GlobalLibraryPath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);

```

```

var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibrarypath,
OpenMode.ReadOnly);

LanguageComposition languages =
globalLibrary.LanguageSettings.Languages;
foreach (Language language in languages)
{
// Work with this language
}

```

Program code: Setting global library languages

Modify the following program code to set global library languages. If you set a new supported language through TIA Portal Openness, the language will be added to the active languages collection.

 Copies the following program code to clipboard.

```

FileInfo m_GlobalLibrarypath = new FileInfo("bla");
TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface);

var globalLibrary =
portal.GlobalLibraries.Open(m_GlobalLibrarypath,
OpenMode.ReadOnly);

LanguageSettings languageSettings =
globalLibrary.LanguageSettings;

LanguageComposition supportedLanguages =
languageSettings.Languages;

LanguageAssociation activeLanguages =
languageSettings.ActiveLanguages;

Language supportedGermanLanguage =
supportedLanguages.Find(CultureInfo.GetCultureInfo("de-DE"));
activeLanguages.Add(supportedGermanLanguage);

languageSettings.EditingLanguage = supportedGermanLanguage;
languageSettings.ReferenceLanguage = supportedGermanLanguage;

```

Note

Adding and modifying languages in global library language setting does not modify languages of released versions in the global library. This behaviour holds true for updating global library languages through UI (language editor) or LanguageSettings navigator.

1.4.6.5 Opening libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application. This requirement is only for accessing project libraries.
See [Opening a project](#)

Application

A global library can be opened using System.IO.FileInfo with a path to the library file on a local storage medium or a network storage. Only user global libraries can be opened by path. A path obtained from a system global library or a corporate global library can not be used to open it.

As of V14 SP1 global libraries can be opened using the GlobalLibraryInfo. The OpenMode is specified in the GlobalLibraryInfo.

A user global library from a previous version of TIA Portal can be upgraded and opened with the current version of TIA Portal. A global library from V13 or a previous version cannot be opened with upgrade. These libraries have to be upgraded to V13 SP1 first.

Libraries opened using TIA Portal Openness will also be added to the global library collection in the TIA Portal, and will be visible in the user interface of TIA Portal.

Program code: Opening a library using System.IO.FileInfo

Modify the following program code:

 Copies the following program code to clipboard.

```
TiaPortal tia = ...  
FileInfo fileInfo = ....  
UserGlobalLibrary userLib =  
tia.GlobalLibraries.Open(fileInfo, OpenMode.ReadWrite);
```

Program code: Opening a library using GlobalLibraryInfo

Modify the following program code:

 Copies the following program code to clipboard.

```
TiaPortal tia = ...

IList<GlobalLibraryInfo> libraryInfos =
tia.GlobalLibraries.GetGlobalLibraryInfos();

GlobalLibraryInfo libInfo = ...; //check for the info you
need from the list, e.g.

GlobalLibrary libraryOpenedWithInfo;

if (libInfo.Name == "myLibrary")

libraryOpenedWithInfo = tia.GlobalLibraries.Open(libInfo);
```

Program code: Upgrading a library

Modify the following program code:

 Copies the following program code to clipboard.

```
TiaPortal tia = ...

FileInfo fileInfo = .... //library from previous TIA Portal
version

UserGlobalLibrary userLib =
tia.GlobalLibraries.OpenWithUpgrade(fileInfo);
```

OpenMode

Value	Description
ReadOnly	Read access to the library.
ReadWrite	Read and write access to the library.

1.4.6.6 Comparing libraries

Requirement

- The application is connected to the TIA Portal using TIA Portal Openness
See [Connecting to the TIA Portal](#)
- The project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to compare the libraries with each other. For example, you can use the feature to find the differences between versions before updating the new version into the project.

You can use the Openness to do the detailed comparison of selected library objects. It can compare two library objects

- Type object
- Version object
- Master copies

The Openness API allows you to compare two type objects for example version object as well as master copies and returns the detailed content compare result.

Method, Parameter and Enum

The TIA Portal Openness supports the following method with its signature type to compare the library objects :

Method	Return type
DetailedCompareResult CompareTo(LibraryType targetLibraryObject)	Siemens.Engineering.Library.Types.LibraryType
DetailedCompareResult CompareTo(LibraryTypeVersion targetLibraryObject)	Siemens.Engineering.Library.Types.LibraryTypeVersion
DetailedCompareResult CompareTo(MasterCopy targetLibraryObject)	Siemens.Engineering.Library.MasterCopies. MasterCopy

The LibraryCompareResult supports the following navigator in TIA Portal Openness:

Name	Types
Parent	Siemens.Engineering.IEngineeringObject
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

The LibraryCompareResultElement supports the following navigator and attribute name in TIA Portal Openness:

Name	Types
Parent	Siemens.Engineering.Library.Compare.LibraryCompareResultElementComposition
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

Attribute name	Type
ComparisonResult	Siemens.Engineering.Library.Compare.LibraryCompareResultState
DetailedInformation	System.String
LeftName	System.String
RightName	System.String
Left	Siemens.Engineering.IEngineeringObject
Right	Siemens.Engineering.IEngineeringObject

The LibraryCompareResultState ENUM supports the following ENUM items:

Enum Items	Description
ContainerContentsDifferent	Container contents has one or more differences
ContainerContentsIdentical	Container content is identical
ObjectsDifferent	Objects are different
LeftMissing	Left object is missing
RightMissing	Right object is missing
ObjectsIdentical	Objects are identical

The DetailedCompareResult supports the following navigator in TIA Portal Openness:

Name	Type
Parent	Siemens.Engineering.IEngineeringObject
RootElement	Siemens.Engineering.Library.Compare.LibraryCompareResultElement

The DetailedCompareResultElement supports the following navigator and attribute name in TIA Portal Openness:

Name	Type
Parents	Siemens.Engineering.IEngineeringObject

Attribute name	Type	Description
Description	System.String	Detail compare property name
LeftValue	System.Object	Detail compare left property value. The LeftValue property supports the following types, which are System.Guid, System.Boolean, System.String, System.DateTime
RightValue	System.Object	Detail compare right property value. The RightValue property supports the following types, which are System.Guid, System.Boolean, System.String, System.DateTime.
DetailCompareStatus	Siemens.Engineering.Library.Compare.DetailCompareStatus	Returns the comparison result of the property values based on the default criteria filter applied. (Original library property of version object is not compared by default)

The DetailCompareStatus supports the following ENUM items:

Enum Items	Value
------------	-------

ObjectsDifferent	Objects are different
ObjectsIdentical	Objects are identical
NotCompared	When filters are applied, objects are not compared

Program code

 Copies the following program code to clipboard.

```
...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryCompareResult libraryCompareResults =
globalLibrary.CompareToLibrary(projectLibrary);
...
```

Modify the following program code to compare two type objects:

 Copies the following program code to clipboard.

```
...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
LibraryType leftTypeObject =
ProjectLib.TypeFolder.Types.Find(typeName);
LibraryType rightTypeObject =
GlobalLib.TypeFolder.Types.Find(typeName);
DetailedCompareResult detailedCompareResult =
leftTypeObject.CompareTo(rightTypeObject);
...
```

Modify the following program code to compare two versions:

 Copies the following program code to clipboard.

```
...
GlobalLibrary globalLibrary = ...;
ProjectLibrary projectLibrary = Project.ProjectLibrary;
```

```

LibraryTypeVersion leftVersionObject =
ProjectLib.TypeFolder.Types.Find(typeName).Versions[0];

LibraryTypeVersion rightVersionObject =
GlobalLib.TypeFolder.Types.Find(typeName).Versions[0];

DetailedCompareResult detailedCompareResult =
leftVersionObject.CompareTo(rightVersionObject);

...

```

Modify the following program code to compare two master copy objects:

 Copies the following program code to clipboard.

```

...

GlobalLibrary globalLibrary = ...;

ProjectLibrary projectLibrary = Project.ProjectLibrary;

LibraryTypeVersion leftMasterCopy =
ProjectLib.MasterCopyFolder.MasterCopies.Find(masterCopyName);

LibraryTypeVersion rightMasterCopy =
GlobalLib.MasterCopyFolder.MasterCopies.Find(masterCopyName);

DetailedCompareResult detailedCompareResult =
leftMasterCopy.CompareTo(rightMasterCopy);

...

```

1.4.6.7 Enumerating open libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)

Application

All opened global libraries in the TIA Portal, regardless if they have been opened via API or via user interface can be enumerated.

Global Libraries from previous versions of TIA Portal will not be enumerated if they are opened with write access.

Program code

Modify the following program code to enumerate open global libraries.

 Copies the following program code to clipboard.

```
TiaPortal tia = ...
foreach (GlobalLibrary globLib in tia.GlobalLibraries)
{
    ///work with the global library
}
```

See also

[Opening a project](#)

1.4.6.8 Saving and closing libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

See [Connecting to the TIA Portal](#)

- A library is open.

See [Opening libraries](#)

Application

User global libraries can be closed or saved. Any changes made to the global library will not be saved automatically. All unsaved changes will be discarded without prompting by closing a global library.

System global libraries and corporate global library cannot be closed or saved.

To save and close a global library:

- Use the Save () method to save a user global library
- Use the SaveAs () method to save a user global library in a different directory
- Use the Close () method to close a user global library

Feature tokens included for SaveAs () are:

- PublicAPI : Necessary for any Published feature
- DenyIfTransaction : Necessary since SaveAs should not be allowed within a transaction as it cannot be undone.

Program code

Modify the following program code to save a user global library:

 [Copies the following program code to clipboard.](#)

```
UserGlobalLibrary userLib = ...
```

```
// save changes and close library
userLib.Save();
userLib.Close();
```

Modify the following program code to save a user global library in a different location:

 Copies the following program code to clipboard.

```
TiaPortal portal = new TiaPortal(TiaPortalMode.WithUserInterface);
GlobalLibraryComposition globalLibraryComposition =
portal.GlobalLibraries;

//please adapt the path and the extension alx to the installed
version of TIA Portal

FileInfo existingLibraryFileInfo = new
FileInfo(@"D:\GlobalLibraries\MyGlobalLibrary\MyGlobalLibrary.alx");
DirectoryInfo targetDirectoryInfo = new
DirectoryInfo(@"D:\GlobalLibraries\GlobalLibrarySaveAs");

UserGlobalLibrary userGlobalLibrary =
globalLibraryComposition.Open(existingLibraryFileInfo,
OpenMode.ReadWrite);

userGlobalLibrary.SaveAs(targetDirectoryInfo);
```

Note

The persistence of the library is changed after SaveAs operation. Thus, after the SaveAs is performed, the newly saved library is available in libraries card. However, the original library on which SaveAs operation was performed, would not be available. The newly saved library would have the same mode as the original library on which SaveAs was performed. This means if the existing library was opened in ReadOnly mode, then the newly saved library in the target path would also open in the ReadOnly mode. Similar is the case for ReadWrite mode.

Modify the following program code to close a user global library:

 Copies the following program code to clipboard.

```
UserGlobalLibrary userLib = ...
// close and discard changes
userLib.Close();
```

1.4.6.9 Archiving and retrieving a library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A library is open.
See [Opening libraries](#)
- A library is save
See [Saving and closing libraries](#)

Introduction

You can use the TIA Portal Openness to archive the open and save global libraries prior to any further modification to prevent from unintended result so that you can use the TIA Portal Openness to retrieve the archived library. You can also share the archived file across the network easily.

Archiving a library

You can use the TIA Portal Openness API interface to archive a user global library. The API is available on the `Siemens.Engineering.UserGlobalLibrary` object.

The API definition looks like the following:

 Copies the following program code to clipboard.

```
public void Archive(System.IO.DirectoryInfo targetDirectory,
string targetName,
Siemens.Engineering.Library.LibraryArchivationMode
archivationMode)
```

The `targetName` is the name of the file created for archived or non archived. This file may or may not contain any file extensions. If you do not provide any extension or provide an extension apart from "zalx" or "zal14" etc., then the archived file could not be retrieved from TIA Portal outside the Openness API.

For `LibraryArchivationMode` value as `Compressed` and `DiscardRestorableDataAndCompressed`, the archived file name is same as it is provided by you. For `LibraryArchivationMode` value as `None` and `DiscardRestorableData`, the Library file extension is automatically decided by TIA Portal Project Manager component, based on the current version of TIA Portal.

Note

You must have saved the library before calling `Archive` API. In case the library contains any unsaved changes, archive will throw an `EngineeringTargetException`.

Library Archivation Mode

The LibraryArchivationMode enumeration have four values:

LibraryArchivationMode	Description
None	• No special action are taken with the orginial files. Mode is similiar to a "save as" operation. • No compressed zip file is created in this mode. • The difference with SaveAs in this case is that the Archive will not change the persistence location to the new Archived folder whereas SaveAs does that.
DiscardRestorableData	• The file storage stores the library in an internal data file and this file grows whenever a library data modification happens. In the case of DiscardRestorableData mode this data file is reorganized (only latest version of the objects are stored and history is removed from the file) and Intermediate data, the files of the IM directory and tmp directory (see Library Directory structure) are not copied to the archive location. • No compressed zip file is created in this mode.
Compressed	The tmp library folder structure, created by Archiving, is compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.
DiscardRestorableDataAndCompressed	The tmp library folder structure, created by Archiving, discards the restorable data and then compressed into a zip compatible archive. The tmp folder structure is removed after creation of the zip file.

Program code: Archiving a library

Modify the following program code to archive a user global library:

 Copies the following program code to clipboard.

```
var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);

//Please adapt the path and the extension alx to the
installed version of TIA Portal

var libraryFilePath = @"E:\Sample1\Sample1.alx";

var userGlobalLibrary = tiaPortal.GlobalLibraries.Open(new
FileInfo(LibraryFilePath), OpenMode.ReadWrite);
```

```
var archivePath = @"E:\Archive";
var archiveFileName = "SampleArchive";
userGlobalLibrary.Archive(new DirectoryInfo(archivePath),
archiveFileName, LibraryArchivationMode.Compressed);
```

Retrieving a library

You can use the TIA Portal Openness API interface to retrieve an archived TIA Portal library. You can only retrieve a compressed archive. The API is available on the

Siemens.Engineering.GlobalLibraryComposition object.

The API definition looks like the following:

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Library.UserGlobalLibrary
Retrieve(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo targetDirectory,
Siemens.Engineering.OpenMode openMode)
```

Note

You cannot retrieve an archived library with 'LibraryArchivationMode.None' or 'LibraryArchivationMode.DiscardRestorableData' enumeration value.

You can call RetrieveWithUpgrade API for the archived library of a previous TIA Portal version.

The API definition looks like the following:

 Copies the following program code to clipboard.

```
public Siemens.Engineering.Library.UserGlobalLibrary
RetrieveWithUpgrade(System.IO.FileInfo sourcePath,
System.IO.DirectoryInfo
targetDirectory, Siemens.Engineering.OpenMode openMode)
```

Open Mode

The OpenMode enumeration have two values:

OpenMode	Description
ReadMode	Read access to the library. Data can be read from the library.
ReadWrite	Write access to the library. Data can be written to the library.

Program code: Retrieving a library

Modify the following program code to access to retrieve a library:

 Copies the following program code to clipboard.

```
//Please adapt the path and the extension zalx to the
installed version of TIA Portal

var archivePath = @"E:\Archive\Sample1.zalx";
var retrievedLibraryDirectory = @"E:\RetrievedLibraries";
var tiaPortal = new
TiaPortal(TiaPortalMode.WithoutUserInterface);
tiaPortal.GlobalLibraries.Retrieve(new FileInfo(archivePath),
new DirectoryInfo(retrievedLibraryDirectory),
OpenMode.ReadWrite));
```

See also

[Opening libraries](#)

1.4.6.10 Creating global libraries

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

See [Connecting to the TIA Portal](#)

Introduction

Global libraries can be created via TIA Portal Openness API by calling the Create method on the GlobalLibraryComposition. A UserGlobalLibrary will be returned

Parameters for creating global libraries

Parameter	Data Type	Type	Description
Author	String	Mandatory	Author of a global library.
Comment	String	Optional	Comment of a global library.
Name	String	Optional	Name of a global library

TargetDirectory	DirectoryInfo	Mandatory	Directory that will contain global library folder.
-----------------	---------------	-----------	--

GlobalLibraryComposition.Create

Modify the following program code:

 Copies the following program code to clipboard.

```
TiaPortal tia= ...;

DirectoryInfo targetDirectory = new
DirectoryInfo(@"D:\GlobalLibraries");

UserGlobalLibrary globalLibrary =
tia.GlobalLibraries.Create<UserGlobalLibrary>
(targetDirectory, "Library1")
```

According to this example:

- a folder "D:\GlobalLibraries\Library1" will be created
- a global library file "D:\GlobalLibraries\Library1\Library1.alXX" will be created

See also

[Opening a project](#)

1.4.6.11 Accessing folders in a library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)
- You have access to the required library.
See [Accessing global libraries](#)

Introduction

You can use the TIA Portal Openness API interface to access the system folders for types and master copies in a library. You can access types, type versions, master copies and user-defined folders within the system folder.

You can access a user-defined folder at any time using the `Find` method, for example `libTypeUserFolder.Folders.Find("SomeUserFolder");`.

Program code: Accessing system folders

Modify the following program code to access the system folder for types in a library:

 Copies the following program code to clipboard.

```
public static void AccessTypeSystemFolder(ILibrary library)
{
    LibraryTypeSystemFolder libTypeSystemFolder =
library.TypeFolder;
}
```

Modify the following program code to access the system folder for master copies in a library:

 Copies the following program code to clipboard.

```
public static void AccessMasterCopySystemFolder(ILibrary
library)
{
    MasterCopySystemFolder libMasterCopySystemFolder =
library.MasterCopyFolder;
}
```

Program code: Accessing user-defined folders via Find() method

 Copies the following program code to clipboard.

```
...
LibraryTypeUserFolderComposition userFolderComposition = ...
LibraryTypeUserFolder userFolder =
userFolderComposition.Find("Name of user folder");
...
```

Program code: Enumerating user defined folders

Modify the following program code to enumerate user-defined subfolders in a system folder for types:

 Copies the following program code to clipboard.

```
public static void
EnumerateUserFoldersInTypeSystemFolder(ILibrary library)
```

```

{
    // Enumerating user folders in type system folder:
    LibraryTypeSystemFolder libTypeSystemFolder =
library.TypeFolder;

    foreach (LibraryTypeUserFolder libTypeUserFolder in
libTypeSystemFolder.Folders)
    {
        //...
    }
}

```

Modify the following program code to enumerate user-defined subfolders in a system folder for master copies:

 Copies the following program code to clipboard.

```

public static void
EnumerateUserFoldersInMasterCopySystemFolder(ILibrary
library)
{
    // Enumerating user folders in master copy system folder:
    MasterCopySystemFolder libMasterCopySystemFolder =
library.MasterCopyFolder;

    foreach (MasterCopyUserFolder libMasterCopyUserFolder in
libMasterCopySystemFolder.Folders)
    {
        //..
    }
}

```

Modify the following program code to enumerate user-defined subfolders in a user-defined folder for types:

 Copies the following program code to clipboard.

```

public static void
EnumerateAllUserFolders(LibraryTypeUserFolder libUserFolder)
{

```

```

        foreach (LibraryTypeUserFolder libSubUserFolder in
libUserFolder.Folders)
        {
            EnumerateAllUserFolders(libSubUserFolder);
        }
    }
}

```

Modify the following program code to enumerate user-defined subfolders in a user-defined folder for master copies:

 Copies the following program code to clipboard.

```

public static void
EnumerateAllUserFolders (MasterCopyUserFolder libUserFolder)
{
    foreach (MasterCopyUserFolder libSubUserFolder in
libUserFolder.Folders)
    {
        EnumerateAllUserFolders (libSubUserFolder);
    }
}

```

Program code: Creating user-defined folders

Modify the following program code to create a user-defined folder for types:

 Copies the following program code to clipboard.

```

var typeFolderComposition =
ProjectLibrary.TypeFolder.Folders;
var newTypeUserFolder =
typeFolderComposition.Create ("NewTypeUserFolder");

```

Modify the following program code to create a user-defined folder for master copies:

 Copies the following program code to clipboard.

```

var masterCopyFolderComposition =
projectProjectLibrary.MasterCopyFolder.Folders;
MasterCopyUserFolder newMasterCopyUserFolder =
masterCopyFolderComposition.Create ("NewMasterCopyUserFolder");

```

Program code: Renaming user-defined folders

Modify the following program code to create a user-defined folder for types:

 Copies the following program code to clipboard.

```
var typeUserFolder =  
project.ProjectLibrary.TypeFolder.Folders.Find("SampleTypeUserFolderName");  
typeUserFolder.Name = "NewTypeUserFolderName";
```

 Copies the following program code to clipboard.

```
var typeUserFolder =  
ProjectLibrary.TypeFolder.Folders.Find("SampleTypeUserFolderName");  
typeUserFolder.SetAttributes(new[] {new KeyValuePair<string,object>  
("Name", "NewTypeUserFolderName")});
```

Modify the following program code to create a user-defined folder for master copies:

 Copies the following program code to clipboard.

```
var masterCopyUserFolder =  
project.ProjectLibrary.MasterCopyFolder.Folders.Find("SampleMasterCopyUserFolderName");  
masterCopyUserFolder.Name = "NewMasterCopyUserFolderName";
```

 Copies the following program code to clipboard.

```
var masterCopyUserFolder =  
ProjectLibrary.MasterCopyFolder.Folders.Find("SampleMasterCopyUserFolderName");  
masterCopyUserFolder.SetAttributes(new[] {new KeyValuePair<string,object>  
("Name", "NewMasterCopyUserFolderName")});
```

See also

[Accessing master copies](#)

1.4.6.12 Accessing types

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)
- You have access to the required library.
See [Accessing global libraries](#).
- You have access to a group for types.
See [Accessing folders in a library](#).

Application

You can access the types contained in a library via the TIA Portal Openness API interface.

- You can enumerate the types.
- You can rename types.
- You can access the following attributes for every type:

Attribute	Data type	Description
Author	String	Returns the name of the author.
Comment	MultilingualText	Returns the comment.
Guid	Guid	Returns the GUID of the type. ¹
Name	String	Returns the name of the type. ²
¹ You can find an individual type in a library using this attribute. The search is recursive. ² You can find an individual type in a folder using this attribute. Subfolders are not included in the search. A type name is not unique. There can be several types with the same name in different groups. However, the type Guid is unique.		

Subclasses for library type objects

With TIA Portal Openness API you can access library type objects via sub-classes. The following subclasses are existing:

- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryType

- Siemens.Engineering.Hmi.Faceplate.FaceplateLibraryTypeVersion
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.VBScriptLibraryTypeVersion
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryType
- Siemens.Engineering.Hmi.RuntimeScripting.CScriptLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.ScreenLibraryType
- Siemens.Engineering.Hmi.Screen.ScreenLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.StyleLibraryType
- Siemens.Engineering.Hmi.Screen.StyleLibraryTypeVersion
- Siemens.Engineering.Hmi.Screen.StyleSheetLibraryTypeVersion
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryType
- Siemens.Engineering.Hmi.Tag.HmiUdtLibraryTypeVersion
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryType
- Siemens.Engineering.SW.Blocks.CodeBlockLibraryTypeVersion
- Siemens.Engineering.SW.Types.PlcTypeLibraryType
- Siemens.Engineering.SW.Types.PlcTypeLibraryTypeVersion

The following code is an example of how to use the library type sub-classes

 Copies the following program code to clipboard.

```
ProjectLibrary library = project.ProjectLibrary;
VBScriptLibraryType vbScriptType = ...;
VBScriptLibraryType libraryTypeAsVbScript = libraryType as
VBScriptLibraryType;
VBScriptLibraryTypeVersion libraryTypeVersionAsVbScript =
libraryTypeVersion as VBScriptLibraryTypeVersion
```

Program code

Modify the following program code to enumerate all types in the system folder of a library:

 Copies the following program code to clipboard.

```
public static void
EnumerateTypesInTypesSystemFolder(LibraryTypeSystemFolder
libraryTypeSystemFolder)
{
```

```

        foreach (LibraryType libraryType in
libraryTypeSystemFolder.Types)
        {
            //...
        }
    }
}

```

Modify the following program code to enumerate all types in a user-defined folder of a library:

 Copies the following program code to clipboard.

```

public static void EnumerateTypesInTypesUserFolder
(LibraryTypeUserFolder libraryTypeUserGroup)
{
    foreach (LibraryType libraryType in
libraryTypeUserGroup.Types)
    {
        //...
    }
}

```

Modify the following program code to access the attributes of a type:

 Copies the following program code to clipboard.

```

public static void InspectPropertiesOfType (LibraryType
libTypeObject)
{
    string typeAuthor = libTypeObject.Author;
    MultilingualText typeComment = libTypeObject.Comment;
    string typeName = libTypeObject.Name;
    Guid typeGUID = libTypeObject.Guid;
}

```

Modify the following program code to find an individual type by its name or GUID:

 Copies the following program code to clipboard.

```

public static void FindTypeObjectInLibrary(ILibrary library)
{

```

```

// Find type object by its GUID in a given library:
System.Guid targetGuid = ...;
LibraryType libTypeByGUID = library.FindType(targetGuid);
// Find type object by its name in a given group:
LibraryTypeFolder libTypeSystemFolder =
library.TypeFolder;
LibraryType libTypeByName =
libTypeSystemFolder.Types.Find("myTypeObject");
}

```

Modify the following program code to rename a type:

 Copies the following program code to clipboard.

```

// Setting the name attribute
var type =
project.ProjectLibrary.TypeFolder.Types.Find("SampleTypeName");
type.Name = "NewTypeName";
//Setting the name attribute dynamically
var type =
project.ProjectLibrary.TypeFolder.Types.Find("SampleTypeName");
type.SetAttributes(new[] {new KeyValuePair<string,object>
("Name", "NewTypeName")});

```

1.4.6.13 Accessing type versions

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)
- You have access to the required library.
See [Accessing global libraries](#).
- You have access to a group for types.
See [Accessing folders in a library](#).

Introduction

You can use the TIA Portal Openness to perform the following functionalities:

- Access the type versions
- Enumerate the type versions of a type
- Determine the type to which a type version belongs
- Enumerate the instances of a type version.
- Create a new instance of a type version.
- Navigate from an instance to its connected version object.

As of TIA Portal Openness v20, the `Siemens.Engineering.Library.Types` namespace is added to access the attributes of a library type version.

The `LibraryTypeVersion` serves as a base class for the following classes in WinCC and STEP7:

Wincc	STEP7
• <code>FaceplateLibraryTypeVersion</code> • <code>CScriptLibraryTypeVersion</code> • <code>VBScriptLibraryTypeVersion</code> • <code>ScreenLibraryTypeVersion</code> • <code>StyleLibraryTypeVersion</code> • <code>StyleSheetLibraryTypeVersion</code> • <code>HmiUdtLibraryTypeVersion</code>	• <code>CodeBlockLibraryTypeVersion</code> • <code>PlcTypeLibraryTypeVersion</code>

You can access the following attribute for every type version:

Attribute	Data type	Description	Access
Author	String	Returns the name of the author.	Read
Comment	MultilingualText	Returns the comment.	Read
Guid	Guid	Returns the GUID of the type version. ¹	Read
ModifiedDate	DateTime	Returns the date and time at which the type version was set to the "Committed" status.	Read

		• Committed: Corresponds to the status "Released".	Read
TypeObject	LibraryType	Returns the type to which this type version belongs.	Read
VersionNumber	Version	Returns the version number as a three digit version identification, for example, "1.0.0". ²	Read
Dependencies	Siemens.Engineering.Library.Types.LibraryTypeVersionAssociation	Returns all versions that this version depends on	Read
Dependents	Siemens.Engineering.Library.Types.LibraryTypeVersionAssociation	Returns all versions that depend on this version	Read
MasterCopiesContainingInstances	Siemens.Engineering.Library.MasterCopies.MasterCopyAssociation	Gets the master copies that contain instances of this version	Read
IsDefault	bool	True if the version is a default version, otherwise false.	Read

OriginalLibrary ³	string	The name of the project or library from which the current version of the type was generated	Read
¹ You can find an individual type version in a library using this attribute. ² You can find an individual type version in a "LibraryTypeVersion" composition using this attribute. ³ The OriginalLibrary is added in TIA Portal Openness v20			

Program code: Accessing attribute of type versions

To access attributes of a library type versions modify the following program code:

 Copies the following program code to clipboard.

```
private static void
GetLibraryTypeVersionProperties (LibraryTypeVersion
typeVersion)
{
    string author = typeVersion.Author;
    MultilingualText comment = typeVersion.Comment;
    Guid guid = typeVersion.Guid;
    DateTime modifiedDate = typeVersion.ModifiedDate;
    Siemens.Engineering.Library.Types.LibraryTypeVersionState
state = typeVersion.State;
    Siemens.Engineering.Library.Types.LibraryType typeObject
= typeVersion.TypeObject;
    Version versionNumber = typeVersion.VersionNumber;
    LibraryTypeVersionAssociation dependencies =
typeVersion.Dependencies;
    LibraryTypeVersionAssociation dependents =
typeVersion.Dependents;
    MasterCopyAssociation masterCopiesContainingInstances =
typeVersion.MasterCopiesContainingInstances;
    bool isDefault = typeVersion.IsDefault;
```

```
        string originalLibrary = typeVersion.OriginalLibrary;
    }
```

Program code: Enumerate all type versions of a type

Modify the following program code:

 Copies the following program code to clipboard.

```
//Enumerate the type versions of a type
public static void EnumerateVersionsInType(LibraryType
libraryType)
{
    foreach (LibraryTypeVersion libraryTypeVersion in
libraryType.Versions)
    {
        //...
    }
}
```

Program code: Accessing the attributes of a type version

Modify the following program code:

 Copies the following program code to clipboard.

```
//Accessing the attributes of a type version
public static void
InspectPropertiesOfVersion(LibraryTypeVersion libTypeVersion)
{
    string versionAuthor = libTypeVersion.Author;
    MultilingualText versionComment = libTypeVersion.Comment;
    Guid versionGUID = libTypeVersion.Guid; DateTime
versionModifiedDate = libTypeVersion.ModifiedDate;
    LibraryTypeVersionState versionStateLibrary =
libTypeVersion.State;
    LibraryType versionParentObject =
libTypeVersion.TypeObject;
    Version versionNumber = libTypeVersion.VersionNumber;
```

```
}
```

Program code: Creating an instance of a type version

You can create a new instance of a type version. The following objects are supported:

- Blocks (FB/FC)
- PLC user data types
- Screens
- VB scripts
- NVT

An instance can be created of a type version from global library and project library. When you create the instance of a type version from a global library, the type version is first synchronized with the project library.

With TIA Portal Openness v20, the `CreateFrom()` method adds a new parameter `updatePathsMode` for the `PlcBlock` and `PLCTypes` to manage conflicts during library type instantiation.

The `UpdatePathsMode` (`Siemens.Engineering.Library.Types.UpdatePathsMode`) accepts the following enum value definitions:

Enum value	Description
<code>UpdatePathsMode.UpdatePathsInTarget</code>	If a path conflict occurs, adjust the target path to match the source path
If a path conflict occurs, adjust the target path to match the source path	If a path conflict occurs, keep the target path the same
<code>UpdatePathsMode.ThrowIfPathsConflict</code>	If a path conflict occurs, cancel the operation

A recoverable Exception will be thrown if an instance cannot be created in the target, then the possible reasons are:

- The library type version is in-work
- An instance of the library type version already exists in the target device

Modify the following program code:

 Copies the following program code to clipboard.

```
VBScriptLibraryTypeVersion scriptVersion = ...;  
VBScriptComposition vbscripts = ...;
```

```
//Using the CreateFrom method to create an instance of the
version in the VBScripts composition

VBScript newScript = vbscripts.CreateFrom(scriptVersion);
```

Modify the following program code:

 Copies the following program code to clipboard.

```
ScreenLibraryTypeVersion screenVersion = ...;

ScreenComposition screens = ...;

//Using the CreateFrom method to create an instance of the
version in the screens composition

Screen newScreen = screens.CreateFrom(screenVersion);
```

Modify the following program code:

 Copies the following program code to clipboard.

```
CodeBlockLibraryTypeVersion blockVersion = ...;

PlcBlockComposition blocks = ...;

//Using the CreateFrom method to create an instance of
//the version in the blocks composition

PlcBlock newBlock =
blocks.CreateFrom(blockVersion, Library.Types.UpdatePathsMode.UpdatePathsMode);
```

Modify the following program code:

 Copies the following program code to clipboard.

```
PlcTypeLibraryTypeVersion plcTypVersion=...;

PlcTypeComposition types=...;

//Using the CreateFrom method to create an instance of the version in the
composition

PlcType newType =
types.CreateFrom(plcTypeVersion, Library.Types.UpdatePathsMode.UpdatePathsMode);
```

Modify the following program code to create NVT file:

 Copies the following program code to clipboard.

```
var nvtFromLibraryType =
MyProject.ProjectLibrary.TypeFolder.Types.Find("NVT_TYPE_LIBRARY_1.nvt");

var nvt_TYPE_LIBRARY_1V1 = nvtFromLibraryType.Versions[0];
```

```
var resultPlcDocument =
unit.TypeGroup.Documents.CreateFrom(nvt_TYPE_LIBRARY_1V1 as
PlcDocumentLibraryTypeVersion);
```

Program code: Determining uses of a type version

The following uses are distinguished for type versions:

- The type version uses other type versions from the library.

Example: A user data type is used in a program block. The program block must have access to the user data type. This means the program block depends on the user data type.

When you access the Dependencies attribute of CodeBlockLibraryVersion through `GetDependencies()` method, a list of LibraryTypeVersions are returned.

- The type is being used by another type version in the library.

Example: A user data type is used in a program block. The program block must have access to the user data type. The user data type has the associated program block. The program block depends on the user data type.

When you access the Dependents attribute of PlcTypeLibraryTypeVersion through `GetDependents()` method, a list of LibraryTypeVersions are returned.

Both attributes return a list that contains objects of the `LibraryTypeVersion` type. If there are no uses, an empty list is returned.

Note

If you use these attributes on type versions with the "InWork" status, an exception can be thrown.

Modify the following program code:

 Copies the following program code to clipboard.

```
//Determine the uses of a type version in a library
public static void
GetDependenciesAndDependentsOfAVersion(LibraryTypeVersion
libTypeVersion)
{
    IList<LibraryTypeVersion> versionDependents =
libTypeVersion.Dependents();

    IList<LibraryTypeVersion> versionDependencies =
libTypeVersion.Dependencies();
}
```

Program code

Modify the following program code to determine the type to which a type version belongs:

 Copies the following program code to clipboard.

```
public static void GetParentTypeOfVersion(LibraryTypeVersion
libTypeVersion)
{
    LibraryType parentType = libTypeVersion.TypeObject;
}
```

Modify the following program code to determine the master copies that contain instances of a type version:

 Copies the following program code to clipboard.

```
public static void
GetMasterCopiesContainingInstances(LibraryTypeVersion
libTypeVersion)
{
    MasterCopyAssociation masterCopies =
libTypeVersion.MasterCopiesContainingInstances;
}
```

Modify the following program code to find an individual type version by its version number:

 Copies the following program code to clipboard.

```
public static void FindVersionInLibrary(ILibrary library,
Guid versionGUID)
{
    LibraryTypeVersion libTypeVersionByVersionNumber =
library.FindVersion(versionGUID);
}
```

1.4.6.14 Accessing blocks located in library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See [Connecting to the TIA Portal](#)

- Opening a project
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to get version information from a library object without instantiating and compiling the object.

The following new Export Action will be provided on the LibraryTypeVersion engineering object to export the version content.

The action will create the Exported xml file at given exportFileInfo.

 Copies the following program code to clipboard.

```
void Export(''FileInfo'' exportFileInfo, ''ExportOptions''
exportOptions)
```

Exception will occur:

- In case of the user exceptions stating "The version data cannot be exported as it in in-work state" and the version to be exported is in "in test" state
- In case of the General user DataExchange for example "FileAlreadyExists" exception stating "The export cannot be made because the file 'D:*.xml' already exists".

Library Type Version engineering Object

The following additions are made to the Library TypeVersion engineering Object:

1. Export action is provided on LibraryTypeVersion
2. Navigator to navigate to the version content browsable:
 - Navigator name : ContentObject
 - ReadPublicationLevel : System
 - Relation name: Engineering.Library.DefaultVersionContentObject
 - Base navigator will provide a default implementation to provide empty browsable collection as version content. Client can override this navigator and provide versioncontent.
3. "LibraryTypeName" and "LibraryTypeGuid" on LibraryTypeVersion object

Attributes	ReadPublicationLevel
LibraryTypeName	System
LibraryTypeGuid	System

Note

This is necessary because without this info you would not be able to understand that exported version info (in the exported xml file) belongs to which type.

Exported Content

Following content are available in the exported file:

- Library Version Info exported

Attributes	SimaticMLAAccess
Author	ReadWrite
Guid	ReadOnly
Modified Data	ReadOnly
VersionNumber	ReadWrite
LibraryTypeName	ReadOnly
Library TypeGuid	ReadOnly

Navigators Exported:

- Comment

Note

Attributes with SimaticMLAAccess readonly get exported only if you choose the exportoption as "ExportOptions.WithReadOnly".

Program code

Modify the following program code to export version content information:

 Copies the following program code to clipboard.

```
Guid g = new Guid("35ad9996-d6d7-40c9-989f-0f0c7e21a7b2");  
//Block1  
var version = m_Project.ProjectLibrary.FindType(g).Versions[0];  
version.Export(new  
FileInfo(@"D:\ExportCodeBlock.xml"), ExportOptions.WithReadOnly);
```

See also

Opening a project

1.4.6.15 Accessing instances

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)
- You have access to the required library.
See [Accessing global libraries](#).
- You have access to a group for types.
See [Accessing folders in a library](#).

Application

You can access instances of type versions via the TIA Portal Openness API interface.

Use the `FindInstances(IInstanceSearchScope searchScope)` method to find all instances of a type version.

You can use the `searchScope` parameter to specify the area of the project to be searched. The following classes implement the `IInstanceSearchScope` interface and can be used to search for instances:

- `PlcSoftware`
- `HmiTarget`

The method returns a list that contains objects of the `LibraryTypeInfo` type. If there are no instances, an empty list is returned.

Note

Instances of faceplates and HMI user data types are always linked to the associated type version.

Instances of all other objects, such as program blocks or screens, can be linked to a type version.

Enumerate the instances of a type version

Modify the following program code:

```
Copies the following program code to clipboard.  
  
//Enumerate the instances of a type version in the project
```

```

LibraryTypeVersion version = ...;
PlcSoftware plcSoftware = ...;
IInstanceSearchScope searchScope = plcSoftware as
IInstanceSearchScope;
if (searchScope==null)
{
    //No search possible
}

IList<LibraryTypeInfo> instanceInfos =
version.FindInstances (searchScope);

IEnumerable<IEngineeringObject> instances =
instanceInfos.Select (instanceInfo =>
instanceInfo.LibraryTypeInfo);

```

Navigate from an instance to its connected version object

Use the LibraryTypeVersion attribute of LibraryTypeInfo service to navigate from an instance to its connected version object.

The following objects provide the LibraryTypeInfo service:

- Blocks FB
- Blocks FC
- PLC user data types
- Screens
- VB scripts

If an instance object is not connected to a version object, then it will not provide the "LibraryTypeInfo" service.



Copies the following program code to clipboard.

```

FC fc = ...;

//Using LibraryTypeInfo service
LibraryTypeInfo instanceInfo =
fc.GetService<LibraryTypeInfo>();
if (instanceInfo != null)
{
    LibraryTypeVersion connectedVersion =
instanceInfo.LibraryTypeVersion;
}

```

```

    FC parentFc = instanceInfo.LibraryTypeInstance as FC;
    //parentFc == fc
}

```

1.4.6.16 Accessing master copies

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)
- You have access to the required library.
See [Accessing global libraries](#)
- You have access to a group for master copies.
See [Accessing folders in a library](#)

Introduction

The TIA Portal Openness API interface supports access to master copies in a global library and the project library:

- Creating master copies
- Enumerating master copies in system folders and user-defined folders
- Renaming master copies
- Querying information from master copies
- Querying information from objects in a master copy

Attribute	Data type	Description
Author	String	Returns the name of the author.
ContentDescriptions	MasterCopyContentDescriptionComposition	Returns a description for the content of the MasterCopy.
CreationDate	DateTime	Returns the creation date.
Name	String	Returns the name of the master copy.

Program code

Modify the following program code to enumerate all master copies in the system folder of a library:

 Copies the following program code to clipboard.

```
public static void EnumerateMasterCopiesInSystemFolder
(MasterCopySystemFolder masterCopySystemFolder)
{
    foreach (MasterCopy masterCopy in
masterCopySystemFolder.MasterCopies)
    {
        //...
    }
}
```

Modify the following program code to access an individual mastercopy by using the find method:

 Copies the following program code to clipboard.

```
...

    MasterCopySystemFolder systemFolder =
projectLibrary.MasterCopyFolder;

    MasterCopyComposition mastercopies =
systemFolder.MasterCopies;

    MasterCopy masterCopy = mastercopies.Find("Copy of ...");
...

```

Modify the following program code to enumerate master copy groups and subgroups:

 Copies the following program code to clipboard.

```
private static void EnumerateFolder(MasterCopyFolder folder)
{
    EnumerateMasterCopies(folder.MasterCopies);

    foreach (MasterCopyUserFolder subFolder in
folder.Folders)
    {
        EnumerateFolder(subFolder); // recursion
    }
}
```

```

}

private static void
EnumerateMasterCopies (MasterCopyComposition masterCopies)
{
    foreach (MasterCopy masterCopy in masterCopies)
    {
        ...
    }
}

```

Modify the following program code to access a MasterCopyUserFolder by using the find method:

 Copies the following program code to clipboard.

```

...

MasterCopyUserFolderComposition userFolderComposition =
...

MasterCopyUserFolder userFolder =
userFolderComposition.Find("Name of user folder");

...

```

Modify the following program code to rename a master copy:

 Copies the following program code to clipboard.

```

//Setting the name attribute
var masterCopy =
projectLibrary.MasterCopyFolder.MasterCopies.Find("SampleMasterCopyName");
masterCopy.Name = "NewMasterCopyName";

//Setting the name attribute dynamically
var masterCopy =
projectLibrary.MasterCopyFolder.MasterCopies.Find("SampleMasterCopyName");
masterCopy.SetAttributes(new[] {new KeyValuePair<string,object>("Name",
"NewMasterCopyName")});

```

Querying information from master copies

Modify the following program code to get information of a master copy:

 Copies the following program code to clipboard.

```
public static void GetMasterCopyInformation(MasterCopy
masterCopy)
{
    string author = masterCopy.Author;
    DateTime creationDate = masterCopy.CreationDate;
    string name = masterCopy.Name;
}
```

Querying information from objects in a master copy

The MasterCopy object contains a navigator called ContentDescriptions, which is a composition of MasterCopyContentDescriptions.

A master copy may contain multiple objects. The MasterCopy object contains ContentDescriptions for each object directly contained in the MasterCopy. If the master copy contains a folder which also contains some items, the MasterCopy object only contains one ContentDescription of the folder.

 Copies the following program code to clipboard.

```
MasterCopy multiObjectMasterCopy = ...;
//Using ContentDescriptions
MasterCopyContentDescriptionComposition
masterCopyContentDescriptions =
multiObjectMasterCopy.ContentDescriptions;
MasterCopyContentDescription contentDescription=
masterCopyContentDescriptions.First();
string name = contentDescription.ContentName;
Type type = contentDescription.ContentType;
```

1.4.6.17 Create master copy from a project in library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Application

If the library is a read-write library, you can create a MasterCopy of an IMasterCopySource at target location.

 Copies the following program code to clipboard.

```
MasterCopy
```

```
MasterCopyComposition.Create(Siemens.Engineering.Library.MasterCopies.IMasterC  
sourceObject);
```

An EngineeringException will be thrown if:

- The target location is read-only
- Creation of the MasterCopy from the source is rejected by the system

The following items are defined as IMasterCopySources:

- Device - HW
- DeviceItem - HW
- DeviceUserGroup - HW
- CodeBlock - SW
- DataBlock - SW
- PlcBlockUserGroup - SW
- PlcTag - SW
- PlcTagTable - SW
- PlcTagTableUserGroup - SW
- PlcType - SW
- PlcTypeUserGroup - SW
- PlcUnit - SW
- VBScript - HMI
- VBScriptUserFolder - HMI
- Screen - HMI
- ScreenTemplate - HMI
- ScreenTemplateUserFolder - HMI
- ScreenUserFolder - HMI
- Tag - HMI
- TagTable - HMI
- TagUserFolder - HMI

Program code

Use the following program code:

 Copies the following program code to clipboard.

```
// create a master copy from a code block in the project
library

public static void Create(Project project, PlcSoftware
plcSoftware)

{

    MasterCopySystemFolder masterCopyFolder =
project.ProjectLibrary.MasterCopyFolder;

    CodeBlock block =
plcSoftware.BlockGroup.Groups[0].Blocks.Find("Block_1") as
CodeBlock;

    MasterCopy masterCopy =
masterCopyFolder.MasterCopies.Create(block);

}
```

1.4.6.18 Cleaning up library

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- You have opened a project with your TIA Portal Openness application.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to remove unused version of the specified types in library, or all the types within a specified folder in a library.

The CleanUpLibrary operation is available for both project and user global libraries for which libraries must be read-write mode.

Method

The CleanUpLibrary method is an overloaded function that supports different argument values for both project-specific and global libraries.

	FolderSelection> selectedTypesAndFolders, CleanupLibraryMode cleanupLibraryMode)
User global library	void CleanUpLibrary(IEnumerable<ILibraryTypeOr FolderSelection> selectedTypesAndFolders)

In project library, the CleanUpLibrary() cleanup the version based on two CleanupLibraryMode flags:

CleanupLibraryMode- flags	Description
CleanupLibraryMode.DeleteUnusedTypes	Deletes the type if the none of its versions are used in the project.
CleanupLibraryMode.PreserveDefaultVersion OfUnusedTypes	The highest version or default version of type will be exempted from deletion. Types will not be deleted completely

In global library, the CleanUpLibrary() by default performs cleanup of Global Library without deleting the complete type.

Criteria for version cleanup

A version is eligible for cleanup if it meets the following criteria:

Mode	Criteria
General	The version is not in use (exception to this rule if version is being used by a version that is also in the selection to be cleaned up AND is eligible for cleanup)
	The version is not in-test
	The version is not the base version for an in-test version
	There doesn't exist an instance of the version in the project or master copy folder
PreserveDefaultVersionOfUnusedTypes mode	The version meets the general criteria

PreserveDefaultVersionOfUnusedTypes mode	The version is not highest version or default version of type
DeleteUnusedTypes mode	The version meets the general criteria

Program code

To use cleanup action for a subset of types and folders in Project Library modify the following program code:

 Copies the following program code to clipboard.

